

Digital Contracting Service

Technisches Handbuch

Stand 2026-07-31

DCS Technisches Handbuch

Dieses Handbuch erklärt den Digital Contracting Service (DCS) auf Systemebene: Aufbau, Abläufe, Schnittstellen und Betriebsverhalten. Zielgruppe sind Entwickler, Betreiber und Integratoren, die das System verstehen, betreiben oder anbinden wollen. Quelle der Wahrheit ist der Code.

Inhaltsverzeichnis

Kapitel	Inhalt
01 Systemüberblick	Was das DCS ist, Einordnung in Gaia-X/XFSC/FACIS, Kernfähigkeiten
02 Architektur	Komponentenlandkarte, Kommunikationsbeziehungen, Gesamtdiagramm
03 Vorlagen und Vertragsinhalte	Template-Lifecycle, Semantic Hub (JSON-LD/SHACL/ODRL), Klausel-Katalog
04 Vertragslebenszyklus	State-Machine, Rollen/Tasks/RBAC, Verhandlung, lokal vs. grenzüberschreitend
05 Identität und Authentifizierung	Endnutzer (OID4VP/SD-JWT), Maschinen-Identitäten (Hydra), Instanz (did:web + HSM), Issuer-Vertrauen
06 Signaturen	Wallet-Ceremony, AES/QES, PAdES/JAdES, DSS, TSA, PID
07 Dokumente und Provenance	pdf-core, deterministisches Rendering, C2PA, IPFS, Verschlüsselung at rest
08 Föderation	Instanz-zu-Instanz-Austausch, Trust-Modell, Federated Catalogue
09 Audit und Compliance	Audit-Trail, Merkle-Checkpoints, Workflow-Gates, Monitoring, ODRL/OPA, Reports
10 Betrieb und Monitoring	Betriebsverhalten im Normal- und Fehlerfall: Probes, Hintergrundprozesse, Signale, Incidents
Anhang A Schnittstellenreferenz	API-Gruppen, Events, well-known-Endpunkte
Anhang B Entscheidungsarchiv	Verweisliste auf die Architecture Decision Records (ADRs)

Abgrenzung zu den übrigen Dokumenten

Dieses Handbuch erklärt. Es enthält bewusst keine Befehle, keine Wertetabellen und keine Schritt-für-Schritt-Prozeduren.

Frage	Dokument
Wie ist das System gebaut, wie läuft ein Vorgang durch, was bedeutet ein Signal?	Dieses Handbuch
Wie installiere, konfiguriere, aktualisiere und deinstalliere ich eine Instanz?	<code>docs/deployment-guide.md</code>
Wie sichere und stelle ich Daten wieder her?	<code>docs/backup-integration-guide.md</code>
Wie bediene ich das System als Fachanwender?	<code>docs/user-manual/</code>
Warum wurde etwas so entschieden?	Die ADRs unter <code>docs/</code> , siehe Anhang B

Konkrete Werte, Befehle und Konfigurationsreferenzen stehen an genau einer Stelle, nämlich im Deployment-Leitfaden. Wo dieses Handbuch einen solchen Wert bräuchte, verweist es dorthin.

Leseempfehlung

- **Neu im Projekt:** Kapitel 01 und 02, danach Kapitel 04 mit dem zentralen Ablauf des Systems.
- **Betreiber:** Kapitel 02, dann Kapitel 10; Kapitel 09 für Audit-Trail und Incident-Sicht. Ausführbare Verfahren stehen im Deployment-Leitfaden.
- **Integrator (Zielsysteme, Peer-Instanzen):** Kapitel 02, 07, 08 und Anhang A.
- **Sicherheits-/Compliance-Review:** Kapitel 05, 06, 09 und Anhang B.

01 Systemüberblick

Was das DCS ist

Der **Digital Contracting Service (DCS)** ist ein föderiertes, cross-federation-fähiges und on-premise hostbares System für die sichere B2B-Kommunikation, Verhandlung und automatisierte Verarbeitung digitaler Verträge.

Der zentrale Gedanke: **Ein digitaler Vertrag ist nicht nur ein signiertes Dokument, sondern eine interoperable, maschineninterpretierbare und kryptographisch nachweisbare Vereinbarung zwischen Organisationen.** Vertragsbedingungen können von Maschinen interpretiert, zwischen Organisationen verhandelt, kryptographisch nachgewiesen, technisch umgesetzt und über ihren gesamten Lebenszyklus hinweg unabhängig auditierbar verfolgt werden.

Jede Organisation betreibt ihre eigene DCS-Instanz. Instanzen unterschiedlicher Betreiber tauschen Vertragsvorlagen und Verträge untereinander aus, ohne einander implizit zu vertrauen. Vertrauen wird pro Interaktion kryptographisch hergestellt und überprüft.

Einordnung: Gaia-X, XFSC, FACIS

Das DCS entsteht im Rahmen von **FACIS** (Federation Architecture for Composed Infrastructure Services) innerhalb des Eclipse-XFSC-Ökosystems (Cross Federation Services Components) und fügt sich in die Gaia-X-Infrastrukturlandschaft ein:

- **Federated Catalogue (XFSC/Gaia-X):** Freigegebene Vertragsvorlagen werden als Self-Descriptions im Federated Catalogue registriert und publiziert und damit föderationsweit auffindbar.
- **ORCE (XFSC Orchestration Engine):** Eine Node-RED-basierte Workflow-Engine dient als Orchestrierungs- und Integrationsschicht, unter anderem für Zeitstempel, Archiv-Notariat, Zielsystem-Anbindung, die föderationsweite Policy-Entscheidung (Trust-PDP) sowie die auslagerbaren Prüf- und Freigabeschritte des Compliance-Subsystems.
- **XFSC Status List Service:** Verwaltet den Widerrufsstatus der vom DCS ausgestellten Lifecycle-Credentials.
- **EUDI Wallet / eIDAS 2.0:** Endnutzer authentifizieren sich über Verifiable Credentials (OpenID4VP), die unter anderem aus einer EUDI Wallet vorgelegt werden können; Signaturen folgen den eIDAS-Signaturformaten (PAdES/JAdES).

Die verbindliche Anforderungsbasis ist die SRS des FACIS-DCS. Die Positionierung gegenüber den übrigen XFSC-Komponenten beschreibt ADR-5 (XFSC Component Posture).

Kernfähigkeiten

Semantische Vertragsvorlagen und maschinenlesbare Bedingungen

Organisationen definieren auf Basis frei wählbarer Ontologien semantisch maschinenlesbare Vertragsvorlagen. Ein instanzlokaler **Semantic Hub** speichert versioniert die JSON-LD-Kontexte, SHACL-Shapes und Validierungsprofile, gegen die jedes Dokument erzeugt wird, und stellt den Klausel-Katalog bereit. Vertragsbedingungen werden mit **ODRL 2.0** formal beschrieben und sind dadurch für Menschen und Maschinen interpretierbar. Vorlagen und Vertragsentwürfe können zwischen unabhängigen DCS-Instanzen ausgetauscht werden.

Verhandlung und formalisierter Abschluss

Aus Vorlagen erzeugte Verträge durchlaufen eine definierte State-Machine (Entwurf, Angebot, Verhandlung, Review, Freigabe, Signatur). Einzelne Vertragsbedingungen werden als Change Requests verhandelt. Jede Instanz führt dabei ihren eigenen Workflow mit eigenen Rollen und Freigabeprozessen; über die Instanzgrenze wandert der Vertrag selbst, nicht der interne Zustand der Gegenseite.

eIDAS-konforme Signatur und Übergabe an Zielsysteme

Abgeschlossene Verträge werden als elektronisch signierte, eIDAS-2.0-konforme und zugleich semantisch maschinenlesbare Vertragsdokumente erzeugt: Das PDF/A-3-Dokument trägt die kanonische JSON-LD-Repräsentation als eingebetteten Anhang und wird per PAdES/JAdES signiert. Zielsysteme können vereinbarte Bedingungen automatisiert umsetzen oder auswerten und vertragsrelevante Zustände, Nachweise und Vertragsgegenstände an die beteiligten DCS-Instanzen zurückmelden. Der Vertrag bleibt damit über seinen gesamten Lebenszyklus eine maschineninterpretierbare Vereinbarung, die mit technischen Systemen interagieren kann.

Zero-Trust-Identitätsmodell

Die Architektur folgt einem Zero-Trust-Modell. Vertrauen entsteht durch kryptographisch verifizierbare Identitäten, Credentials, elektronische Signaturen, Vertrauenslisten und unabhängig nachvollziehbare Prüfprozesse, nicht aus Netzwerkpositionen oder organisatorischer Zugehörigkeit. Konkret:

- **Endnutzer** authentifizieren sich über Verifiable Presentations (OpenID4VP, SD-JWT VC), z. B. aus einer EUDI Wallet; daraus entsteht eine OIDC-Session über Ory Hydra.
- **Maschinelle Aufrufer** authentifizieren sich über eigene, vom DCS ausgestellte OAuth2-Clients. Ihre Rechte stehen in einer Registry der Instanz, nie in einem Token-Claim.
- **Jede Instanz** besitzt eine `did:web`-Identität, deren privates Schlüsselmateriale ausschließlich in einem PKCS#11-Token (HSM) liegt. Instanz-zu-Instanz-Aufrufe werden per Challenge-Response-Signatur und Zertifikatskette verifiziert. Zusätzlich konsultiert jede ein- und ausgehende Föderationsinteraktion einen lokalen Policy-Endpunkt (Trust-PDP), fail-closed.
- **Ausstellervertrauen ist zweckgebunden:** Ein Credential-Aussteller wird getrennt danach zugelassen, ob er Anmeldenachweise, Nachweise einer Partnerinstanz oder Identitätsnachweise einer natürlichen Person ausstellen darf, und für welche Organisationen er sprechen darf (ADR-31).

Kryptographisch nachvollziehbare Auditierbarkeit

Vertragsänderungen, Verhandlungseignisse, Signaturvorgänge, Zustandsänderungen und weitere relevante Ereignisse werden in einem manipulationsnachweisbaren Audit-Trail erfasst. Die Integrität wird durch kryptographische Hash-Verkettungen und Merkle-Tree-Strukturen sichergestellt: Jeder Audit-Eintrag referenziert seinen Vorgänger pro Ressource, jeder Verankerungs-Batch wird durch einen Merkle-Checkpoint committet, dessen Root einen RFC-3161-Zeitstempel erhält und in **IPFS** verankert wird, einem verteilten, föderationsübergreifend nutzbaren Speichersystem. Durch die Verankerung von Vertragsrepräsentationen und Audit-Trail-Strukturen (bzw. ihrer Hashes und Merkle-Roots) entsteht ein unabhängig überprüfbarer Integritätsnachweis: Es lässt sich belegen, dass eine bestimmte Vertrags- oder Audit-Repräsentation zu einem bestimmten Zeitpunkt in einer bestimmten Form existiert hat und nachträglich nicht unbemerkt verändert wurde.

Unabhängige Validierung: maschinenlesbar ↔ menschenlesbar

Ein zentraler Bestandteil des DCS ist die unabhängige Validierung der semantischen gegen die menschenlesbare Vertragsrepräsentation. Der Dokumentendienst **pdf-core** rendert deterministisch: Dieselbe JSON-LD-Payload erzeugt byte-identischen sichtbaren Seiteninhalt. Jede beteiligte Instanz extrahiert aus einem eingehenden signierten Dokument die eingebettete JSON-LD, rendert sie unabhängig neu und vergleicht das Ergebnis mit dem erhaltenen Dokument. Damit prüft jede Instanz eigenständig, ob die maschinenlesbaren Vertragsbedingungen der für Menschen sichtbaren Darstellung entsprechen, ohne sich auf die Aussage der ausstellenden oder übermittelnden Instanz zu verlassen. Eine **C2PA-Provenance-Kette** im Dokument macht zusätzlich die Herkunft jedes sichtbaren Bytes nachweisbar.

Vertraulichkeit und Lösbarkeit

Jedes in IPFS abgelegte Artefakt ist verschlüsselt. Der Schlüssel ist pro Vertrag zufällig gezogen und liegt im Ruhezustand ausschließlich in einer Form vor, die nur das HSM der jeweiligen Instanz öffnen kann. Die Löschung eines Vertrags im Sinne von Art. 17 DSGVO ist deshalb die protokollierte Vernichtung dieser Schlüsselkopien auf beiden beteiligten Instanzen. Bytes aus einem Speicher zu entfernen, der konstruktionsbedingt nicht löschen kann, wäre keine belastbare Zusage. Die Merkle-Beweise des Audit-Trails bleiben gültig, weil sie nie vom lesbaren Inhalt abhängen (ADR-28, Kapitel 7 und 9).

Zusammenfassung

Das DCS verbindet föderierte digitale Identitäten, Verifiable Credentials, semantische Vertragsmodelle, ODRL-basierte maschinenlesbare Vertragsbedingungen, interoperable Vertragsverhandlung, eIDAS-konforme elektronische Signaturen, automatisierte Vertragsumsetzung und kryptographisch verankerte Auditierbarkeit in einer gemeinsamen Infrastruktur. Es bildet damit eine föderierte Vertrauens- und Kommunikationsinfrastruktur für digitale Verträge.

Wie diese Fähigkeiten auf Komponenten verteilt sind, beschreibt Kapitel 02 Architektur.

02 Architektur

Dieses Kapitel beschreibt die Komponentenlandkarte einer DCS-Instanz: welche Bausteine es gibt, welche Verantwortung sie tragen und wer mit wem spricht. Die fachlichen Abläufe innerhalb der Komponenten behandeln die Folgekapitel (03 bis 09), das Betriebsverhalten Kapitel 10, die Installation der Deployment-Leitfaden.

Grundschnitt

Eine DCS-Instanz besteht aus drei selbst entwickelten Diensten (**Backend**, **pdf-core**, **Frontend**) und einer Reihe von Infrastruktur- und XFSC-Komponenten, die gemeinsam mit ihnen deployt werden. Das Backend ist ein **modularer Monolith**: Alle Fachdomänen laufen in einem Prozess und teilen sich eine PostgreSQL-Instanz, sind aber über klare Modulgrenzen und einen internen Event-Bus (NATS, CloudEvents) entkoppelt. Microservices sind es bewusst nicht. Die Trennlinie zu eigenständigen Prozessen verläuft dort, wo eine andere technische Verantwortung beginnt: Byte-Arbeit am PDF in pdf-core, Orchestrierung und auslagerbare Prüfschritte in ORCE, AdES-Signaturvalidierung in DSS.

Zwei Schnitte prägen die Architektur stärker als alles andere:

1. **Alle privaten Schlüssel der Instanz liegen im PKCS#11-Token, und nur das Backend greift darauf zu** (ADR-1). Jede Komponente, die eine Signatur braucht, holt sie über einen Backend-Endpunkt.
2. **Jedes dauerhaft gespeicherte Artefakt ist verschlüsselt, bevor es den Prozess verlässt** (ADR-28). Der Schlüssel ist an einen Löschbereich gebunden: pro Vertrag, pro Vorlage, oder instanzweit.

Komponentenlandkarte

Selbst entwickelte Dienste

Komponente	Technologie	Verantwortung
Backend	Go, Goa v3 (design-first)	Gesamte Fachlogik: Template- und Vertragslebenszyklus, Verhandlung, Signaturmanagement, Föderation, Audit-Trail, Authentifizierung, Semantic Hub, Schlüsselinventar. Bedient die HTTP-API
pdf-core	Go	Deterministischer PDF/A-3a-Compiler: JSON-LD zu byte-stabilem PDF mit eingebetteter kanonischer JSON-LD und C2PA-Provenance-Kette; Re-Rendering-Verifikation, inkrementelle Amendments, Provenance-Re-Anchor. Hält kein Schlüsselmaterial; C2PA-Signaturen werden per Callback vom Backend erzeugt
Frontend	Vue 3, Vite, Pinia	Browser-UI; spricht ausschließlich die Backend-API und wird von Login bis Signatur-Ceremony durch das Backend geführt. Wird vom Backend mit ausgeliefert

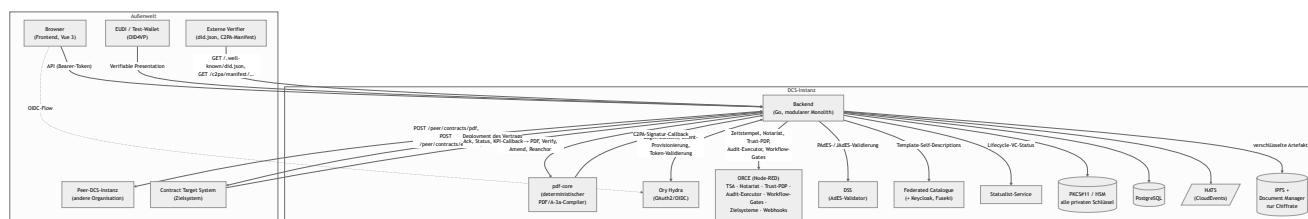
Das Backend gliedert sich intern in Fachdomänen, die jeweils eine API-Gruppe tragen: Template Repository, Contract Workflow Engine, Signature Management, PDF Generation, Process Audit & Compliance, Template Catalogue Integration, DCS-to-DCS (Föderation), Auth, Semantic Hub, Contract

Storage & Archive, Key Inventory sowie öffentliche Auskunftsdienste (DID-Dokument, C2PA-Manifest-Store). Die vollständige Endpunktübersicht liefert Anhang A.

Fremd- und Infrastrukturkomponenten

Komponente	Rolle im System
PostgreSQL	Transaktionale Persistenz aller Backend-Domänen (eine Datenbank, domänengetrennte Tabellen) plus separate Datenbanken für Hydra und den Federated Catalogue
NATS	Interner Event-Bus (CloudEvents). Trägt Domänenereignisse aus der transaktionalen Outbox an nachgelagerte Verbraucher: PDF-Regenerierung, Föderations-Versand, Webhook-Plattform, Auto-Deployment
IPFS (+ Document Manager)	Content-adressierter Speicher für alle Artefakte: Vertrags- und Vorlagen-PDFs, Archiv-Snapshots, Audit-Einträge, Checkpoints, Reports. Der Document Manager stellt eine mandantenfähige API davor. Das Backend legt dort ausschließlich Chiffre ab
Ory Hydra	OAuth2/OIDC-Provider der Instanz. Stellt die Tokens aus, mit denen Frontend-Nutzer (nach OID4VP-Wallet-Login) und maschinelle Aufrufer die API aufrufen
ORCE	XFSC Orchestration Engine (Node-RED). Beherbergt die orchestrierbaren Randdienste: RFC-3161-TSA, Archiv-Notariat und Audit-Log-Senke, Trust-PDP, Checkpoint-Anker, Zielsystem-Anbindung, Event-Webhook-Verarbeitung, den externen Audit-Executor und die Workflow-Gate-Ausführung
DSS	EU Digital Signature Service (eSignature Building Block) als externer AdES-Validator. Ohne ihn nimmt der Signatur-Submit-Pfad keine Signatur an
Federated Catalogue (+ Keycloak, Fuseki, eigene PostgreSQL)	XFSC-Katalog, in dem freigegebene Templates als Self-Descriptions registriert und publiziert werden. Keycloak stellt die Service-Account-Tokens für den Katalogzugriff, Fuseki ist dessen Graph-Store
Statuslist-Service	XFSC Status List Service: führt den Widerrufsstatus der vom DCS ausgestellten Lifecycle-Credentials und wird außerdem bei jeder Credential-Prüfung konsultiert
PKCS#11-Token (HSM)	Hält sämtliche privaten Schlüssel der Instanz (DID-, VC-, OID4VP-JAR-, C2PA- und Schlüsselvereinbarungs-Schlüssel). Nur das Backend greift darauf zu
Ingress	Externe Erreichbarkeit der Instanz im Kubernetes-Deployment. Hinter ihm liegen die öffentlichen Auflösungspfade (DID-Dokument, Agreement Credential, C2PA-Manifeste) ebenso wie die authentifizierte API

Wer spricht mit wem



Die wichtigsten Beziehungen im Einzelnen

Frontend → Backend. Der Browser spricht ausschließlich die Backend-API, authentifiziert per Hydra-Token. Der Login beginnt mit einer OID4VP-Presentation-Anfrage (QR-Code oder Deep-Link an die Wallet). Nach erfolgreicher Credential-Prüfung schließt das Backend den Hydra-Login/Consent-Flow ab, und der Browser erhält seine OIDC-Session (Kapitel 05).

Backend ↔ pdf-core. Das Backend erzeugt zu jeder relevanten Vertragsänderung die JSON-LD-Repräsentation und lässt pdf-core daraus das PDF/A-3a-Dokument kompilieren; eingehende Dokumente lässt es von pdf-core verifizieren (Re-Rendering und Vergleich der Seiteninhalte). Für die C2PA-Manifestsignatur ruft pdf-core seinerseits einen internen Signatur-Endpunkt des Backends auf. pdf-core hält nie einen Signer, und das Schlüsselmaterial verlässt den PKCS#11-Token nie.

Backend intern: Outbox → Verankerung → NATS → Verbraucher. Jede fachliche Änderung schreibt ihr Domänenereignis in derselben Transaktion in eine Outbox-Tabelle. Ein Hintergrundprozess publiziert die Ereignisse auf NATS und verankert die audit-sichtbaren davon als hash-verkettete, verschlüsselte Einträge in IPFS, gebündelt zu zeitgestempelten Merkle-Checkpoints. Auf die publizierten Events reagieren die PDF-Regenerierung, der Föderations-Versand, die Webhook-Plattform und das Auto-Deployment (Kapitel 09 und 10).

Backend ↔ Peer-Instanz (Föderation). Zwischen Instanzen ist das signierte PDF das Wire-Format: Es trägt die maschinenlesbare JSON-LD, die C2PA-Provenance-Kette und etwaige Signaturen in sich. Versendet wird in den Zuständen `OFFERED`, `NEGOTIATION`, `SIGNED` und `REVOKED`; eine beiliegende JADES macht die Sendung zur Signatur der Gegenseite. Der Empfänger verifiziert den Absender über eine did:web-Challenge-Response-Signatur und dessen Zertifikatskette, konsultiert den Trust-PDP (fail-closed, ein- wie ausgehend) und baut seine lokale Vertragskopie aus der eingebetteten JSON-LD auf. Interner Workflow-Zustand, Tasks und RBAC überqueren die Instanzgrenze nicht; jede Instanz führt ihren eigenen Prozess (Kapitel 08). Über denselben Kanal läuft die Löschanforderung eines Vertrags. Fehlgeschlagene Sendungen landen in einer Retry-Tabelle und werden periodisch erneut versucht.

Backend ↔ ORCE. ORCE bündelt die orchestrierbaren Randdienste der Instanz. Zwei davon sind harte Startabhängigkeiten, weil sie im Freigabe- und Prüfpfad sitzen: der **Workflow-Gate-Executor**, der vor Angebot, Einreichung, Freigabe, Signatur und Deployment über einen unveränderlichen Vertragsschnappschuss entscheidet, und der **Audit-Executor**, der die Auswertung eines Audit-Abrufs übernimmt. Daneben stellt ORCE die RFC-3161-Zeitstempel, nimmt Archiv-Notariats- und Audit-Log-Meldungen entgegen, beantwortet die Trust-PDP-Konsultationen der Föderation, holt den Checkpoint-Head zur externen Verankerung ab und trägt die Flows für Zielsystem-Übergabe und Event-Webhooks. Als Credential-Aussteller läuft ORCE zusätzlich in eigenen Releases: einer je Instanz für die Handlungsvollmacht (PoA), einer föderationsweit als dritter Identitätsaussteller (PID, ADR-31).

Backend ↔ Federated Catalogue / Statuslist. Template-Publikation läuft über Self-Descriptions gegen den Federated Catalogue (Service-Account-Token aus dessen Keycloak; Trust-Modell in ADR-18). Ist der Katalog konfiguriert, prüft das Backend beim Start seine Erreichbarkeit funktional und synchronisiert die Katalog-Schemata; ein konfigurierter, aber unerreichbarer Katalog verhindert den Start. Der Statuslist-Service führt den Widerrufsstatus der Lifecycle-Credentials, die die Instanz unter ihrer eigenen DID als Issuer ausstellt.

Backend ↔ Zielsystem. Ein vollständig signierter Vertrag wird an das von ihm designierte Contract Target System übergeben. Dessen Bestätigung kommt asynchron als Callback zurück und aktiviert den Vertrag. Das Zielsystem authentifiziert sich dabei als sein eigener, ihm ausgestellter OAuth2-Client (Kapitel 04 und 05).

Öffentliche Auskunftsf lächen. Ohne Authentifizierung erreichbar sind die Artefakte, die externe Verifier auflösen können müssen: das DID-Dokument der Instanz (`/ .well-known/did.json`), das Föderations-Agreement-Credential und das Regelwerk, der C2PA-Manifest-Store signierter Verträge (`/c2pa/manifest/{contract_did}`) sowie die Semantic-Hub-Auflösungsendpunkte.

Vertrauens- und Schlüsselarchitektur

Alle privaten Schlüssel der Instanz liegen in einem PKCS#11-Token, auf den nur das Backend zugreift. Die Schlüssel sind nach Zweck getrennt: Instanz-DID (`did:web`), VC-Ausstellung, OID4VP-Request-Signatur (JAR), C2PA-Manifestsignatur und Schlüsselvereinbarung für die Artefaktverschlüsselung. pdf-core, Frontend und alle Fremdkomponenten sind gegenüber der Instanzidentität schlüssellos; wer eine Signatur braucht, erhält sie über einen Backend-Endpunkt. Damit hat die Instanz genau einen Ort, an dem Signaturerzeugung stattfindet, auditierbar und HSM-gestützt (Details in Kapitel 05 und 06). Einen Vertrags-Signaturschlüssel hält die Instanz bewusst nicht, den hält der Signatar (Kapitel 06).

Die Vertrauensprüfung gegenüber Peers ruht auf unabhängigen Schichten: Zertifikatskette der Peer-DID (regulatorischer Anker), Challenge-Response-Signatur pro Request (Besitznachweis des privaten `did:web`-Schlüssels), das Föderations-Trust-Gate aus Agreement-Credential und lokalem Policy-Endpunkt (ADR-19) sowie die zweckgebundene Prüfung der Handlungsvollmacht hinter jeder Signatur der Gegenseite (ADR-31). Alle vier arbeiten fail-closed, in beide Richtungen (Kapitel 08).

Betriebssicht

Alle Komponenten einer Instanz werden über ein gemeinsames Helm-Chart deployt; zwei vollständige Instanzen lassen sich parallel betreiben, um organisationsübergreifende Abläufe nachzustellen. Erforderliche externe Abhängigkeiten prüft das Backend beim Start und schlägt hart fehl, statt degradiert zu laufen. Betriebsverhalten, Probes und Signale beschreibt Kapitel 10; Installation, Konfiguration und Upgrade der Deployment-Leitfaden.

03 Vorlagen und Vertragsinhalte

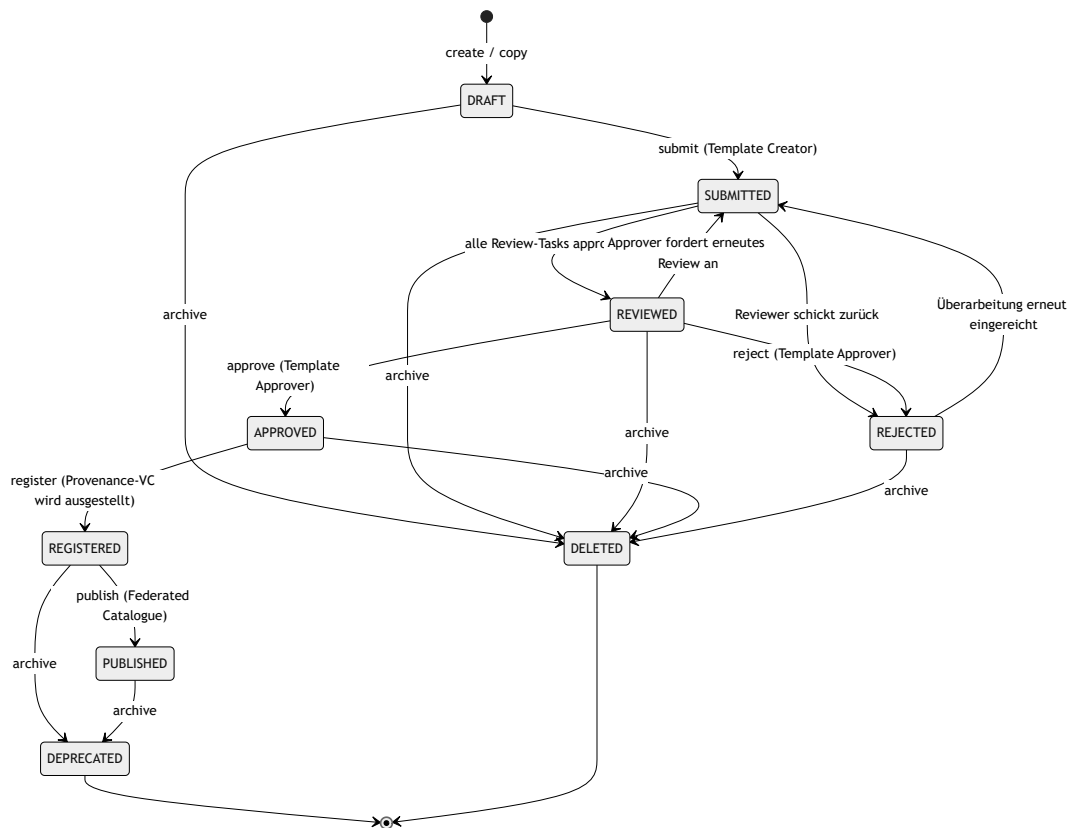
Dieses Kapitel beschreibt, wie Vertragsvorlagen im DCS entstehen, geprüft, versioniert und veröffentlicht werden, und wie der Semantic Hub die maschinenlesbare Bedeutung aller Vorlagen- und Vertragsinhalte definiert: JSON-LD-Kontext, SHACL-Shapes, Domänenfeld-Katalog, ODRL-Profil und Klausel-Katalog. Am Ende steht das Contract-Field-Modell, das erklärt, wie ein Dokument Felddeklarationen, Geschäftsdaten, menschenlesbare Struktur und ODRL-Regeln zu einem selbsttragenden Ganzen verbindet.

3.1 Beteiligte Komponenten

Komponente	Verantwortung
Template Repository	Verwaltung der Vertragsvorlagen: Lifecycle, Review-/Approval-Tasks, Versionierung, Provenance
Semantic Hub	Versionierter Speicher und Auslieferungspunkt aller semantischen Artefakte
Federated Catalogue (XFSC FC)	Externer Katalog, in den registrierte Vorlagen als Self-Descriptions veröffentlicht werden
Template Catalogue Integration	Lesezugriff auf den Federated Catalogue; so beziehen andere Instanzen veröffentlichte Vorlagen
pdf-core	Deterministisches Rendering der Vorlagen-/Vertragsdokumente (Kapitel 07)

3.2 Template-Lifecycle

Eine Vorlage durchläuft einen mehrstufigen Freigabeprozess mit Vier-Augen-Prinzip, bevor sie für die Vertragserstellung nutzbar und im Federated Catalogue sichtbar wird.



Die wesentlichen Stationen:

- **Erstellung und Bearbeitung (DRAFT).** Ein Template Creator legt eine Vorlage als kanonisches JSON-LD-Dokument an und bearbeitet sie, bis sie eingereicht wird. Jede schreibende Operation prüft per optimistischer Nebenläufigkeitskontrolle den mitgelieferten Zeitstempel gegen den Stand der Vorlage; ein veralteter Stand wird mit der Aufforderung zum Neuladen abgewiesen (Lost-Update-Schutz).
- **Einreichung (SUBMITTED).** Beim Submit werden Review- und Approval-Tasks geöffnet. Jeder Review-Task ist ein eigenes kleines Zustandsobjekt: Der Reviewer muss die Vorlage erst verifizieren (semantische Prüfung, siehe 3.3), bevor er sie freigeben kann. Erst wenn kein Review-Task mehr offen ist, wechselt die Vorlage nach **REVIEWED**. Schickt ein Reviewer die Vorlage zurück, landet sie in **REJECTED**, und alle Tasks werden für die nächste Runde wieder geöffnet.
- **Freigabe (APPROVED).** Ein Template Approver, der über den Approval-Task als zuständig ausgewiesen sein muss, trifft die finale Entscheidung: Freigabe oder Ablehnung mit Begründung. Rollenprüfung und Task-Zuständigkeit sind zwei getrennte Hürden: Die RBAC-Rolle erlaubt die Operation grundsätzlich, der Task bindet sie an die konkrete Vorlage.
- **Registrierung (REGISTERED).** Die Registrierung fixiert eine Version als die veröffentlichungsfähige. In diesem Moment stellt die Instanz ein Provenance-Verifiable-Credential über die Vorlagenversion aus, signiert mit dem VC-Schlüssel der Instanz. Die Herkunftsaussage ist damit an genau diese Version gesiegelt.
- **Veröffentlichung (PUBLISHED).** Publish übermittelt die Vorlage als Self-Description an den Federated Catalogue. Als Aussteller tritt dabei die DCS-Instanz (deren `did:web`) auf, nicht der klickende Mitarbeiter; der Katalog kennt Organisationen, keine Einzelnutzer (ADR-18). Der Katalog-Aufruf läuft bewusst außerhalb der Datenbanktransaktion. Meldet der Katalog ein Duplikat, gilt die Veröffentlichung als erfolgreich, so dass ein wiederholter Publish nach einem Teilfehler idempotent ist.

- **Ausmusterung.** `archive` wirkt zustandsabhängig: Eine registrierte oder veröffentlichte Vorlage wird `DEPRECATED` (bleibt referenzierbar, laufende Verträge zeigen den Deprecation-Hinweis), alle früheren Zustände werden `DELETED`.

Die Vorlagen-Lifecycle-Ereignisse (Anlegen, Freigabe, Aktualisierung, Registrierung und Ausmusterung) erreichen registrierte externe Abonnenten über die Webhook-Plattform (`template.created` , `template.approved` , `template.updated` , `template.registered` , `template.deprecated`). Nutzer einer Vorlage erfahren so von neuen Versionen und von der Ablösung, ohne zu pollen.

Versionierung durch Kopie

Vorlagen werden nicht in-place versioniert, sondern per Kopie: Der Copy-Befehl dupliziert eine Vorlage unter neuer DID. Ist die Quelle noch nicht registriert bzw. veröffentlicht, beginnt die Kopie eine neue Versionslinie; ist sie es, wird die Kopie die nächste Version derselben Linie. Ein einziger Mechanismus deckt damit „Entwurf duplizieren“ und „nächste Version einer veröffentlichten Vorlage erstellen“ ab, und jede neue Version durchläuft den vollen Freigabeprozess erneut.

Rollen im Template-Lifecycle

Rolle	Darf
Template Creator	Vorlagen anlegen, bearbeiten, kopieren, einreichen
Template Reviewer	Eingereichte Vorlagen verifizieren und reviewen
Template Approver	Reviewte Vorlagen freigeben oder ablehnen
Template Manager	Übergreifende Verwaltungsrechte über alle Stufen; einziger Rolleninhaber, der Semantic-Hub-Versionen registrieren darf

Vorlagen werden, anders als Verträge, nicht direkt zwischen Instanzen synchronisiert. Der Austauschweg für Vorlagen ist die Veröffentlichung in den Federated Catalogue und der Lesezugriff anderer Instanzen über die Catalogue-Integration.

3.3 Semantic Hub

Der Semantic Hub ist der versionierte Speicher für alles, wogegen das DCS seine Dokumente erzeugt und prüft. Jeder Eintrag ist ein Paar aus Name und Art (`kind`) mit monotoner, unveränderlicher Versionshistorie und genau einer aktiven Version. Die mit dem Backend ausgelieferten Basisdokumente:

Hub-Eintrag (Name / Art)	Inhalt und Wirkung
facis-dcs / context	Der JSON-LD-Kontext, den jedes Dokument als <code>@context</code> verankert; seine Präfix-zu-IRI-Zuordnungen werden auf jedem erzeugten Artefakt erzwungen
facis-dcs / shapes	SHACL-Shapes der kanonischen Dokumenthülle, der blockierende Validierungsgraph
clause-catalog / shapes	Typisierte Klausel-NodeShapes; zugleich die Deklaration ihrer Begriffe (3.4)
facis-sla / ontology	Der Domänenfeld-Katalog: <code>dcs:DomainField</code> -Einträge mit Wertebereichen und Taxonomie-Optionen. Wird als RDF-Konfiguration geparkt, nicht als Axiomenmenge
dcs-odrl-profile / ontology	Das ODRL-Profil: unterstützte Constraint-Operatoren, Aktionen, Default-Aktion. Treibt den Bedingungs-Editor
facis.sla.basic / profile	Fachliche Validierungsregeln auf Statement-Ebene
beliebige Shape-Bibliotheken / shapes	Über den Hub registrierte externe SHACL-Vokabulare (z. B. Gaia-X-Klassen): Bausteine für typisierte Geschäftsobjekte in <code>dcs:contractData</code> (3.5)

Es gibt bewusst keine OWL-Ontologie der Dokumenthülle. Was einen Hüllen-Begriff einschränkt, sind die Shapes, der einzige Graph, gegen den Dokumente validiert werden. Eine zweite, OWL-förmige Deklaration derselben Begriffe könnte diese nur wiederholen oder ihr widersprechen, und im System läuft kein Reasoner, der sie auswerten würde. Die beiden Einträge der Art `ontology` sind geparkte RDF-Konfiguration: Der Domänenfeld-Katalog ist der Index, aus dem der Feld-Picker seine Auswahl und die Prüfung ihre Liste bekannter Feld-IRIs zieht; das ODRL-Profil ist das Operator-Vokabular des Editors.

Die Wirkmechanik:

- **Seed beim Start.** Die ausgelieferten Basisdokumente werden beim Start in den Hub installiert. Fehlt ein Eintrag, wird er Version 1 (aktiv); weicht das ausgelieferte Dokument vom zuletzt gespeicherten ab, wird es als nächste Version registriert und aktiviert. Versionen sind unveränderlich. Aktualisierungen der Auslieferung erreichen laufende Deployments als gewöhnliche Versionssprünge, während auf ältere Versionen gepinnte Dokumente diese weiterhin auflösen können. Schlägt der Seed fehl, startet die Instanz nicht: Der Hub ist eine Pflichtabhängigkeit der Dokumentnormalisierung.
- **Pinning statt Mitwandern (ADR-8).** Ein neu erzeugtes Dokument verankert die jeweils aktive Version über versionsfeste URLs (`@context`, `sh:shapesGraph`, Profilverweis). Eine spätere Aktivierung einer neuen Version ändert, wogegen neue Dokumente validiert werden; bereits erzeugte Dokumente bleiben an ihre Version gebunden und dadurch dauerhaft reproduzierbar prüfbar. Für Korrekturen existiert ein expliziter Rollback auf eine ältere Version.
- **Öffentliche Auflösung.** Die Auflösungsendpunkte sind unauthentifiziert. Ein externer Prüfer muss die im Dokument verankerten Anker ohne DCS-Konto dereferenzieren können.

Was wo geprüft wird

Die Prüfschichten haben unterschiedliche Reichweite; sie zu verwechseln ist der häufigste Irrtum über das System.

Schicht	Gegenstand	Wirkung
SHACL gegen den gepinnten Shapes-Graphen	Struktur, Typen, Kardinalitäten, Node-Kinds der Hülle; zusätzlich die Wertbedingungen des Klausel-Katalogs und aller aktiven Shape-Bibliotheken	Blockierend bei Angebot, Einreichung und Signaturvorbereitung. Kanonische Shapes, Klausel-Katalog und registrierte Bibliotheken bilden einen gemeinsamen Graphen
ODRL-Constraint-Auswertung	Die im Vertrag inline getragenen Werte gegen die eigenen ODRL-Bedingungen	Blockierend bei Freigabe und Signaturvorbereitung (Kapitel 09)
Abschluss-Prüfung	Offene Pflichtfelder und von ODRL-Regeln referenzierte, unbelegte Werte	Blockierend bei Angebot, Freigabe und Signaturvorbereitung
Validierungsprofil (Statement-Regeln)	Fachliche Regeln über Aussagen im Dokument	Fließt als lokale Bewertung in das Workflow-Gate ein: ein Befund der Schwere „error“ blockiert den Übergang, eine Warnung stellt ihn unter manuelle Prüfung (Kapitel 09)
Wertbedingungen des Domänenfeld-Katalogs (<code>dcs:pattern</code> , <code>dcs:minInclusive</code> , <code>dcs:allowedValue</code>)	Zulässige Werte eines Domänenfelds	Nicht serverseitig durchgesetzt. Sie erzeugen das passende Eingabefeld und die Auswahlliste im Editor und werden clientseitig geprüft. Wer eine Wertgrenze serverseitig erzwingen will, drückt sie als SHACL aus

Bei der Signaturvorbereitung wird SHACL-Evidenz (die Shapes-Version und ein Hash über die Befundliste) in das Signing-Summary-Credential eingebettet, so dass externe Prüfer die Validierung gegen exakt die gepinnten Shapes wiederholen können.

3.4 Klausel-Katalog

Der Klausel-Katalog ist ein eigenständig versionierter Shapes-Eintrag im Hub: eine Sammlung typisierter Klausel-NodeShapes (SHACL) mit Zielklassen, Datentypen, Wertelisten, Bereichsgrenzen und Mustern. Er bedient zwei Konsumenten aus einer Quelle:

- **Vorlagen-Builder (Palette).** Der Klausel-Endpunkt liefert die Liste der verfügbaren Klauseltypen (Typ, Label, Shape-IRI), kompaktiert gegen die Präfixe des aktiven Kontexts. Die Formulargenerierung geschieht clientseitig aus dem rohen Turtle; der Server zählt auf, welche Shapes existieren (ADR-10).
- **Validierung.** Eingereichte Klausel-Instanzen werden serverseitig gegen denselben Shapes-Graphen validiert, aus dem die Formulare erzeugt wurden. Palette und Prüfung können damit nicht auseinanderlaufen.

Der Katalog ist zugleich die **Deklaration** seiner Begriffe. Eine Property-Shape trägt Datentyp, zulässige Werte, Grenzen, Namen und Beschreibung. Eine separate Vokabulardatei würde dasselbe noch einmal sagen, deshalb gibt es keine.

Eine Klausel im Dokument (`dcs:Clause`) trägt menschenlesbaren Inhalt, eine Mischung aus Prosa und Feldreferenzen. Der Klausel-Katalog schränkt darüber hinaus typisierte Geschäftsobjekte ein, die im Datengraphen des Vertrags liegen (3.5).

3.5 Die kanonische Dokumenthülle und das Contract-Field-Modell

Jedes DCS-Dokument, Vorlage wie Vertrag, ist eine einzige kanonische JSON-LD-Hülle (`dcs:ContractTemplate` bzw. `dcs:Contract`) mit klar getrennten Ebenen:

Ebene	Inhalt
<code>dcs:metadata</code>	Titel, Beschreibung, Vorlagentyp
<code>dcs:documentStructure</code>	Menschenlesbare Struktur: geordnete Blöcke (<code>dcs:Section</code> , <code>dcs:TextBlock</code> , <code>dcs:Clause</code>) und ein Layout-Baum
<code>dcs:contractFields</code>	Flache Liste der Felddeklarationen (<code>dcs:ContractField</code>)
<code>dcs:contractData</code>	Generischer Graph typisierter Geschäftsobjekte (z. B. <code>dcs:PaymentClause</code> , Gaia-X-Klassen); Eigenschaften tragen Literale, Feldreferenzen oder Referenzen auf andere Objekte
<code>dcs:policies</code>	Genau eine umschließende ODRL-Policy
<code>dcs:signatureFields</code>	Die deklarierten Signaturfelder mit ihrem jeweils geforderten Signaturniveau (Kapitel 06)

Das Contract-Field-Modell (ADR-22, erweitert durch ADR-23) trennt Felddeklaration und Geschäftsdaten:

- Jedes `dcs:ContractField` deklariert `@id` , Label, Datentyp und Pflichtkennzeichen; optional eine Shape und, auf einem ausgefüllten Vertrag, den Laufzeitwert.
- `dcs:contractData` ist ein generischer Objektgraph: Jedes Geschäftsobjekt ist ein typisierter, über `@id` adressierbarer Knoten. Eine Eigenschaft trägt genau eines von dreien: ein **Literal** (beim Authoring fixierte Daten), eine **Referenz auf ein deklariertes Feld** (ein verhandelbares Blatt) oder eine **Referenz auf ein anderes Geschäftsobjekt** (Struktur, beliebige Tiefe). Jede Referenz muss sich im Dokument selbst auflösen; eingebettete Blank Nodes sind unzulässig, so dass Klausel-Prosa, ODRL-Operanden und KPI-Beobachtungen jeden Teil des Graphen per IRI benennen können.
- Die Dokumentstruktur referenziert dieselben Felder in Klauselinhalten; Layout-Kinder sind geordnete `@list` -Referenzen.
- Auch ODRL-Operanden referenzieren dieselben Feldbezeichner.

Das Dokument ist dadurch selbsttragend: Authoring, Validierung, Rendering und Policy-Auswertung lösen ein Feld über seine `@id` auf, ohne einen Vorlagen-Schnappschuss konsultieren zu müssen. Ein Vertrag ergänzt die Hülle um Herkunft (die Vorlage, aus der er entstand) und die Vertragsparteien.

Offene Felder und das Abschluss-Gate. Auf einer Vorlage darf ein Feld offen sein, also deklariert, aber ohne Wert. Offene Pflichtfelder werden während Vertragserzeugung und Verhandlung gefüllt. Angebot, Freigabe und Signaturvorbereitung weisen einen Vertrag zurück, der noch ein ungefülltes Pflichtfeld oder einen von einer ODRL-Regel referenzierten, leeren Wert trägt; keine Partei kann ein Dokument mit offenen Bedingungen signieren (Kapitel 04).

Hierarchie. Ein Vertrag darf auf genau einen Elternvertrag verweisen; die Verknüpfung zeigt ausschließlich vom Kind zum Elternteil (ADR-7). Rückwärtsverweise auf Kindverträge werden beim Normalisieren abgewiesen. Sie wären ein Weg, aus einem Vertrag heraus auf Dokumente zu zeigen, die der Leser nicht sehen darf.

Shape-Bibliotheken treiben Editor und Validierung. Beliebige über den Hub registrierte SHACL-Bibliotheken treten dem aktiven Validierungsgraphen bei. Der Vorlagen-Editor leitet daraus seine Palette typisierter Datenobjekte ab: Jede NodeShape mit Zielklasse lässt sich per Klick als typisiertes Geschäftsobjekt in `dcs:contractData` einfügen, und die Eingabefelder entstehen aus den Shape-Eigenschaften. Validiert wird gegen eine feld-materialisierte Kopie des Dokuments: Referenzen auf gefüllte Felder werden zu ihrem Wert dereferenziert, so dass eine gewöhnliche, gegen einfache Instanzdaten geschriebene Bibliothek das lebende Dokument direkt beschränkt, ohne die Feld-Indirektion zu kennen. Referenzen auf ungefüllte Felder fehlen in der Kopie; die Kardinalitätsregeln der Bibliothek benennen dann exakt, was die Verhandlung noch liefern muss. Das gespeicherte Dokument selbst wird dabei nie umgeschrieben, und seine Shapes-Anker benennen die maßgeblichen Shapes (ADR-23).

ODRL-Regeln. Die Policy ist bis zur ersten Signatur ein `odr1:offer` und wird durch die Signatur in ein `odr1:Agreement` gesiegelt. Jede Regel trägt Aktion, Assigner, Assignee, Ziel und die menschenlesbare Klausel, die sie operationalisiert. Damit ist jede maschinenlesbare Bedingung an ihre menschenlesbare Darstellung gebunden; die unabhängige Gegenprüfung beider Repräsentationen beschreibt Kapitel 07.

Zur Laufzeit gilt: JSON-LD ist die Quelle der Wahrheit, und es gibt genau **eine** interne Form, die kanonische `dcs:`-präfixierte Hülle. Sie wird an den Eingangskanten erzwungen: Ein Dokument, dessen `@context` einen vom Hub deklarierten Präfix auf eine andere IRI umbiegt, wird abgewiesen, statt in einer zweiten Schreibweise weiterzuleben. Weil das PDF genau diese Bytes als Anhang trägt (Kapitel 07), kommt ein von einer Peer-Instanz empfangenes Dokument bereits in dieser Form an und muss nicht umgeschrieben werden. RDF wird für Validierung und Interoperabilität abgeleitet; OWL-Inferenz findet nicht statt und wird nicht vorausgesetzt.

3.6 Betriebsverhalten

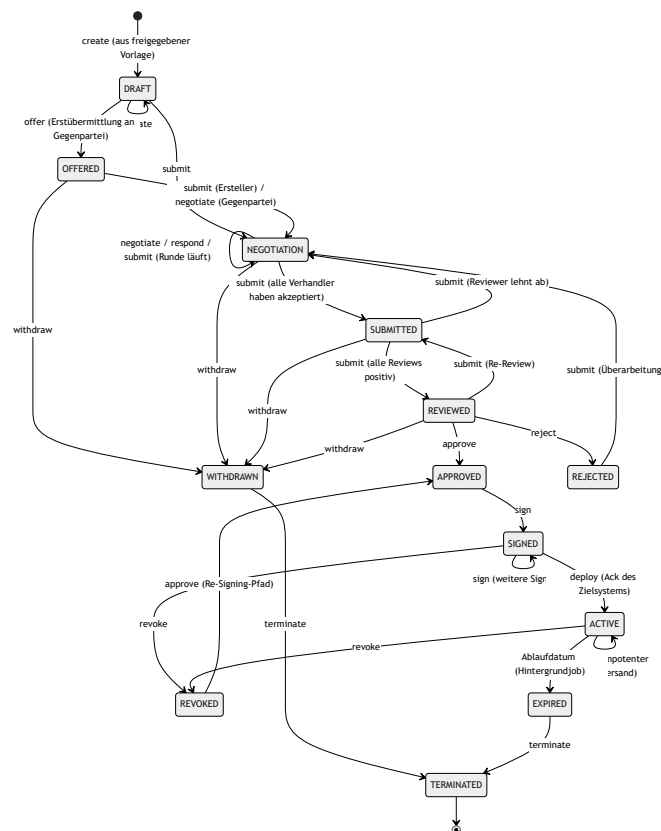
- Ohne konfigurierten Federated Catalogue schlägt `publish` mit einem eindeutigen Konfigurationsfehler fehl; alle vorgelagerten Lifecycle-Schritte funktionieren unabhängig davon.
- Ist ein Katalog konfiguriert, ist er eine harte Startabhängigkeit: Das Backend prüft ihn beim Start funktional und synchronisiert seine Schemata, bevor es bereit meldet.
- Schlägt nach erfolgreicher Katalog-Veröffentlichung die lokale Zustandsänderung fehl, heilt der nächste Publish-Aufruf den Zustand.
- Konkurrierende Bearbeitung wird über den Änderungszeitstempel erkannt und als Client-Fehler („bitte neu laden“) beantwortet. Ein stiller Überschreib findet nicht statt.
- Jeder Lifecycle-Schritt erzeugt ein Audit-Ereignis in derselben Transaktion wie die Zustandsänderung (Kapitel 09).

04 Vertragslebenszyklus

Dieses Kapitel beschreibt den Lebenszyklus eines Vertrags von der Erstellung aus einer freigegebenen Vorlage über Verhandlung, Review und Freigabe bis zu Signatur, Aktivierung, Beendigung und Löschung. Es erklärt die zentrale State-Machine der Contract Workflow Engine, das zweischichtige Berechtigungsmodell aus Rollen und Tasks, die Workflow-Gates, den Verhandlungsmechanismus sowie den Unterschied zwischen einem rein lokalen und einem instanzübergreifenden Vertrag.

4.1 Die State-Machine

Es gibt genau eine Vertrags-State-Machine im gesamten Backend (ADR-2). Zulässige Übergänge sind als explizite Tabelle aus (Zustand $\times$ Ereignis $\rightarrow$ Zielzustände) hinterlegt; jeder Command-Handler validiert gegen diese Tabelle, bevor er seine Fachlogik ausführt. Ein unzulässiger Übergang wird als Client-Fehler klassifiziert, nicht als interner Fehler.



Ergänzend zum Diagramm:

- **Terminate** ist aus jedem nicht-terminalen Zustand erreichbar (im Diagramm nur exemplarisch gezeigt) und führt immer nach `TERMINATED`.
- **Submit ist bewusst überladen:** Seine Wirkung hängt vom aktuellen Zustand ab (siehe 4.4). Die Übergangstabelle begrenzt lediglich, welche Ausgänge legal sind; welcher konkret eintritt, entscheidet die Task-Orchestrierung.
- **Withdraw** ist nur bis zur Freigabe möglich. Ein einmal freigegebener Vertrag lässt sich nur noch terminieren.

- **Deploy aus ACTIVE** ist ein idempotenter Wiederversand. Nach Signaturabschluss wird, sofern der Vertrag ein Zielsystem designiert hat (4.6), automatisch deployt und das Zielsystem bestätigt; ein manueller Deploy auf einen bereits aktivierten Vertrag bleibt gültig.
- **Expired** wird nicht durch einen API-Aufruf gesetzt, sondern von einem Hintergrundjob anhand des Ablaufdatums. Leser sehen den Zustand bereits vorher: Eine Datenbanksicht berechnet ihn zum Abfragezeitpunkt, der Job holt die physische Persistenz und das Ereignis nach. Mit dem Übergang wird `contract.expired` publiziert und erreicht registrierte Webhook-Abonnenten.
- **Renew** ist kein Zustandsübergang. Die Verlängerung erzeugt eine neue, verknüpfte Vertragsinstanz in `DRAFT` mit Rückreferenz auf Original-DID und -Version; das Original bleibt unverändert. Die Signaturen der Vorperiode werden aus dem neuen Entwurf entfernt, denn ein ungezeichneter Entwurf darf niemanden als Unterzeichner ausweisen.

Extrinsischer Lebenszyklus

Neben dem internen Zustand präsentiert jeder Vertrag über die Instanzgrenze hinweg einen abgeleiteten, peer-gerichteten Verhandlungslebenszyklus: Alle Formationszustände bis einschließlich `REVIEWED` erscheinen als `proposed`, die beidseitige interne Freigabe als `agreed` (hier öffnet sich das Signatur-Gate), ein Vertrag mit vollständig vorliegenden Signaturen als `executed`. Maßgeblich für `executed` ist, dass tatsächlich jede deklarierte Signatur vorliegt, einschließlich der kryptographisch geprüften Signatur der Gegenseite; die eigene erste Unterschrift genügt nicht. Beide Instanzen leiten denselben Wert aus dem gemeinsamen Artefakt und ihrem eigenen Zustand ab. Es wird kein Zustand über die Föderationsgrenze synchronisiert.

4.2 Rollen, Tasks und RBAC

Die Autorisierung hat zwei getrennte Schichten:

1. **Lokale RBAC-Rollen** bestimmen, welcher lokal authentifizierte Nutzer eine Operation grundsätzlich ausführen darf. Für Verträge sind das Contract Creator, Contract Reviewer, Contract Approver, Contract Negotiator, Contract Manager, Contract Signer und Contract Observer. Maschinelle Aufrufer treten in einer eigenen `Sys.`-Klasse auf, die nur einen Teil davon spiegelt: Anlegen, Prüfen, Freigeben, Verwalten und Signieren. Für Verhandeln und Beobachten gibt es kein maschinelles Pendant, und die maschinelle Signaturklasse trägt keine Signaturberechtigung (Kapitel 05 und 06). Audit-Lesezugriff haben Auditor, Archive Manager und Compliance Officer. Die Integrationsflächen verwaltet der Sys. Administrator; die Zielsystem-Registry zusätzlich der Integration Manager.
2. **Tasks binden Zuständigkeit an den konkreten Vertrag.** Bei der Vertragserstellung öffnet die Instanz ihre eigenen Review-, Verhandlungs- und Approval-Tasks für die benannten Zuständigen. Die Rolle erlaubt die Operation, der Task entscheidet, ob dieser Aufrufer für diesen Vertrag zuständig ist.

Die Tasks sind kleine Sub-State-Machines, die als Fan-in-Gates die großen Übergänge steuern:

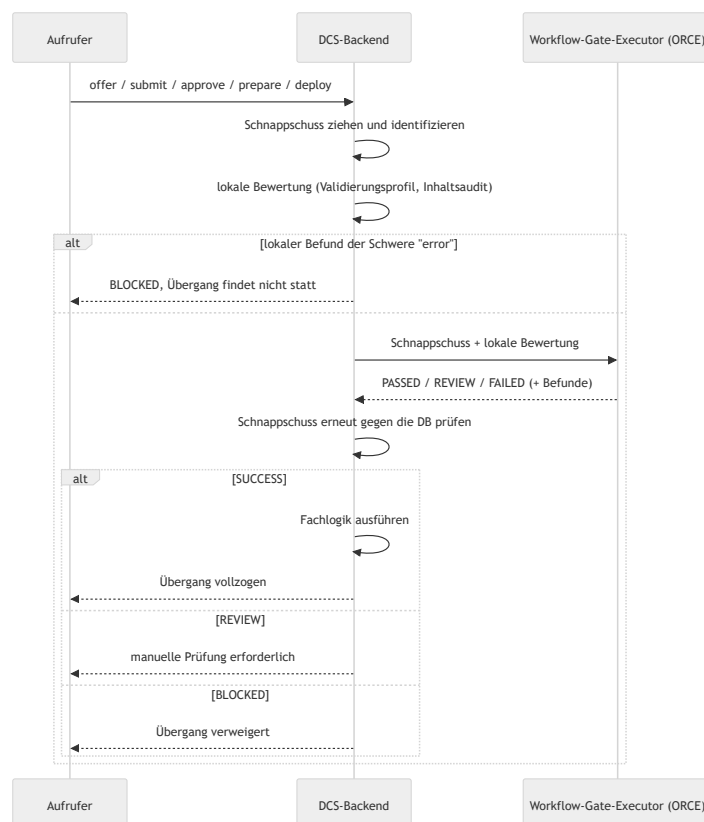
Task	Zustände	Gate
Negotiation-Task	offen → akzeptiert	Erst wenn kein Verhandler-Task mehr offen ist, kann NEGOTIATION → SUBMITTED erfolgen
Review-Task	offen → freigegeben / abgelehnt	Steuert SUBMITTED → REVIEWED ; eine Ablehnung öffnet die Verhandlung erneut
Approval-Task	offen → freigegeben / abgelehnt	Steuert REVIEWED → APPROVED

Jede DCS-Instanz erstellt und besitzt ausschließlich ihre eigenen Tasks; Task-Zustände überqueren nie die Instanzgrenze. Die Gegenpartei eines instanzübergreifenden Vertrags autorisiert sich über ihre Eigenschaft als benannte Gegenpartei (siehe 4.5), nicht über lokale Tasks.

Für Lesezugriffe gilt zusätzlich eine Parteiprüfung: Das kanonische Vertragsdokument und seine auflösbare Ressourcen-IRI sind nur für autorisierte Parteien des Vertrags lesbar. Eine Verweigerung wird nicht nur beantwortet, sondern als Audit-Artefakt festgehalten, bevor die Ablehnung zurückgeht. Daraus speist sich das Compliance-Risiko „unautorisierter Zugriff“ (Kapitel 09).

4.3 Workflow-Gates: der auslagerbare Prüfschritt vor dem Übergang

Fünf Lebenszyklusübergänge laufen nicht direkt in die Fachlogik, sondern zuerst durch ein **Workflow-Gate**: Angebot, Einreichung, Freigabe, Signaturvorbereitung und Deployment. Ein Gate arbeitet auf einem **unveränderlichen Schnappschuss** des Vertrags (Version, Zustand, Inhalts-Hash, die gepinnten Shapes und die Profilversion) und läuft in drei Schritten:



Wesentliche Eigenschaften:

- **Der Lauf ist Evidenz.** Jeder Gate-Lauf wird mit Korrelations-ID, Schnappschuss-Identität, Inhalts-Hash, wirksamen Shapes, Profilversion, Anfrage und Antwort persistiert und ist über die Compliance-API abrufbar. Zwei Läufe desselben Schnappschusses am selben Gate sind derselbe Lauf; die Ausführung ist idempotent.
- **Der Schnappschuss wird nach der Antwort erneut geprüft.** Hat sich der Vertrag zwischenzeitlich inhaltlich verändert, wird das Ergebnis verworfen und der Übergang blockiert. Rein technische Schreibvorgänge (PDF-Regeneration, Audit) verschieben zwar den Änderungszeitstempel, ändern aber den bewerteten Inhalt nicht und blockieren deshalb nicht.
- **REVIEW ist eine Wartestellung, kein Fehler.** Ein Compliance Officer entscheidet den Lauf mit Begründung; bei Freigabe wird genau der Übergang fortgesetzt, der angehalten wurde. Die dafür nötigen Angaben liegen am Lauf.
- **Fail-closed.** Ein nicht konfigurierter oder nicht erreichbarer Executor blockiert den Übergang; der Lauf wird trotzdem mit der Ursache festgehalten. Der Executor ist deshalb eine harte Startabhängigkeit der Instanz.

4.4 Verhandlung

Die Verhandlung findet im Zustand `NEGOTIATION` statt, oder auf einem eingegangenen Angebot in `OFFERED`, und dreht sich um Change-Requests:

- **Vorschlagen.** Ein Verhandler reicht einen Change-Request ein. Der Request ist entweder Freitext (eine Anmerkung, die nur im Verhandlungs-Audit-Trail landet) oder ein strukturierter Redline auf die Vertragsdaten. Ein strukturierter Redline wird sofort auf die Vertragsdaten angewendet, dadurch rendert das Vertrags-PDF mit dem vorgeschlagenen Wert neu und wird über den PDF-Austausch an die Gegenpartei verschifft (4.5); die Gegenpartei begutachtet den Redline im empfangenen Dokument. Jeder Change-Request wird unabhängig davon für den Audit-Trail aufgezeichnet.
- **Entscheiden.** Jeder zuständige Verhandler akzeptiert oder lehnt einen konkreten Change-Request ab (Ablehnung mit Begründung).
- **Runde abschließen.** Submit ist der Rundenabschluss: Sind noch Verhandler-Tasks offen, bleibt der Vertrag in `NEGOTIATION`; sind alle Entscheidungen gefallen und akzeptierte Change-Requests zu mergen, werden sie zusammengeführt, die Vertragsversion zählt hoch, und eine neue Runde beginnt; ist nichts mehr zu mergen, wechselt der Vertrag nach `SUBMITTED` in das Review.

Verhandlungs-Drafts

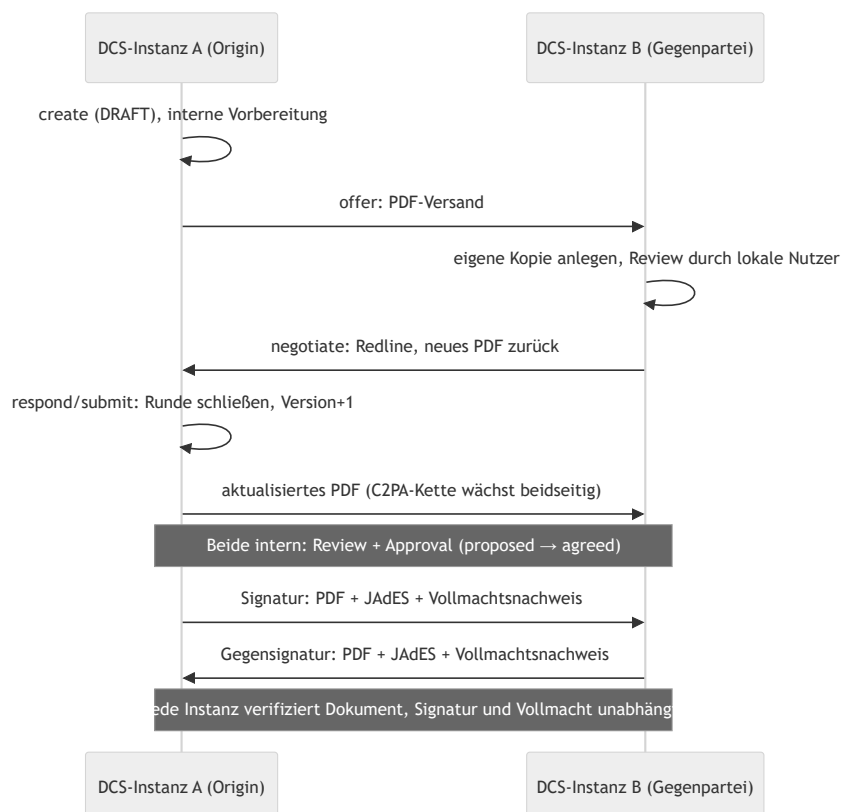
Getrennt vom Vorschlagen können Verhandler einen Change-Request **vorbereiten**: Der Draft wird pro (Vertrag, Partei) gespeichert und von allen autorisierten Verhandlern derselben Partei geteilt, so dass jeder die gemeinsame Verhandlungsposition der Partei fortsetzt. Ein Draft ändert keinen Vertragszustand, erhöht keine Version, erzeugt kein Audit-Ereignis und verlässt die Instanz nicht. Erst das Vorschlagen macht ihn zum Change-Request (und löscht ihn), Verwerfen löscht ihn ebenfalls. Drafts sind nur speicherbar, solange der Vertrag verhandelbar ist.

4.5 Lokal vs. grenzüberschreitend

Ein Vertrag hat genau eine **Ursprungsinstanz** (Origin) und genau eine **Gegenpartei**, ein Peer-DCS, identifiziert über `did:web`, das bei der Vertragserstellung benannt wird. Daraus ergeben sich zwei Betriebsarten:

- **Lokal:** Origin und aufrufende Instanz sind identisch. Alle Zuständigkeiten sind lokale Tasks, nichts verlässt die Instanz (bis auf ein optionales Deployment an ein Zielsystem).
- **Grenzüberschreitend:** Zustandsverändernde Aufrufe auf der Nicht-Origin-Instanz werden an die Origin-Instanz weitergeleitet; die Origin führt die State-Machine. Beide Instanzen betreiben ihren eigenen Workflow und ihre eigene RBAC. Über die Grenze wandert ausschließlich das Vertragsdokument selbst (ADR-13).

Das PDF ist das Wire-Format. Zwischen den Instanzen wird bei jedem relevanten Lebenszyklusschritt das Vertrags-PDF verschifft: Es trägt das eingebettete maschinenlesbare JSON-LD, die C2PA-Provenance-Kette und vorhandene Signaturen. Ein nacktes PDF ist ein Vorschlag (Angebot oder Gegenvorschlag); ein PDF mit beigefügter JAdES-Signatur ist die Signatur der Gegenseite. Der Empfänger authentifiziert den Absender über eine did:web-Challenge-Response-Signatur, nicht per Session-Token, denn es gibt keine gemeinsame Endnutzeridentität über Betreibergrenzen. Anschließend prüft pdf-core, dass der Seiteninhalt der Re-Render der eingebetteten Payload ist, extrahiert das JSON-LD, und die Instanz aktualisiert ihre lokale Vertragskopie.



Die Autorisierung spiegelt die Rollenteilung: Auf der Origin-Instanz verlangt das Verhandeln einen lokalen Verhandler-Task; auf der Empfängerseite leitet sich das Recht, ein eingegangenes Angebot zu verhandeln, aus der Eigenschaft als benannte Gegenpartei ab. Auch die optimistische Nebenläufigkeitskontrolle

unterscheidet die Fälle: Bei einem veralteten Stand auf einem synchronisierten Vertrag lautet die Antwort „Synchronisation erzwingen und neu laden“.

Details zu Trust-Modell und Peer-Authentifizierung behandelt Kapitel 08, die Signatur-Zeremonie Kapitel 06, die unabhängige Neu-Rendering und den Abgleich der Repräsentationen Kapitel 07.

4.6 Nach der Signatur: Deployment, Widerruf, Beendigung

Zielsystem-Registry und Designation (ADR-25)

Contract Target Systems sind eine administrierte Registry der Instanz mit Name, Endpunkt, Beschreibung und Aktiv-Kennzeichen. Jeder Vertrag **designiert** das Ziel, an das er deployt wird, mit derselben Staleness-Prüfung wie jede andere Vertragsmutation; das designierte Ziel wird in Vertragsliste und Einzelabruf mit Namen ausgewiesen. Ein deaktiviertes Ziel kann nicht designiert werden, und ein Ziel, das noch von Verträgen designiert wird, kann nicht gelöscht werden. Ein Vertrag ohne designiertes Ziel deployt schlicht nicht, und das ist ein normaler Ausgang: Der automatische Trigger nach Signaturabschluss tut dann nichts; ein manuell angeforderter Deploy wird mit sprechender Begründung abgewiesen, ebenso bei einem unbekannten oder deaktivierten Ziel.

Deployment

Ein vollständig signierter Vertrag wird als maschinenlesbares JSON-LD-Payload an das designierte Contract Target System übermittelt. Ein manueller Deploy darf per Override an ein anderes registriertes Ziel gerichtet werden, ohne die Designation des Vertrags zu ändern.

Das **Deploy-Gate** verlangt, dass jedes deklarierte Signaturfeld tatsächlich signiert ist. Für die eigenen Felder ist der Nachweis die lokal aufgezeichnete Signatur; für die Gegenseite ist es das verifizierte JAdES-Artefakt, das der Peer mit seiner signierten Kopie mitgeschickt hat. Ohne diesen Nachweis kommt keine Aktivierung zustande. Eine Partei kann einen Vertrag nicht allein in die Ausführung bringen. Die Instanz hält je Vertrag genau eine Gegenseiten-Signatur vor; ein Vertrag mit drei oder mehr Parteien wird deshalb nicht deployt, statt eine unvollständige Signaturlage als vollständig zu behandeln.

Der Deployment-Datensatz referenziert den Registry-Eintrag und hält zusätzlich den Endpunkt fest, wie er beim Versand galt; eine spätere Änderung des Eintrags schreibt die Historie nicht um. Der Versand erhält eine Correlation-ID und einen Content-Hash über die kanonisierte Payload, den das Zielsystem selbst nachrechnen kann. Ein fehlgeschlagener Versand wird am Datensatz vermerkt und vom Compliance-Monitor als Risiko gemeldet.

Die Bestätigung des Zielsystems kommt asynchron als Callback und schaltet den Vertrag auf **ACTIVE**. Das Zielsystem authentifiziert sich dabei als sein eigener, dem Registry-Eintrag ausgestellter OAuth2-Client: Nur das Ziel, an das ein Deployment ging, kann es quittieren, und ein kompromittiertes Ziel kann nicht für andere sprechen (ADR-27). Über denselben Callback meldet das Zielsystem später KPI-Werte, die gegen die vertraglichen Schwellen aus den ODRL-Constraints geprüft und am Vertrag ausgewiesen werden (Kapitel 09).

Widerruf, Beendigung, Ablauf

- **Widerruf.** Der Widerruf einer Signatur versetzt den Vertrag von `SIGNED / ACTIVE` nach `REVOKED`; von dort führt ein erneutes Approve zurück nach `APPROVED`, um Re-Signing zu ermöglichen. Bei einem grenzüberschreitenden Vertrag wird der Widerruf unmittelbar an die Gegenpartei verschickt. Er ist der einzige Peer-Zustand, den der Empfänger übernimmt (Kapitel 08).
- **Beendigung.** Termine mit Begründung beendet den Vertrag endgültig.
- **Ablauf.** Der Hintergrundjob persistiert den `EXPIRED` -Zustand und publiziert das Ereignis. Die am Vertrag hinterlegte Expiration-Policy (Verlängerung, Archivierung, Beendigung) wird dabei **aufgezeichnet und im Ereignis mitgeführt, aber nicht automatisch ausgeführt**. Die Folgehandlung ist ein manueller oder über Webhooks orchestrierter Schritt. Ein Vertrag ohne hinterlegte Expiration-Policy wird vom Job übersprungen und protokolliert: Sein Zustand wird nicht persistiert und kein `contract.expired` publiziert, obwohl Lesesichten ihn bereits als `EXPIRED` zeigen.

Archiv und Löschung

Abgeschlossene Verträge liegen mit ihrer Evidenz im Langzeitarchiv. Das Archiv ist strukturiert durchsuchbar, neben Volltext, Name, Zustand und Annotations-Tags auch nach Vertragspartei und Gültigkeitszeitraum. Das Archiv-Dashboard weist Bestands- und Compliance-Statistiken sowie die demnächst ablaufenden archivierten Verträge aus.

Die Löschung eines Archiveintrags ist ein begründungspflichtiger Vorgang mit zwei Wirkungen:

1. Der Eintrag wird als gelöscht markiert und bleibt für Compliance- und Streitfälle nachweisbar; die zugehörigen archivierten Snapshots werden aus dem Speicher entfernt.
2. Die **Inhaltsschlüssel des Vertrags werden vernichtet** (ADR-28): Die Instanz zerstört ihre gewickelten Kopien des Vertragsschlüssels, schreibt dazu ein Zerstörungseignis in den Audit-Trail und fordert jede Gegenpartei-Instanz auf, dasselbe zu tun. Danach sind Export, Verifikation und Bundle-Abruf dieses Vertrags dauerhaft nicht mehr möglich; Listen- und Metadatensichten funktionieren weiter. Eine nicht erreichbare Gegenseite blockiert die lokale Löschung nicht: Die Anforderung bleibt offen und wird periodisch erneut zugestellt. Der Fortschritt dieser Kette ist über eine eigene Statusabfrage sichtbar (Kapitel 09).

4.7 Betriebsverhalten

- **Optimistische Nebenläufigkeit:** Fast alle zustandsverändernden Endpunkte verlangen den Zeitstempel des Standes, den der Aufrufer gesehen hat; ein veralteter Stand wird mit „bitte neu laden“ (bzw. bei synchronisierten Verträgen „Synchronisation erzwingen“) abgewiesen. Verglichen wird gegen den Inhalts-Zeitstempel, so dass gutartige Hintergrundschreibvorgänge keine Fehlalarme auslösen.
- **Unzulässige Übergänge** werden von der Übergangstabelle abgefangen und als Client-Fehler mit sprechender Meldung beantwortet.
- **Semantische Validierung blockiert:** Angebot, Einreichung und Signaturvorbereitung scheitern bei SHACL-Fehlerbefunden gegen die gepinnten Hub-Shapes (Kapitel 03).

- **Abschluss-Gate:** Angebot, Freigabe und Signaturvorbereitung verlangen einen geschlossenen Vertrag. Jedes Pflichtfeld und jeder von der ODRL-Policy referenzierte Wert muss gefüllt sein; ein Vertrag mit offenen Bedingungen wird mit der Liste der ungelösten Felder abgewiesen.
- **Jede Zustandsänderung** schreibt ihr Audit-Ereignis in derselben Datenbanktransaktion; der Audit-Trail eines Vertrags ist damit lückenlos gegenüber seiner Zustandshistorie (Kapitel 09).
- **Lebenszyklus-Ereignisse für externe Systeme:** Domänenereignisse des Vertrags- und Vorlagen-Lebenszyklus werden vom internen Event-Bus über die Webhook-Plattform an registrierte Abonnenten zugestellt; Zustellungen sind über ein Delivery-Log nachvollziehbar und per Callback quittierbar (Anhang A).
- **Deployment-Vorgänge** sind über Correlation-IDs mit ihren Callbacks korrelierbar. Ein Versand, den das Zielsystem nie erhalten hat, ist von einem quittierten unterscheidbar: Der Zustellfehler steht am Deployment-Datensatz, und der Compliance-Monitor erzeugt dafür ein Risiko.

05 Identität und Authentifizierung

Das DCS kennt drei Identitätsebenen, die strikt getrennt gehalten werden: **Endnutzer** (Menschen, die sich mit einer Wallet anmelden), **Maschinen-Identitäten** (automatisierte Aufrufer mit eigenem OAuth2-Client) und die **DCS-Instanz selbst** (eine `did:web`-Identität mit HSM-gestütztem Schlüsselmaterial). Vertrauen entsteht auf jeder Ebene kryptographisch, aus Verifiable Credentials, signierten Tokens und verifizierbaren DID-Dokumenten; Netzwerkposition oder Konfigurationsnähe begründen nichts (Zero-Trust-Modell, vgl. Kapitel 01).

Quer über alle drei Ebenen liegt eine vierte Frage, die dieses Kapitel zuletzt behandelt: **Wem darf die Instanz als Aussteller von Credentials glauben, und wofür?**

5.1 Endnutzer: Login per OID4VP und SD-JWT VC

Beteiligte Komponenten

Komponente	Rolle
EUDI Wallet (bzw. kompatible Wallet)	Hält die Credentials des Nutzers und legt sie per OpenID4VP vor
DCS-Backend (Auth-Domäne)	OID4VP-Verifier: baut das Presentation Request, prüft die Vorlage, entscheidet über den Login
Ory Hydra	OAuth2/OIDC-Provider: stellt Access-, Refresh- und ID-Token aus, verwaltet die Browser-Session
Frontend	Zeigt den QR-Code / Deep Link, pollt den Login-Status, führt den Browser durch den OIDC-Flow

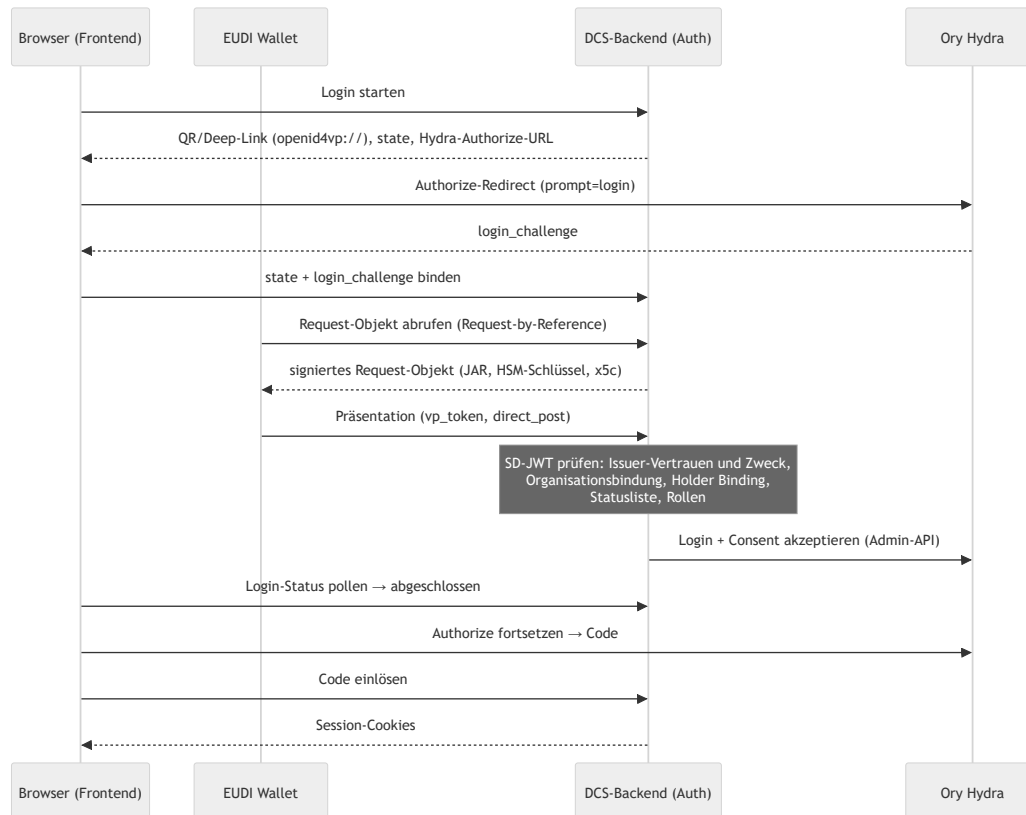
Hydra kennt selbst keine Nutzer und keine Passwörter. Die eigentliche Authentifizierungsentscheidung trifft ausschließlich das DCS-Backend anhand der verifizierten Wallet-Vorlage. Hydra dient als Token-Fabrik, deren Login- und Consent-Challenges das Backend über die Hydra-Admin-API akzeptiert und dabei die verifizierten Claims (Subjekt, Organisation, Rollen) in die Token-Session übernimmt.

Das Credential: Proof of Authority (PoA)

Für den Login verlangt das DCS ein **Proof-of-Authority-Credential** vom Typ `urn:dc:poa:v1` im Format `dc+sd-jwt` (SD-JWT VC mit selektiver Offenlegung). Es trägt zwei entscheidende Claims: `organization`, die Partei, für die der Nutzer handelt, und `roles`, die Rollen, die der Aussteller dem Nutzer eingeräumt hat. Welche Credentials und Claims angefragt werden, beschreibt eine DCQL-Query; die Standard-Query verlangt kryptographisches Holder Binding und ist per Konfiguration vollständig überschreibbar.

Die `organization` ist eine Parteikennung, nicht die Identität des Deployments: Eine Instanz bedient legitimerweise mehrere Organisationen gleichzeitig. Welcher Aussteller für welche Organisation sprechen darf, entscheidet die Trust-Konfiguration (5.4).

Ablauf



Das Request-Objekt (JAR) wird mit einem eigenen HSM-Schlüssel signiert und trägt die Zertifikatskette der Instanz im Header. Der der Wallet präsentierte Client-Identifizier folgt dem Schema `x509_san_dns` und entspricht dem SAN des Leaf-Zertifikats. Eine Wallet außerhalb einer vorab vereinbarten Föderation lehnt einen unpräfixierten, „pre-registered“ gemeinten Identifizier ab, bevor sie überhaupt ein Credential ansieht; ein bloßer Schlüssel im Header würde nur Besitz beweisen und an nichts verankern. Login-States sind zeitlich begrenzt und verlängerbar, ohne den OIDC-State zu wechseln.

Verifikation der Vorlage

Die eingehende Präsentation durchläuft eine feste Prüfkette. Jede Stufe ist ein hartes Abbruchkriterium:

1. **Aussteller und Zweck.** Der Aussteller muss in der Trust-Konfiguration stehen und für den Zweck `login` zugelassen sein; sein Verifikationsschlüssel wird über den für ihn deklarierten Mechanismus aufgelöst (5.4). Der Credential-Typ muss zu den zugelassenen `vct`-Werten gehören.
2. **Organisationsbindung.** Die im Credential offengelegte `organization` muss zu den Organisationen gehören, für die dieser Aussteller sprechen darf.
3. **Holder Binding.** Der Bindungsschlüssel des Credentials ist maßgeblich: Das Subjekt muss dem daraus abgeleiteten Identifizier entsprechen, und der Key-Binding-JWT muss mit genau diesem Schlüssel signiert sein sowie Audience, Nonce und den Hash der Offenlegungen der laufenden Präsentation tragen (Replay-Schutz).
4. **Statusliste.** Der Widerrufsstatus wird gegen die referenzierte Statusliste geprüft, auf jedem Pfad, für jeden Zweck, ohne Ausnahme. Unterstützt werden mehrere Mechanismen (IETF Token Status List, W3C Bitstring Status List in verschiedenen Proof-Formaten, XFSC-Statuslisten); auch die Statusliste

selbst muss signiert und vertrauenswürdig sein. Eine nicht erreichbare Statusliste führt zur Ablehnung.

5. **Rollen.** Jede offengelegte Rolle muss eine gültige DCS-Rolle sein; ohne offengelegte Rollen wird der Login abgelehnt. Die gewährten Rollen landen im Access-Token und sind die Grundlage der endpunktweisen RBAC-Prüfung (Kapitel 04).

PID-Präsentation

Neben dem PoA-Login gibt es einen eigenständigen, sessionlosen PID-Präsentationsfluss für Fälle, in denen die natürliche Person selbst nachgewiesen werden muss, insbesondere als Vorstufe der Signatur-Ceremony (Kapitel 06). Die Verifikation folgt derselben Kette wie beim Login, mit dem Zweck `pid` statt `login`; die Organisationsbindung entfällt, weil ein Identitätsnachweis eine Person attestiert und keine Organisation.

Welcher PID-Aussteller vertraut wird, ist reine Trust-Konfiguration. Ein PID-Aussteller ist konstruktionsbedingt ein **Dritter**: Die Instanz darf den Identitätsnachweis nicht ausstellen, den sie später als Beweis akzeptiert, wer unterschrieben hat. Das wäre die vertrauende Partei, die sich selbst attestiert (ADR-31).

Schutzmechanismen und Audit

- **IP-Lockout:** Fehlgeschlagene Token-Validierungen werden je Quell-IP gezählt; nach fünf Fehlversuchen im Zeitfenster wird die IP für 15 Minuten gesperrt. Eine gültige Anmeldung mit fehlender Rolle ist dagegen eine Autorisierungsentscheidung und zählt nicht zum Lockout; sie wird als authentifiziertes Zugriffsereignis auditiert.
- **Audit-Trail:** Zugriffs- und Präsentationsversuche, erfolgreiche wie fehlgeschlagene, werden persistiert und in den manipulationsnachweisbaren Audit-Trail eingespeist (Kapitel 09).
- **Anfragebudget je Credential:** Jede authentifizierte API-Anfrage wird gegen ein festes Ein-Minuten-Fenster je Credential gezählt; oberhalb des Budgets antwortet das DCS mit `429 Too Many Requests` und `Retry-After`. Unauthentifizierte Routen (Login, DID-Dokument, statische Assets, Probes) zählen nicht. Rohe Tokens werden dafür nie im Speicher gehalten; gezählt wird über einen gekürzten Hash des Credentials.

5.2 Maschinen-Identitäten: ausgestellte Credentials über Hydra

Maschinelle Aufrufer (Integrationen, Orchestrierungs-Flows, Zielsysteme) authentifizieren sich mit dem **Client-Credentials-Grant** direkt bei Hydra. Jeder Aufrufer erhält dafür einen **eigenen OAuth2-Client**, den das DCS über die Hydra-Admin-API provisioniert (ADR-27). Eine **Machine-Identity-Registry** verzeichnet je Identität den OAuth2-Client, die Partei, der die Aktionen des Aufrufers zugerechnet werden, die Rollen (typisch die `sys.`-Rollenvarianten) und ob die Identität aktiv ist.

Berechtigungen werden bei jeder Anfrage anhand der Client-ID aus der Registry gelesen, nie aus Token-Claims: Ein Client-Credentials-Token trägt keine Rollenangaben, und nichts, was ein Aufrufer vorlegt, kann seine Rechte erweitern. Ohne Registry-Eintrag oder mit deaktiviertem Eintrag wird kein maschineller Aufrufer akzeptiert.

Die Credentials sind **ausgestellt, nicht konfiguriert**: Beim Anlegen einer Identität und bei jeder Rotation wird das Client-Secret genau einmal in der Antwort angezeigt. Das DCS speichert es nie, Hydra hält nur einen Hash und hat keine Schnittstelle, es zurückzulesen; „einmal sichtbar“ ist damit eine Eigenschaft des Systems, keine UI-Konvention. Eine Rotation stellt ein neues Secret aus und invalidiert das alte sofort; das Löschen einer Identität löscht auch ihren OAuth2-Client, sodass kein Credential den Eintrag überlebt, der es rechtfertigt. Das Deaktivieren weist die Identität bei der nächsten Anfrage ab, ohne auf einen Secret-Ablauf zu warten. Verwaltet wird das als Administrationsvorgang durch den Sys. Administrator; Ausstellen, Rotieren und Widerrufen sind auditierte Betriebshandlungen und keine Deployment-Schritte.

Die Deployment-Konfiguration `DCS_SYSTEM_CLIENTS` ist ein **Seed**: Ihre Einträge werden beim Start in die Registry übernommen, weil ein Deployment Aufrufer braucht, die existieren, bevor sich ein Mensch anmelden kann, um sie anzulegen. Zur Laufzeit wird ausschließlich die Registry gelesen; es gibt einen Auflösungspfad, nicht zwei. Eine ungültige Rolle in der Deklaration lässt den Start scheitern.

Contract Target Systems sind keine allgemeinen Maschinen-Identitäten. Die Rolle `Contract Target System` autorisiert ausschließlich den Deployment-Callback, und dort nur Deployments, die an genau dieses Ziel versandt wurden. Das Callback-Credential eines Ziels ist derselbe Mechanismus (eigener OAuth2-Client, Secret einmal sichtbar, Rotation) und ist an den Registry-Eintrag des Ziels gebunden; ein Ziel ohne ausgestelltes Credential kann kein Deployment quittieren (Kapitel 04).

Rollengrenze zwischen Mensch und Maschine. Die `sys.`-Rollen sind bewusst nicht deckungsgleich mit den menschlichen. Der Katalogzugriff etwa ist nur menschlichen Vertragsrollen zugänglich, und die Registrierung neuer Semantic-Hub-Versionen verlangt den Template Manager, für den es keine `sys.`-Variante gibt. Diese Grenze zu weiten, um einen Ablauf ohne Wallet-Login durchzuspielen, hebt eine bewusste Entscheidung auf.

Der clusterinterne Dienst-Token. Daneben existiert ein Pfad für den ereignisgetriebenen PDF-Regenerator: Er trägt kein Nutzer-Token, sondern ein im Deployment gesetztes Dienst-Credential und erhält damit exakt die Scopes des jeweiligen Endpunkts. Dieser Abgleich findet **vor** Token-Validierung, Rollenprüfung und Lockout statt: Wer den Wert besitzt, ist auf jeder authentifizierten Route ein Aufrufer mit vollen Rechten. Dass er den Cluster nicht verlässt, ist eine Eigenschaft der vorgesehenen Aufrufer, nicht der Prüfung; der Wert ist entsprechend zu behandeln. Das Chart erzwingt, dass ein Betreiber ihn setzt, und einen mitgelieferten Vorgabewert gibt es nicht. Herkunft und Ablage beschreibt der Deployment-Leitfaden.

Wichtig für die Einordnung: Maschinelles Signieren ist im DCS **keine** AES-Signatur einer Person (ADR-17). Maschinen-Identitäten handeln als System, nicht als Signatar.

5.3 Instanz-Identität: did:web mit HSM-Schlüsseln

Jede DCS-Instanz besitzt eine eigene `did:web`-Identität. Das DID-Dokument wird unter `/well-known/did.json` ausgeliefert und publiziert die öffentlichen Schlüssel der Instanz. Die Auflösung folgt der `did:web`-Methode vollständig, auch für Identifier mit Pfadsegmenten: ein bloßes `did:web:host` wird unter `/well-known/did.json` aufgelöst, ein `did:web:host:pfad:segment` unter `/pfad/segment/did.json`. So können mehrere Instanzen einen Host teilen, ohne dass alle DIDs auf dasselbe Dokument zeigen; auch der Föderationsversand adressiert einen Peer unter dessen eigenem Pfadpräfix (Kapitel 08).

Die zugehörigen privaten Schlüssel liegen **ausschließlich** in einem PKCS#11-Token und verlassen es nie (ADR-1). Es gibt bewusst keinen Software-Fallback: Sind Modulpfad, Token-Label oder PIN falsch, wird der Prozess nicht gesund. Ein fehlendes Token ist dabei von einer Fehlkonfiguration unterschieden. Der Prozess wartet auf das Token, weil bei einer frischen Installation die Provisionierung noch laufen kann (Kapitel 10).

Die Schlüssel sind nach Zweck getrennt (alle ECDSA P-256):

Standard-Label	Zweck
<code>dcs-did</code>	Instanz-Identität: JAdES-Signaturen der Föderation, DID-Challenge-Response
<code>dcs-vc</code>	Lifecycle-, Provenance- und Federation-Agreement-Credentials (bewusst vom Identitätsschlüssel getrennt)
<code>dcs-oid4vp-jar</code>	Signieren der OID4VP-Request-Objekte
<code>dcs-c2pa</code>	COSE-Signaturen der C2PA-Manifeste (Kapitel 07)
<code>dcs-ecdh</code>	Schlüsselvereinbarung: entpackt die Inhaltsschlüssel der Artefaktverschlüsselung (ADR-28)

Der Schlüsselvereinbarungs-Schlüssel ist eine harte Startabhängigkeit. Beim Start wickelt die Instanz einen Testschlüssel gegen den im DID-Dokument publizierten `keyAgreement`-Eintrag und packt ihn mit dem HSM wieder aus. Schlägt das fehl, passen publizierter und tatsächlicher Schlüssel nicht zusammen oder dem Token fehlt die Ableitungsfähigkeit. In beiden Fällen wäre kein gespeichertes Artefakt lesbar, und die Instanz startet nicht.

Einen Vertrags-Signaturschlüssel hält die Instanz bewusst nicht: Vertragssignaturen erzeugt ausschließlich der Signatar mit dem eigenen Schlüssel (ADR-12/ADR-20, Kapitel 06).

Schlüsselrotation erfolgt über versionierte Labels; alte Versionen bleiben im Token, damit historische Signaturen verifizierbar bleiben. Welche Schlüssel die Instanz hält, mit welchem Zweck und in welcher aktiven Version, ist über eine Inventar-Abfrage für den Sys. Administrator sichtbar. Die Rotation selbst ist ein Betriebsverfahren, keine API-Handlung (siehe Key-Management-Konzept und den Deployment-Leitfaden).

Weil `pdf-core` nie einen Signer hält, liefert es nur die zu signierenden Daten zurück; signiert wird ausschließlich im Backend.

5.4 Ausstellervertrauen: zweckgebunden, organisationsgebunden, mechanismusoffen

Ob ein Credential akzeptiert wird, entscheidet nicht eine einzige Frage („ist die Signatur dieses Ausstellers in Ordnung?“), sondern drei getrennte (ADR-31):

Zweck. Jeder Aussteller wird für eine explizite Menge von Zwecken zugelassen:

Zweck	Bedeutung
login	Seine Credentials dürfen eine Session auf dieser Instanz begründen
peer	Seine Credentials werden in einer Signatur-Ceremony akzeptiert, und seine Aussage über die Vollmacht einer Gegenpartei wird geglaubt
pid	Er darf die Identität einer natürlichen Person attestieren

Ein Eintrag ohne Zweckangabe wird beim Laden abgewiesen. Ihn stillschweigend als „alle Zwecke“ zu lesen wäre genau die Vermischung, die diese Trennung beseitigt. Welche Aussteller welchen Zweck erhalten, ist Betreiberentscheidung; in Produktion sind mehrere Login-Aussteller normal.

Organisation. Ein Aussteller darf nur Organisationen attestieren, die in seinem eigenen Eintrag stehen. Ein Credential, dessen `organization` dort fehlt, wird unabhängig vom Zweck abgelehnt, so dass ein vertrauenswürdiger Aussteller nicht für eine Partei sprechen kann, die ihm niemand zugestanden hat. Ist ein Aussteller selbst die Mandantenverwaltung seines Deployments, deklariert er einen expliziten Platzhalter; eine fehlende Liste bedeutet niemals „beliebig“. PID-Aussteller sind ausgenommen, weil ein Identitätsnachweis eine Person attestiert und keine Organisation.

Mechanismus. Jeder Aussteller deklariert, wie sein Verifikationsschlüssel aufgelöst wird:

Mechanismus	Auflösung
jwks	Im Eintrag hinterlegte Schlüssel, über die Schlüssel-ID zugeordnet
x5c	Zertifikatskette im Credential-Header, verifiziert gegen konfigurierte Vertrauensanker
did:jwk	Schlüssel aus dem Ausstellerbezeichner selbst dekodiert
did:web	Schlüssel aus dem DID-Dokument des Ausstellers geholt
orcb	An einen konfigurierten ORCB-Flow delegiert

Ein Mechanismus, den der laufende Stand nicht auflösen kann, wird **beim Laden** abgewiesen. Ein Deployment erfährt von einer nicht unterstützten Trust-Konfiguration beim Start, nicht wenn die erste Wallet ankommt. Neue Registrierungsverfahren erreichen den Verifier über `orcb` ohne Codeänderung.

Der `x5c` -Pfad ist bewusst streng: Ohne konfigurierte Vertrauensanker wird ein Credential mit Zertifikatskette **abgewiesen** und nie seinem eigenen eingebetteten Zertifikat geglaubt. Eine gültige Kette allein genügt außerdem nicht. Das Leaf-Zertifikat muss den Aussteller auch benennen und darf kein für einen anderen Zweck ausgestelltes Zertifikat sein, damit ein gewöhnliches TLS-Zertifikat desselben Hosts nicht plötzlich Credentials signieren kann.

Über der Trust-Konfiguration liegt eine **Autorisierungs-Policy** als eigenes Artefakt. Sie formuliert die Regeln, nach denen aus Zweck, Organisation und Mechanismus eine Entscheidung wird. Eine Policy, die sich nicht übersetzen lässt, stoppt den Prozess beim Start; eine, die sich zur Laufzeit nicht auswerten lässt, verweigert. Sie rangiert über dem Trust-Dokument: Eine Regel, die alles zulässt, hebt jeden Eintrag darin auf. Deshalb werden beide von der Startup-Attestierung erfasst (Kapitel 09).

5.5 Trust-Modell der Föderation im Überblick

Ob eine fremde DCS-Instanz als Peer akzeptiert wird, entscheidet der **Federation Trust Gate** (ADR-19). Er wird auf beiden Pfaden konsultiert, beim Versand eines Vertrags an einen Peer wie bei dessen Empfang, und besteht aus zwei Schichten:

1. **Agreement-Credential.** Jede Instanz publiziert unter `/.well-known/dcs-agreement-credential.json` ein selbstsigniertes Verifiable Credential, dessen `termsOfUse`-Hash das im Binary eingebettete Föderationsregelwerk benennt (abrufbar unter `/.well-known/dcs-federation-rules.md`). Der Gate lädt das Credential des Peers, prüft dessen Signatur gegen den dedizierten VC-Schlüssel im DID-Dokument des Peers, verlangt, dass Aussteller-Hostname und Bezugsquelle übereinstimmen, und vergleicht den Regelwerk-Hash mit dem eigenen.
2. **Policy-Endpunkt.** Anschließend wird der lokale Policy-Decision-Point (`DCS_TRUST_PDP_URL`) mit Peer, Credential, Richtung und Zielzustand konsultiert. Er ist die alleinige Autorisierungsinstanz für Peers: Peer-Vertrauen ist Policy, keine Datenbanktabelle.

Der Gate arbeitet **fail-closed**. Ein nicht gesetzter, nicht erreichbarer oder nicht mit 2xx antwortender Policy-Endpunkt verweigert genauso wie ein explizites Deny. Jede Ablehnung wird als Incident im Audit- und Compliance-Subsystem festgehalten. Den vollständigen Föderationsablauf und die Prüfung der Handlungsvollmacht hinter einer Peer-Signatur beschreibt Kapitel 08.

06 Signaturen

Dieses Kapitel beschreibt, wie ein Vertrag im DCS elektronisch signiert wird: die wallet-getriebene Signatur-Ceremony, die Signaturniveaus, die beiden Signaturformate PAdES (mensenlesbares PDF) und JAdES (maschinenlesbares JSON-LD), die Validierung über den EU Digital Signature Service (DSS) und die Zeitstempelung.

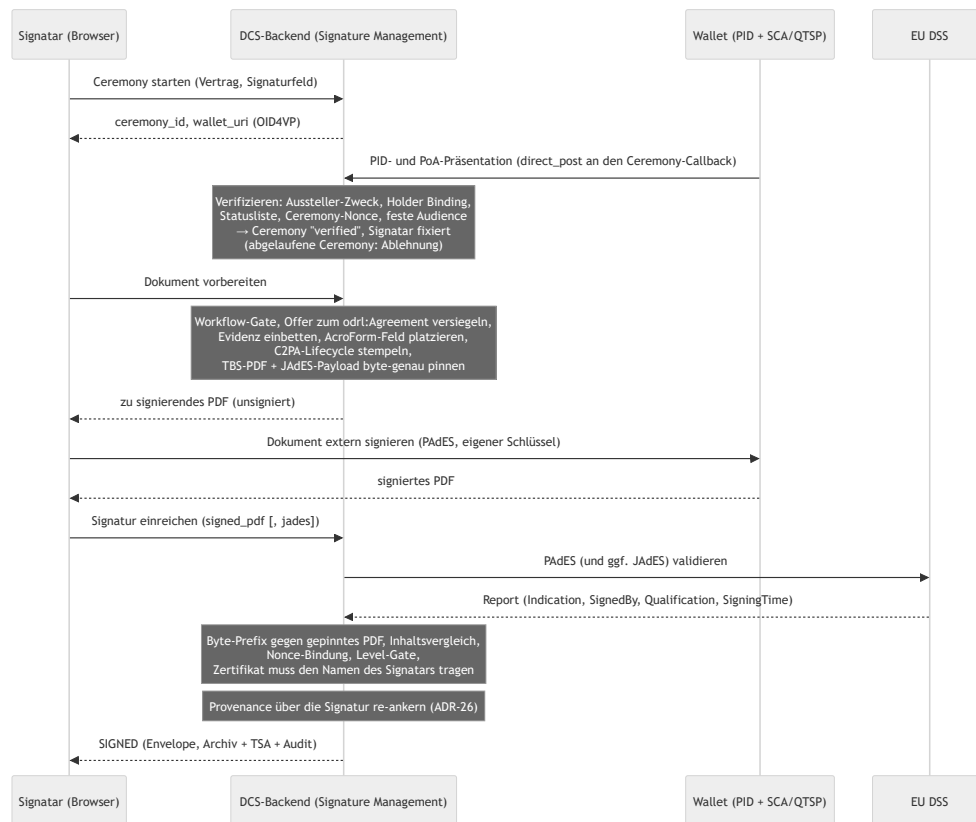
Das tragende Prinzip (ADR-12, ADR-3, ADR-20): **Das DCS signiert Verträge nie selbst und hält keinen Vertrags-Signaturschlüssel des Signatars.** Es bereitet das zu signierende Dokument vor, der Signatar signiert extern mit dem eigenen Schlüssel, und das DCS validiert und finalisiert das Ergebnis. Nur so ist die alleinige Kontrolle des Signatars über seinen Schlüssel (eIDAS Art. 26) überhaupt erfüllbar. Der Annahmepfad ist gehärtet (ADR-20): Was das DCS akzeptiert, muss byte-genau auf dem selbst vorbereiteten Dokument aufbauen, an die Ceremony gebunden sein und ein Zertifikat tragen, das den verifizierten Signatar benennt.

6.1 Beteiligte Komponenten

Komponente	Rolle
Signature Management (Backend)	Ceremony-Verwaltung, Dokumentvorbereitung, Annahme und Validierung externer Signaturen, Compliance-Sicht
Wallet + SCA/QTSP des Signatars	Legt das Identitätscredential vor, holt das Dokument, erzeugt die PAdES-Signatur mit dem Schlüssel des Signatars
pdf-core	Deterministisches Rendering des Basis-PDF, Inhaltsvergleich, Provenance-Re-Anchor; hält nie einen Signer (Kapitel 07)
EU DSS	Externer AdES-Validator (ETSI EN 319 102-1) für eingereichte PAdES- und JAdES-Signaturen
ORCE + TSA	RFC-3161-Zeitstempel für den Archiv-Eintrag; das Backend verifiziert die Zeitstempel-Antwort gegen das hinterlegte TSA-Zertifikat
Frontend (Signierliste, Secure Contract Viewer, Signature Compliance Viewer)	Führt den Signatar durch die Ceremony; zeigt Signaturstatus, Integritäts- und DSS-Befunde

6.2 Die Signatur-Ceremony

Signieren darf nur, wer zuvor in einer **Ceremony** seine Identität als natürliche Person nachgewiesen hat: einer OID4VP-Präsentation, die an den Vertrag und ein deklariertes Signaturfeld gebunden ist. Ohne abgeschlossene Ceremony lehnt das DCS jede Dokumentvorbereitung und Signaturannahme mit einem typisierten Fehler ab.



Die Schritte im Einzelnen:

1. **Ceremony starten.** Bindet Vertrag und Signaturfeld, liefert eine Ceremony-ID und eine Wallet-URI für die Präsentation. Der Status ist pollbar (pending , verified , expired , failed).
2. **Identitätsvorlage.** Die Wallet legt ihr Präsentationstoken mit Identitäts- **und** Vollmachts-Credential direkt am Callback der Ceremony vor, autorisiert über die nicht erratbare Ceremony-ID. Das Backend verifiziert die Präsentation vollständig (Aussteller und Zweck, Holder Binding, Statusliste, Bindung an die Nonce der Ceremony, feste Audience) und fixiert daraus den Signatar. Organisation und Rollen des Vollmachts-Credentials fließen als Signaturevidenz ein und werden an der Ceremony aufbewahrt, weil die Gegenseite sie später nachprüft (Kapitel 08). Die Ceremony-Frist wird am Callback durchgesetzt: Eine Präsentation an einer abgelaufenen Ceremony wird abgelehnt.
3. **Vorbereitung.** Das DCS durchläuft das Workflow-Gate (Kapitel 04), versiegelt das verhandelte Angebot zum odr1:Agreement , prüft Abschluss und SHACL-Konformität, wertet die ODRL-Bedingungen aus, bettet die Signaturevidenz **innerhalb** des Byte-Bereichs ein, den die spätere Signatur abdecken wird (embed-then-sign, ADR-3), stempelt die C2PA-Lifecycle-Assertion in das Basis-PDF und platziert das AcroForm-Signaturfeld. Die exakten zu signierenden Bytes, das TBS-PDF und die kanonische JAdES-Payload, werden zusammen mit den Finalisierungs-Metadaten (Content-Hash, Renderer-Version, gefordertes Signaturniveau des Felds) an der Ceremony **gepinnt** (ADR-20). Alle Seiteneffekte passieren genau einmal, hier. Der Inhalt, den der Signatar signiert, ist ab jetzt eingefroren.
4. **Externe Signatur.** Der Signatar signiert das PDF mit seinem eigenen Schlüssel, per Wallet/QTSP oder in einem Desktop-Signer im vorbereiteten Feld.
5. **Einreichung und Finalisierung.** Ein reiner Validierungs- und Aufzeichnungsschritt ohne erneute Vorbereitung. Das DCS prüft die Einreichung (6.4), **re-ankert die Provenance über die Signatur**, zeichnet den Signature-Envelope auf, erzeugt den Archiv-Eintrag mit Snapshot, Notarisierung und

Zeitstempel und setzt den Vertrag auf `SIGNED`. Der Verbrauch der Ceremony wird **atomar** in derselben Transaktion wie die Finalisierung markiert, so dass zwei konkurrierende Einreichungen derselben Ceremony nicht beide finalisieren können (ADR-20).

Signatur und Provenance-Re-Anchor (ADR-26). C2PA-Hard-Binding und PAdES-Signatur wollen beide „das Letzte im Dokument“ sein: Das Lifecycle-Manifest bindet die gesamte Datei, die Signatur bindet alles bis zu ihrem eigenen Byte-Bereich. Das DCS löst den Konflikt in fester Reihenfolge auf. Das Lifecycle-Manifest wird **vor** der Signatur geschrieben, sodass die Signatur die Provenance mit abdeckt; nach der validierten Einreichung hängt die Finalisierung ein **reines Provenance-Update-Manifest** an, dessen Hard-Binding die signierten Bytes abdeckt. Das ist ein inkrementelles Update: Der Byte-Bereich der Signatur bleibt unberührt, die Signatur verifiziert weiter in externen Werkzeugen. Archiviert und ausgeliefert wird das re-ankerte PDF. Der akzeptierte Preis: PDF-Reader und der DSS melden das Dokument als „nach der Signatur geändert“. Das ist eine zutreffende Aussage, deren einzige zulässige Ursache dieser Re-Anchor ist; die Verifikation weist genau das nach (Kapitel 07). Die Rangfolge ist entschieden: **Signaturintegrität geht dem C2PA-Hard-Binding vor**. Nichts darf je angehängt werden, was den Byte-Bereich der Signatur bricht.

Multi-Signer. Deklariert ein Vertrag mehrere Signaturfelder, muss für *jedes* Feld eine verifizierte Ceremony existieren, bevor die *erste* Signatur angebracht wird: Die Evidenz aller Signaturen muss im PDF stecken, bevor eine PAdES-Signatur es einfriert, denn ein nachträglich eingebetteter Anhang würde die standardkonforme Diff-Analyse verletzen. Ein bereits signiertes Feld kann nicht erneut signiert werden; der Wiederholungsschutz wird unter der Regenerierungssperre erneut geprüft, nachdem die Signaturlage frisch gelesen wurde.

QR-/Deep-Link-Variante: OID4VP Document Retrieval

Für vollständig wallet-getriebenes Signieren publiziert die Ceremony ein signiertes Request-Objekt nach dem EUDI-„wallet-driven-signer“-Profil: Signatur-Antworttyp, Dokument-Hashes und Abruforte, Signatur-Qualifier aus dem CSC-Vokabular. Angeboten werden **zwei** Dokumente: das zu signierende PDF und die gepinnte kanonische JSON-LD-Payload, sodass eine Ceremony beide Artefakte, PAdES und JAdES, über denselben Inhalt liefert. Die Wallet holt das Request-Objekt und die Dokumente, betreibt ihre eigene SCA/QTSP-Strecke und postet die signierten Dokumente per `direct_post` an den Callback, der denselben Validierungs- und Finalisierungspfad wie die direkte Einreichung durchläuft. Der Callback ist über die nicht erratbare Ceremony-ID autorisiert, denn der Aufrufer ist die Wallet des Signatars und keine eingeloggte Session; zusätzlich muss die Antwort die frische Request-Nonce der Ceremony binden (6.4). Das Request-Objekt trägt die Zertifikatskette der Instanz im Header und einen DNS-gebundenen Client-Identifizierer, der dem SAN dieses Zertifikats entspricht.

6.3 Signaturniveaus und -formate

Das geforderte Niveau ist eine Eigenschaft des Vertrags, nicht des Aufrufers. Jedes Signaturfeld deklariert es im Dokument; fehlt die Angabe oder ist das Feld nicht deklariert, gilt AES. Das Niveau wird bei der Vorbereitung gepinnt und bei der Einreichung gegen das **tatsächlich erreichte** Niveau durchgesetzt: Eine kryptographisch gültige AES auf einem QES-pflichtigen Feld wird abgelehnt, und aufgezeichnet wird das erreichte, nicht das angefragte Niveau (ADR-20). Der Vergleich läuft auf der Skala

SES < AES < QES; ein Wert, den das System nicht kennt, zählt wie SES. Das ist die strikte Lesart für das *angebotene* Niveau. Für ein *gefordertes* Niveau unterhalb von AES ist der Vergleich damit wirkungslos; wer eine Anforderung durchsetzen will, fordert AES oder QES.

Die Compliance-Prüfung bewertet das erreichte Niveau eigenständig und protokolliert das Ergebnis als Ereignis im Audit-Trail.

v1-Leistungsumfang (ADR-24). Die Signaturarchitektur ist nicht AES-limitiert. Welches Niveau eine Signatur erreicht, bestimmen das vorgelegte Credential und der dahinterstehende Vertrauensdienst. Mit den testgradigen v1-Providern (projekteigene CA, Test-Wallet, Demonstrations-TSA, DSS-Testinstanz) ist das Ergebnis eine wallet-basierte AES. QES ist aus dem v1-Liefergegenstand ausgenommen, weshalb Verträge mit gesetzlichem Schriftformerfordernis in v1 nicht demonstrierbar sind. Die Anbindung eines qualifizierten Vertrauensdiensteanbieters und einer produktiven Wallet hebt denselben Fluss ohne Architekturänderung Richtung QES.

Was ein Signatur-Level tatsächlich aussagt, ist außerdem durch den vertrauten PID-Aussteller begrenzt. Der als Beispiel-Flow mitgelieferte PID-Aussteller läuft als eigenständiger Dritter mit eigener DID und eigener CA, führt aber keine Identitätsprüfung der Person durch; sein Credential ist folglich kein EUDI-PID und darf nicht als solches behandelt werden. Kettenprüfung, Zweckbindung und Statuslistenprüfung greifen unverändert (ADR-31).

Die beiden Formate:

- **PAdES** ist die Signatur des Signatars über das menschenlesbare PDF, sichtbar im AcroForm-Feld, extern erzeugt.
- **JAdES** (ETSI TS 119 182-1, Baseline-B) ist das Gegenstück über die maschinenlesbare Repräsentation: ein kompaktes JWS über die kanonisierte Form aus Vertrags-DID, Version und vollständigem JSON-LD-Dokument, mit Zertifikatskette und kritischem Signaturzeit-Header. Bei einer publizierten Wallet-Ceremony ist die JAdES des Signatars **Pflicht**, weil sie die Nonce-Bindung trägt; auf dem authentifizierten Einreichungspfad (Desktop-Signer) ist sie optional und wird, falls vorhanden, dennoch validiert.

Unabhängig davon signiert die **Instanz** jeden Vertrag, den sie an Peers verschickt, mit ihrem HSM-gestützten DID-Schlüssel als JAdES; die Empfängerinstanz verifiziert die Signatur und ihre Bindung an den `did:web`-Schlüssel des Absenders, bevor sie den Vertrag akzeptiert (Kapitel 08). Wichtig zur Abgrenzung: Diese Instanzsignatur ist eine Systemhandlung, keine AES-Signatur einer Person (ADR-17). Ein elektronisches **Siegel** einer Organisation ist als Richtung entschieden, aber nicht implementiert (ADR-21); der Maschinen-Rollenklasse bleibt jede Signaturfähigkeit verwehrt.

6.4 Validierung: interne Prüfungen plus EU DSS

Eingereichte Signaturen durchlaufen eine mehrstufige Annahmeprüfung (ADR-20). Jede Stufe ist ein hartes Abbruchkriterium mit typisiertem Fehler:

1. **Byte-Pinning.** Das eingereichte PDF muss byte-genau mit dem bei der Vorbereitung gepinnten TBS-PDF **beginnen**. Eine PAdES-Signatur ist ein inkrementelles Update, das nur Bytes anhängt, „dasselbe Dokument plus die eigene Signatur“ ist also exakt eine Byte-Prefix-Beziehung.

Manipulation am sichtbaren Inhalt scheitert hier, unabhängig davon, ob die Signatur selbst gültig wäre. Die JAdES-Payload wird genauso geprüft: Ihr Payload-Segment muss byte-gleich der gepinnten kanonischen Payload sein.

2. **Inhaltsvergleich.** Zusätzlich vergleicht pdf-core den sichtbaren Seiteninhalt des eingereichten Dokuments mit dem vorbereiteten. Beide Seiten sind Dokumente, die das Backend bereits hält; es wird nichts neu gerendert.
3. **Nonce-Bindung.** Bei einer publizierten Wallet-Ceremony muss die mitgelieferte JAdES im signaturgeschützten Header eine Nonce tragen, die der frischen Request-Nonce der Ceremony entspricht. Sie kann nicht entfernt oder ersetzt werden, ohne die JAdES selbst zu invalidieren. Wer nur die Ceremony-URL kennt, kann ohne den Schlüssel des Signatars keine annehmbare Antwort fälschen.
4. **Extern über DSS.** Das signierte PDF (und eine mitgelieferte JAdES) wird an die REST-Validierung des EU DSS übergeben. Der Report liefert die ETSI-Indication (`TOTAL-PASSED` , `INDETERMINATE` , `TOTAL-FAILED`), Sub-Indication, Qualification, das erkannte Format/Level, den Zertifikats-Subject und die Signierzeit, bei Mehrfachsignaturen bewusst der jüngsten, gerade eingereichten Signatur zugeordnet. **Ohne konfigurierten DSS nimmt der Einreichungspfad überhaupt keine Signatur an;** auf den Lesepfad entfällt ohne DSS lediglich der DSS-Report.
5. **Level-Gate.** Das erreichte Niveau wird gegen das gepinnte geforderte Niveau des Felds gehalten. Für **AES** ist das Kriterium bewusst **nicht** `TOTAL-PASSED` , denn `TOTAL-PASSED` verlangt zusätzlich eine qualifizierte EU-Trust-List-CA, also eine QES-Eigenschaft. Für AES genügen kryptographische Integrität und die eindeutige Zuordnung zum Signatar; ein `INDETERMINATE` mit reiner Trust-Chain-Lücke wird akzeptiert, jeder Krypto- oder Integritätsfehler und jedes `TOTAL-FAILED` wird abgelehnt. Für **QES** kommt hinzu: `TOTAL-PASSED` **und** die DSS-Qualification eines qualifizierten Zertifikats mit QSCD.
6. **Sole-Control-Gate (Zertifikat ↔ Identität).** Das Signaturzertifikat wird direkt aus der CMS-Struktur des eingereichten PDFs gelesen. Das Byte-Pinning garantiert, dass es genau die eben eingereichte Signatur benennt, und sein Subject muss, normalisiert, dem Vor- und Nachnamen der verifizierten Identität der Ceremony entsprechen. Für QES ist dieser Abgleich zwingend (eIDAS Annex I); für AES ist er standardmäßig aktiv und abschaltbar nur als dokumentierte Betreiberentscheidung. Zusätzlich muss derselbe Signatar über alle eigenen Signaturen eines Vertrags hinweg dasselbe Zertifikat verwenden; ein Zertifikatswechsel mitten im Vertrag ist das Signal eines kompromittierten oder geteilten Schlüssels. Subject und Seriennummer des validierten Zertifikats werden für den Signature Compliance Viewer aufgezeichnet.

Ein konfigurierter, aber nicht erreichbarer DSS ist ein Fehler, keine stillschweigend übersprungene Prüfung. Eine ungültige JAdES wird abgelehnt, nie stillschweigend mitaufgezeichnet.

Nachträgliche Prüfung eines signierten Vertrags

Unabhängig von der Annahme lässt sich ein signierter Vertrag jederzeit erneut prüfen. Diese Prüfung liest die im PDF eingebettete Signatur-Evidenz, die Signing-Summary-Credentials, und **verifiziert sie zuerst gegen den Schlüssel, den ihr Aussteller für Aussagen publiziert**, bevor irgendetwas aus ihnen verwendet wird. Darauf setzen drei Auswertungen auf:

- **Signatarbindung:** Der pseudonyme Signatar-Identifizier aus der verifizierten Evidenz wird gegen die aufgezeichneten Signaturen gehalten; eine Abweichung ist ein Befund. Der im Credential mitgeführte Hash der Wallet-Bindung wird ausgewiesen, aber nicht gegengerechnet.

- **SHACL-Drift:** Die Validierung wird gegen exakt die Shapes-Version wiederholt, unter der signiert wurde, und der Befund-Hash gegen den vom Aussteller versiegelten verglichen. Eine Abweichung bedeutet, dass sich die Vertragsdaten seit der Signatur verändert haben. Ein Versionssprung des Hubs erzeugt hier keinen Fehlalarm, weil die Evidenz an ihre Version gepinnt ist (ADR-8).
- **DSS-Bewertung** wie oben, sofern konfiguriert.

6.5 Zeitstempelung

Mit Erreichen von `SIGNED` entsteht der Archiv-Eintrag des Vertrags; dessen Notarisierungs-Evidenz wird mit einem **RFC-3161-Zeitstempel** versehen. Die Zeitstempelanfrage läuft über ORCE, das den eigentlichen Austausch mit dem TSA-Anbieter abwickelt; das Backend verifiziert die zurückkommende Antwort gegen das vertrauenswürdige TSA-Zertifikat. Ein TSA-Wechsel bedeutet deshalb zweierlei: den TSA-Flow in ORCE umstellen und den Vertrauensanker austauschen. Dieselbe Zeitstempel-Maschinerie nutzen die Audit-Checkpoints; ist die TSA vorübergehend nicht erreichbar, wird der Zeitstempel nachgeholt (Kapitel 09).

6.6 Sichtbarkeit: Viewer, Verifikation, Widerruf

- **Signierliste.** Listet für den Signatar ausstehende (`APPROVED`), abgeschlossene (`SIGNED / ACTIVE`) und widerrufene (`REVOKED`) Verträge. Ein widerrufener Vertrag verschwindet nicht aus der Liste, sein Status wird gezeigt. Jede Zeile klappt in die Signaturdetails auf. Jede Signaturanwendung hinterlässt zudem einen Audit-Eintrag mit Signatar, verwendetem Credential-Niveau und Zeitstempel.
- **Secure Contract Viewer.** Der Signatar öffnet den Vertrag aus der Signierliste, stößt die Integritätsprüfung an (Neuaufbau des Basis-PDF und Hash-Abgleich gegen den gespeicherten Stand, plus C2PA- und Widerrufsprüfung, die unabhängige Validierung aus Kapitel 01) und durchläuft anschließend die Ceremony. Die C2PA-Provenance-Kette ist über einen eigenen Abruf einsehbar.
- **Signature Compliance Viewer.** Pro Signatur: Signatar, Feld, gefordertes und erreichtes Credential-Niveau samt Qualified-Kennzeichen, Status und Zeitpunkte, Subject und Seriennummer des Signaturzertifikats (Sole-Control-Evidenz) sowie die kryptographischen Bindungen aus dem eingebetteten Signing-Summary-Credential (Content-Hash, PDF-Hash, Wallet-Bindungshash, Ceremony-ID) plus Integritätsbefunde und, falls konfiguriert, den DSS-Report.
- **Widerruf.** Setzt eine Signatur auf `REVOKED` ; der Envelope behält Zeitpunkte von Signatur und Widerruf, der Vorgang ist auditiert. Bei einem grenzüberschreitenden Vertrag wird der Widerruf unmittelbar an die Gegenpartei verschickt, die den Zustand übernimmt (Kapitel 08).

6.7 Bekannte Grenzen

- **Extraktion des eingebetteten JSON-LD.** Der Abgleich zwischen sichtbarem Text und maschinenlesbarem Inhalt löst das eingebettete Dokument über die letzte Definition des Anhangsobjekts im Dateiinhalt auf, nicht über das Querverweisverzeichnis des PDFs. Ein gezielt präpariertes Dokument kann diese Auflösung von der Auflösung eines PDF-Readers abweichen lassen.

- **Mehr als zwei Parteien.** Der Nachweis der Gegenseiten-Signatur wird je Vertrag und nicht je Signaturfeld vorgehalten. Verträge mit drei oder mehr Parteien werden deshalb nicht ausgerollt: Das System verweigert die Ausführung, statt eine unvollständige Signaturlage als vollständig zu behandeln (Kapitel 04).
- **„Nach der Signatur geändert“.** Prüfprogramme melden ein DCS-signiertes Dokument als nach der Signatur verändert. Die Signatur selbst bleibt gültig und unverändert prüfbar; das Verhalten ist die bewusste Folge des Provenance-Re-Anchors (ADR-26) und wird von der eigenen Verifikation nachgewiesen statt ignoriert (Kapitel 07).

07 Dokumente und Provenance

Dieses Kapitel beschreibt, wie das DCS aus dem maschinenlesbaren Vertrag (JSON-LD) das menschenlesbare Vertragsdokument (PDF/A-3) erzeugt, warum dieses Rendering deterministisch ist, wie jede Instanz unabhängig prüfen kann, dass beide Repräsentationen übereinstimmen, wie die Herkunft des Dokuments über C2PA-Manifeste nachvollziehbar bleibt und wie die Artefakte im Speicher geschützt und löschar sind.

Der Leitgedanke: Das PDF ist kein Ausdruck des Vertrags, sondern eine zweite, jederzeit gegen die maschinenlesbare Form verifizierbare Repräsentation derselben Vereinbarung.

7.1 Beteiligte Komponenten

Komponente	Rolle
pdf-core	Eigenständiger, zustandsloser Dienst; alleiniger Eigentümer aller Byte-Arbeit am PDF: deterministisches Kompilieren, inkrementelle Amendments, Provenance-Re-Anchor, Extraktion des eingebetteten JSON-LD, C2PA-Einbettung, Re-Render- und Inhaltsvergleich. Das Backend parst nie selbst PDF-Bytes
Backend / PDF-Generierung	Orchestriert: hört auf Lifecycle-Ereignisse, lässt pdf-core rendern bzw. amendieren, stellt Lifecycle-Credentials aus, legt das Ergebnis verschlüsselt ab und führt den PDF-Zustand je Vertrag/Vorlage (CID, Renderer-Version, C2PA-Zustand, Payload-Hash)
Artefaktspeicher	Verschlüsselnde Schicht über IPFS: jedes Artefakt wird vor dem Schreiben verschlüsselt, jeder Lesevorgang entschlüsselt authentifiziert
C2PA-Manifest-Dienst	Öffentlicher, unauthentifizierter Abruf des C2PA-Manifest-Stores eines Vertrags, das öffentliche Gegenstück zum DID-Dokument
HSM (über das Backend)	Signiert die COSE-Struktur der C2PA-Manifeste und entpackt die Inhaltsschlüssel. pdf-core hält kein Schlüsselmaterial: Es baut die zu signierende Struktur, das Backend signiert
Statuslist-Dienst	Wird bei der Verifikation für den Live-Widerrufsstatus der in den Manifesten eingebetteten Lifecycle-Credentials befragt

7.2 Deterministisches Rendering

pdf-core garantiert: **Dieselbe JSON-LD-Payload erzeugt immer byte-identischen Seiteninhalt.** Dafür sorgen mehrere bewusste Entscheidungen:

- Der Render-Zeitpunkt ist eine feste Epoche, nicht die Wanduhr. Die vertrauenswürdige Vertragszeit ist der Zeitstempel der PAdES-Signatur (Kapitel 06).
- Die eingereichte Payload wird **verbatim** als Anhang in das PDF/A-3-Dokument eingebettet, exakt die übermittelten Bytes und nie eine re-kanonisierte Form. Der SHA-256 dieser Bytes ist die Content-Adresse des Dokuments; jeder Prüfer kann ihn aus dem Anhang nachrechnen.
- Die Schrift ist im Dokument eingebettet (PDF/A-Konformität), Layout und Struktur (Tagged PDF, Outline) sind vollständig aus der Payload abgeleitet.

Weil der sichtbare Seiteninhalt eine reine Funktion der eingebetteten Payload ist, lässt sich die Übereinstimmung von menschen- und maschinenlesbarer Form jederzeit beweisen: Payload extrahieren, neu kompilieren, Seiteninhalte vergleichen.

Compiler und Contract Fields

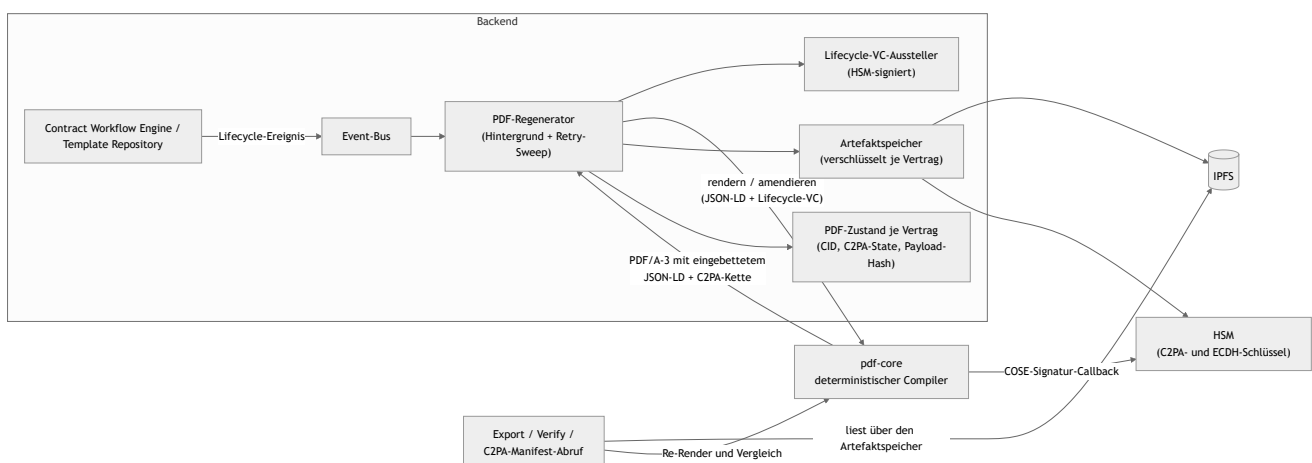
Der Compiler arbeitet auf dem kanonischen Vertragsmodell (Kapitel 03):

- `dcs:documentStructure` beschreibt die menschenlesbare Struktur, einen Layout-Baum über geordnete Blöcke, dessen Reihenfolge über JSON-LD-Listen stabil ist.
- `dcs:contractFields` ist die flache Liste der ausfüllbaren Felder. Klauseln, `dcs:contractData` und ODRL-Operanden referenzieren dieselben Feld-IDs. Authoring, Rendering und Policy-Auswertung lösen ein Feld ausschließlich über seine `@id` auf, ohne Vorlagen-Schnappschuss.
- `dcs:signatureFields` erzeugt AcroForm-Signaturfelder für die spätere PAdES-Signatur.
- Semantische Zusatzdaten, die nicht Teil der Dokumentstruktur sind (etwa ODRL-Policies), werden nicht gerendert, aber unverändert im eingebetteten JSON-LD mitgeführt. Die Maschinenlesbarkeit geht beim Rendern nie verloren.

Mehrfache Signaturen sind konstruktionsbedingt **sequenziell**: PDF/A-3 verlangt, dass jede Signatur alle vorherigen Bytes abdeckt. Ein paralleles, reihenfolgeunabhängiges Signieren mit anschließendem Zusammenführen ist mit dem Standard nicht vereinbar; Änderungen nach einer Signatur erfolgen als inkrementelle Revisionen über den signierten Bytes.

7.3 Der Rendering- und Provenance-Fluss

PDFs entstehen nie auf Zuruf im Request-Pfad, sondern **ereignisgetrieben im Hintergrund**. Jedes Lifecycle-Ereignis eines Vertrags oder einer Vorlage (Anlegen, inhaltliche Änderung, Zustandswechsel, Verhandlungsschritt) läuft über den Event-Bus in den Regenerator. Der stellt für den neuen Zustand ein Lifecycle-Verifiable-Credential aus, lässt pdf-core das PDF rendern bzw. per Amendment fortschreiben (die C2PA-Kette wächst, sie beginnt nie neu), legt das Ergebnis verschlüsselt ab und aktualisiert den PDF-Zustand.



Der Export bedient sich immer aus diesem Bestand: Er liefert das aktuelle PDF und **wartet**, wenn nach der letzten Änderung noch eine Regeneration läuft. Er gibt nie einen veralteten Stand und rendert nie selbst. Ein bereits PAdES-signiertes PDF ist **eingefroren**: Es wird unverändert ausgeliefert, auch wenn der

Lifecycle-Zustand danach weiterläuft. Die letzte Lifecycle-Assertion vor der Signatur wird bereits vor dem Signieren eingebettet, sodass die Signatur die Provenance mit abdeckt. Die einzige Revision **nach** der Signatur ist der Provenance-Re-Anchor der Signatur-Finalisierung (ADR-26, Kapitel 06): ein reines C2PA-Update-Manifest, dessen Hard-Binding die signierten Bytes abdeckt, angehängt als inkrementelles Update, das den Byte-Bereich der Signatur unberührt lässt. Danach mutiert nichts mehr, denn jede weitere Änderung gälte standardkonformen PAdES-Validatoren als unerklärte Manipulation.

7.4 C2PA-Manifeste

Jede erzeugte PDF-Version trägt einen C2PA-Manifest-Store (JUMBF), der den gesamten sichtbaren Seiteninhalt abdeckt. Die Manifeste **stapeln** sich über den Lebenszyklus: Jeder Zustandswechsel hängt eine Lifecycle-Assertion an (Zustände wie `draft`, `active`, `amended`, `suspended`, `terminated`, `expired`), zusammen mit einem Verifiable Credential der Instanz, das den Zustandswechsel, den Asset-Hash und den Zeitpunkt bezeugt und über eine Statusliste widerrufbar ist. Die COSE-Signatur jedes Manifests entsteht über den Signatur-Callback im Backend mit dem HSM-gehaltenen C2PA-Schlüssel der Instanz; die zugehörige Zertifikatskette ist konfiguriert und wird pdf-core als öffentliches Material bereitgestellt.

Ein signierter Vertrag trägt ein Manifest mehr als ein unsignierter: das Lifecycle-Manifest, auf das sich die Signatur festlegt, und das Re-Anchor-Manifest, das sich auf die Signatur festlegt (ADR-26). Die Kette liest sich in genau dieser Reihenfolge.

Die Manifestkette ist öffentlich abrufbar: Der C2PA-Endpunkt liefert die rohen Manifest-Store-Bytes, auf Wunsch eine JSON-Aufzählung der Kette mit Manifest-Labels und Lifecycle-Assertions. Der Endpunkt ist bewusst unauthentifiziert, denn ein externer Prüfer muss die Herkunftskette eines signierten Vertrags ohne Konto auflösen können. Damit hängt die Sichtbarkeit einer Kette zugleich an der Verfügbarkeit ihres Inhaltsschlüssels (7.6).

Ausgeliefert wird die Provenance doppelt (ADR-4): eingebettet als JUMBF im PDF und über diesen Remote-Endpunkt.

7.5 Unabhängige Validierung: menschen- vs. maschinenlesbar

Die Konsistenz zwischen maschinen- und menschenlesbarer Form wird an drei Stellen erzwungen bzw. prüfbar gemacht:

1. **Auf Abruf** über die Verify-Endpunkte: Das gespeicherte PDF wird geladen, sein eingebettetes JSON-LD extrahiert, mit demselben deterministischen Compiler neu gerendert und der Seiteninhalt gegen den gespeicherten Stand verglichen. Zusätzlich werden C2PA-Signatur, das eingebettete Lifecycle-Credential und dessen Live-Widerrufsstatus geprüft.
2. **Beim Empfang eines Peer-PDFs** (Kapitel 08): Bevor eine Instanz einen von einer anderen Instanz verschickten Vertrag übernimmt, verlangt sie, dass dessen Seiteninhalt exakt der Re-Render seiner eigenen eingebetteten Payload ist. Der Vergleich betrachtet nur die Seiteninhalte, sodass legitime C2PA-, Signatur- und Amendment-Schichten des Absenders nicht anschlagen. Echte Divergenz zwischen Text und Daten führt zur Ablehnung.
3. **Bei der Signaturannahme** (Kapitel 06): Das eingereichte Dokument wird gegen das vorbereitete inhaltlich verglichen.

Damit ist keine Instanz auf die Aussage der ausstellenden Instanz angewiesen. Jede Partei rechnet die Übereinstimmung selbst nach.

Bei einem PDF mit angehängten Revisionen spielt die Verifikation jede Revision deterministisch nach und unterscheidet dabei, **welche Art** Revision sie vor sich hat: Eine Revision, die ein neues Lifecycle-Credential trägt, wird als Amendment reproduziert; eine Revision mit unveränderter Payload und ohne neues Credential ist der Provenance-Re-Anchor nach der Signatur und wird genauso reproduziert, wie sie erzeugt wurde. Ein signiertes PDF gilt daher nur dann als gut, wenn das einzige nach der Signatur Angehängte byte-genau die Provenance ist, die diese Instanz selbst produziert hat. Die „nach der Signatur geändert“-Meldung der PDF-Reader wird nachgewiesen, nicht ignoriert.

Das Verifikationsergebnis ist bewusst ehrlich geschnitten. Der Text/Daten-Abgleich und der C2PA-Check sind unabhängig benannte Prüfungen. Trägt das PDF (noch) keine PAdES-Signatur oder prüft dieser Pfad sie nicht kryptographisch nach, meldet das Ergebnis für die PDF-Signatur ausdrücklich „nicht verfügbar“ statt einer scheinbar bestanden Prüfung; die kryptographische PAdES/JAdES-Validierung ist Gegenstand der Signaturverwaltung (Kapitel 06). Zusätzlich benennt das Ergebnis seine **Fehlerklasse** direkt, statt sie den Aufrufer aus Kombinationen von Ja/Nein-Feldern erraten zu lassen (ADR-30):

Klasse	Bedeutung
content_hash_mismatch	Manifest vorhanden, aber der Inhalt weicht vom eingebetteten JSON-LD ab
artifact_not_authentic	Die gespeicherten Bytes haben die authentifizierte Entschlüsselung nicht bestanden; sie sind nicht die, die diese Instanz geschrieben hat
verification_failed	Eine andere Prüfung ist fehlgeschlagen
(leer)	Die Prüfung ist bestanden

7.6 Speicherung: Verschlüsselung, Manipulationsnachweis, Löschung

Jedes Artefakt wird **vor** dem Schreiben verschlüsselt (ADR-28). Der Inhaltsschlüssel ist pro **Löschbereich** zufällig gezogen: einer je Vertrag (dessen PDFs, Archiv-Snapshot und Audit-Ereignisrumpfe), einer je Vorlage, einer instanzweit für Checkpoints und instanzbezogene Reports. Der Bereichsbezeichner geht als zusätzliche authentifizierte Angabe in das Chifftrat ein, sodass ein Artefakt nicht unbemerkt einem anderen Bereich untergeschoben werden kann.

Der Inhaltsschlüssel existiert im Ruhezustand **nur gewickelt**: Jede haltende Instanz wickelt ihn gegen den eigenen, im DID-Dokument publizierten Schlüsselvereinbarungs-Schlüssel; der passende private Schlüssel ist im PKCS#11-Token nicht extrahierbar. Auspacken erfordert also das HSM genau dieser Instanz. Bei jeder Vertragssendung an einen Peer reist zusätzlich eine für dessen publizierten Schlüssel gewickelte Kopie mit. Der Empfänger übernimmt sie einmal, packt sie aus, wickelt sie gegen den eigenen Schlüssel neu und ignoriert die Kopie bei allen späteren Sendungen. Beide Instanzen halten danach denselben Inhaltsschlüssel, jede Kopie nur vom eigenen HSM zu öffnen.

Drei Folgen sind für das Verständnis wichtig:

- **Manipulation am Speicher ist ein Prüfergebnis, kein Serverfehler (ADR-30).** Scheitert die authentifizierte Entschlüsselung, kann das keine Schlüsselverwaltungsstörung sein, denn Schlüssel und Bereichsangabe stammen aus dem eigenen Bestand. Es bedeutet, dass die gespeicherten Bytes

nicht die geschriebenen sind. Genau das ist die Antwort auf die Frage „ist dieses Artefakt unversehrt?“, und sie lautet „nein“. Was genau verändert wurde, ist dabei nicht ermittelbar: Klartext wird nie aus unauthentifiziertem Chiffre abgeleitet.

- **CIDs zweier Instanzen unterscheiden sich** für denselben Vertrag, weil jede Seite dasselbe PDF unter eigenem Zufallswert verschlüsselt. Das ist folgenlos: Über die Peer-Schnittstelle wandert nie eine CID, Verträge reisen als rohe Bytes, und jede Seite adressiert nur ihren eigenen Speicher.
- **Löschung ist Schlüsselvernichtung.** Erasure zerstört die gewickelten Kopien und behält die Zeile als Zerstörungsnachweis (Kapitel 04 und 09). Danach liefern Export, Verifikation und Bundle-Abwurf dieses Vertrags eine definierte „Inhalt gelöscht“-Antwort, als Nicht-gefunden und nie als interner Fehler; Listen- und Metadatensichten arbeiten weiter.

Bekannte Grenze. Die Schlüsselvernichtung entfernt die Vertrags-Chiffre nicht aus dem IPFS-Knoten; entfernt werden lediglich die Archiv-Snapshots. Solange eine Datenbanksicherung existiert, die vor der Vernichtung liegt, und der Schlüsselvereinbarungs-Schlüssel im HSM weiterlebt, ließe sich aus dieser Sicherung wieder ein lesbarer Schlüssel gewinnen. Die Löschzusage reicht damit genau so weit wie das Aufbewahrungsfenster der Sicherungen; das Nachziehen der Löschmarkierungen nach einer Wiederherstellung ist Bestandteil des Backup-Leitfadens.

7.7 Schnittstellen und Artefakte

Backend (authentifiziert, rollenbasiert; vgl. Anhang A):

Endpunkt	Zweck
GET /pdf/export/contract/{did} , GET /pdf/export/template/{did}	Aktuelles PDF (PDF/A-3, eingebettetes JSON-LD, C2PA-Kette)
GET /pdf/verify/contract/{did} , GET /pdf/verify/template/{did}	Text/Daten- und Provenance-Verifikation inklusive Fehlerklasse
GET /contract/export/{did}	ZIP-Bundle: JSON-LD, signiertes PDF, C2PA-Store, Credentials, Signaturzustände, Manifest mit Hash je Eintrag, Eltern-Kette und lokal bekannte Hierarchie-Verwandte
GET /template/export/{did}	ZIP-Bundle einer Vorlage
GET /c2pa/manifest/{contract_did}	Öffentlicher C2PA-Manifest-Store; optional die Kettenaufzählung

pdf-core (intern, vom Backend angesprochen):

Endpunkt	Zweck
POST /render	JSON-LD zu PDF/A-3 kompilieren
POST /render/amendment	Bestehendes PDF inkrementell fortschreiben (neue Payload + Lifecycle-Credential)
POST /render/reanchor	Reines Provenance-Update-Manifest über die aktuellen (signierten) Bytes anhängen; Payload unverändert, Signatur unberührt (ADR-26)
POST /verify	Re-Render-Verifikation gegen die eingebettete Payload
POST /verify/content	Seiteninhalt gegen den Re-Render der eingebetteten Payload (Empfangsprüfung für Peer-PDFs)
POST /verify/content-match	Seiteninhalt zweier vorliegender PDFs vergleichen (Signaturannahme)
POST /claim	Externe Payload gegen ein vorgelegtes PDF binden
POST /payload/extract	Eingebettetes JSON-LD extrahieren
POST /manifest/extract , POST /manifest/chain	C2PA-Store bzw. Kettenaufzählung extrahieren
POST /c2pa/embed	Vom Backend erzeugte COSE-Signaturen in den Manifest-Store einsetzen
POST /evidence/embed , POST /evidence/extract	Signatur-Evidenz als Anhang ein-/auslesen
GET /version	Renderer-Version (Teil des persistierten PDF-Zustands)
GET /ontology/dcs-pdf-core , GET /ontology/dcs-pdf-core.shacl	Die vom Renderer verwendeten Schema-Artefakte

Die Verbindungsdaten von pdf-core (Adresse, Ontologie-Basis-IRI, Signatur-Callback, Zertifikatskette) sind Deployment-Konfiguration und im Deployment-Leitfaden beschrieben.

7.8 Betriebsverhalten

- **Export wartet statt zu lügen.** Läuft nach der letzten Änderung noch eine Regeneration, pollt der Export und bricht nach seinem Zeitfenster mit einem klaren Fehler ab („wird gerade regeneriert, gleich erneut versuchen“); die blockierende Bedingung steht im Log.
- **Verify setzt einen vorhandenen Bestand voraus.** Ohne mindestens einen vorherigen Export existiert kein gespeichertes PDF, und die Verifikation schlägt mit entsprechender Meldung fehl. Ist der Lifecycle-Zustand seit dem letzten Stand fortgeschritten, regeneriert die Verifikation transparent, ein Lese-Endpunkt mit Schreibpfad.
- **Verlorene Regenerationen holt ein Sweep nach.** Ein Lifecycle-Ereignis wird höchstens einmal zugestellt; scheitert die Regeneration, wäre die Entität sonst dauerhaft weder exportierbar noch verschiffbar. Ein periodischer Durchlauf sucht deshalb Entitäten, deren gespeichertes PDF nicht dem aktuellen Dokument entspricht, arbeitet sie in begrenzten Stapeln ab und wiederholt jede Entität nur begrenzt oft mit wachsendem Abstand. So blockiert ein dauerhaft unrenderbarer Vertrag nicht die heilbaren hinter ihm.

- **IPFS ist eventual consistent.** Frisch geschriebene CIDs sind nicht immer sofort lesbar; der Client wiederholt Lesezugriffe mit Backoff.
- **Bundle-Exporte verweigern statt unvollständig zu liefern.** Fehlt eine referenzierte Komponente, antwortet der Export mit einer maschinenlesbaren Befundliste statt eines lückenhaften Archivs.
- **Ein Amendment ohne inhaltliche Änderung** wird von pdf-core abgelehnt; identischer Inhalt erzeugt keine neue Revision. Die bewusste Ausnahme ist der Provenance-Re-Anchor nach der Signatur: Er trägt konstruktionsbedingt eine unveränderte Payload und läuft deshalb über einen eigenen Endpunkt.
- **Ein fehlerhaftes Peer-PDF ist ein Ablehnungsgrund, kein Absturz.** Die Verarbeitung eingehender Dokumente ist in Größe und Aufwand begrenzt; überschreitet ein Dokument diese Grenzen, wird es abgewiesen, bevor eine inhaltliche Prüfung stattfindet.

08 Föderation

Dieses Kapitel beschreibt, wie zwei unabhängige DCS-Instanzen, betrieben von verschiedenen Organisationen, Verträge austauschen und verhandeln, welches Trust-Modell jede Interaktion absichert und wie Vorlagen über den XFSC Federated Catalogue instanzübergreifend auffindbar werden. Das Grundprinzip: Über die Instanzgrenze wandern **Artefakte, kein Zustand**. Jede Instanz führt ihren eigenen Workflow, ihre eigene RBAC und ihre eigene Kopie des Vertrags (ADR-13).

8.1 Beteiligte Komponenten

Komponente	Rolle
DCS-to-DCS-Synchronizer	Ausgehende Seite: hört auf Lifecycle-Ereignisse und verschickt das Vertrags-PDF an die Gegenpartei; Fehlversuche landen in einer Retry-Tabelle
Peer-Endpunkte	Eingehende Seite: authentifizieren den Absender, prüfen das PDF (und bei signierten Sendungen JAdES und Vollmachtsnachweis) und übernehmen es als lokale Kopie; nehmen außerdem Löschanforderungen entgegen
Trust Gate	Vertrauensschicht vor jeder Interaktion, in beide Richtungen: Agreement Credential des Peers und lokaler Policy-Endpunkt (fail-closed)
Counterparty-PoA-Gate	Prüft die Handlungsvollmacht hinter jeder Signatur, die eine Gegenpartei mitschickt (ADR-31)
did:web-Identität + HSM	Instanzidentität mit Zertifikatskette; private Schlüssel im HSM (Kapitel 05)
pdf-core	Prüft beim Empfang, dass der Seiteninhalt des Peer-PDFs der Re-Render seiner eingebetteten Payload ist, und extrahiert das JSON-LD (Kapitel 07)
Audit-Trail	Jede Trust-Gate-Ablehnung wird als Incident erfasst (Kapitel 09)
Federated Catalogue	Instanzübergreifende Vorlagen-Discovery

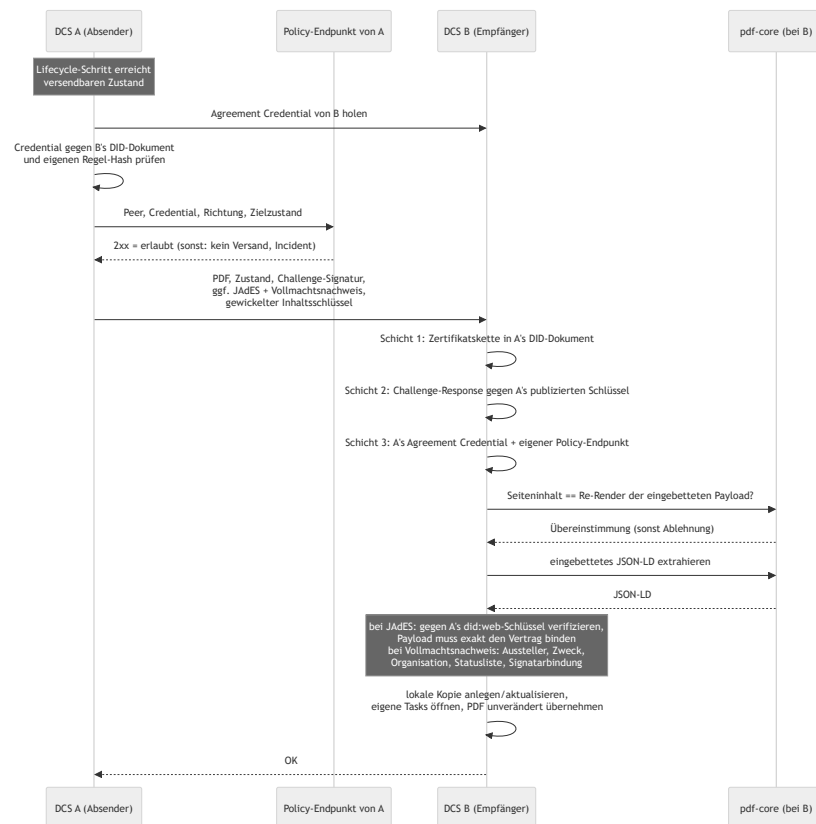
8.2 Instanz-zu-Instanz-Austausch: das PDF ist das Wire-Format

Zwei DCS-Instanzen tauschen im Vertragsablauf drei Dinge aus: **das Vertrags-PDF pro sichtbarem Lebenszyklusschritt, die JAdES-Signatur nach dem Signieren und den Nachweis der Handlungsvollmacht hinter dieser Signatur**. Das PDF ist selbsttragend, es enthält das maschinenlesbare JSON-LD, die C2PA-Provenance-Kette und etwaige Signaturen (Kapitel 07). Ein bloßes PDF ist ein Vorschlag (Angebot oder Verhandlungs-Gegenvorschlag); ein PDF mit JAdES ist die Signatur der Gegenseite. Zusätzlich reist der für den Peer gewickelte Inhaltsschlüssel mit, damit beide Seiten dieselben Artefakte lesen können (ADR-28, Kapitel 07).

Verschickt wird nur, was die Gegenseite sehen muss: die Zustände `OFFERED`, `NEGOTIATION`, `SIGNED` und `REVOKED`. Der Widerruf einer angebrachten Signatur muss die Gegenpartei unmittelbar erreichen, weil er die Vereinbarung entwertet. Interne Zustände (Entwurf, Review, Freigabe, Aktivierung, Terminierung) bleiben lokal; Review- und Freigabeprozesse der einen Organisation sind für die andere unsichtbar.

Die Verhandlung verläuft in drei getrennten Phasen: **verhandeln** (Gegenvorschläge als neue PDF-Versionen), **einigen** (beide Seiten bestätigen dieselbe Version; nachweisbar aus der Provenance des Artefakts) und erst danach **signieren**. Ein einseitig signierter, noch nicht beidseitig geeinigter Vertrag wäre ein disputables Artefakt, deshalb ist die Phasentrennung Teil des Protokolls.

Der Ablauf



Beim **ersten Empfang** entsteht die lokale Kopie im Zustand **OFFERED**, ein Angebot auf dem eigenen Tisch, mit dem Absender als Origin und den eigenen Nutzern in den lokalen Rollen; die eigenen Review-, Freigabe- und Verhandlungs-Tasks werden lokal geöffnet. Ein **erneuter Empfang** aktualisiert den Inhalt und erhöht die lokale Version, ohne den eigenen Workflow-Fortschritt zu überschreiben. Der vom Absender deklarierte Vertragszustand ist dabei rein informativ, mit genau einer Ausnahme: Meldet die authentifizierte Gegenpartei **REVOKED**, übernimmt der Empfänger diesen Zustand, denn der Widerruf der Signatur der Gegenseite entwertet die Vereinbarung. Kein anderer Peer-Zustand überschreibt je den lokalen Workflow.

Das empfangene PDF wird **exakt so, wie es kam**, als eigene Kopie gespeichert. Ein Neu-Rendern würde die C2PA-Provenance-Kette der Gegenseite zerstören; spätere eigene Änderungen amendieren diese Basis, sodass die Kette über beide Instanzen hinweg wächst.

Transport folgt der Identität, nicht der Netztopologie

Der Peer wird aus seinem **did:web**-Identifizier aufgelöst, einschließlich etwaiger Pfadsegmente, sodass mehrere Instanzen einen Host teilen können und jede unter ihrem eigenen Pfadpräfix angesprochen wird (Kapitel 05).

Die Auflösung ist HTTPS-only. Ein Rückfall auf Klartext ließe einen Angreifer auf dem Pfad sowohl das DID-Dokument mit dem Schlüssel des Peers als auch das dagegen geprüfte Agreement Credential ausliefern; das Föderations-Gate wäre wertlos. Zwei eng gefasste Ausnahmen gibt es: Loopback-Hosts, weil Entwicklungs- und CI-Stacks sich so gegenseitig auflösen, und Hosts, die ein Deployment ausdrücklich benennt, weil eine clusterinterne Identität hinter einem Service-Namen liegt, der weder Loopback ist noch TLS terminiert. Die Ausnahme ist damit explizit und pro Deployment sichtbar.

Ergänzend gilt für jeden ausgehenden Abruf: keine Weiterleitungen (eine Weiterleitung ließe die Gegenstelle das Ziel nach der Prüfung wechseln) und keine Verbindungen zu Adressen, an denen eine publizierte Identität nie liegt. Jeder Abruf ist mit einem Timeout begrenzt, damit „stumm“ nicht „hängend“ bedeutet.

Authentifiziert wird der Versand nicht auf Transportebene, sondern per Challenge-Response-Signatur im Request selbst.

8.3 Trust-Modell zwischen Instanzen

Zwischen Instanzen gibt es keine gemeinsame Nutzeridentität und keine Netzwerk-Vertrauensstellung; Vertrauen entsteht ausschließlich aus verifizierbaren Schichten:

Schicht	Frage	Mechanismus	Anker
1: Identität	Wer bist du?	<code>did:web</code> mit Zertifikatskette, optional gegen den EU-Trust-Pool geprüft	EU-Vertrauenslisten / QTSP
2: Besitznachweis	Sprichst gerade du?	Challenge-Response pro Request: Zufallswert, mit dem HSM-Schlüssel des Absenders signiert, verifiziert gegen dessen publizierten Schlüssel	Schicht 1
3a: Regelakzeptanz	Spielst du nach den Regeln?	Selbstsigniertes Agreement Credential unter <code>/.well-known/dcs-agreement-credential.json</code> , signiert mit dem dedizierten VC-Schlüssel aus dem eigenen DID-Dokument, dessen <code>termsOfUse</code> -Hash das Föderationsregelwerk benennt	In die Binärdatei einkompiliertes Regelwerk; der Hash muss dem eigenen entsprechen
3b: Policy	Darf diese Interaktion stattfinden?	Eigener lokaler Policy-Endpunkt (<code>DCS_TRUST_PDP_URL</code>): 2xx erlaubt, alles andere verweigert	Was auch immer der Endpunkt konsultiert (Allowlist, Trust-Listen, Clearing House, ...)
3c: Vollmacht	Durfte der, der unterschrieben hat, für diese Partei handeln?	Mitgeschickter Vollmachtsnachweis, geprüft gegen einen für <code>peer</code> zugelassenen Aussteller mit Organisationsberechtigung, inklusive Statusliste (ADR-31)	Ausstellervertrauen der empfangenden Instanz
4: Rechtswirkung	Bindet der Vertrag?	AES/QES auf dem Dokument (PADES/JAdES, Kapitel 06)	eIDAS

Die Schichten 3a/3b bilden zusammen das **Trust Gate** (ADR-19), das auf beiden Pfaden konsultiert wird, vor jedem Versand und bei jedem Empfang. Kernentscheidungen:

- **Fail-closed.** Ein nicht konfigurierter, nicht erreichbarer oder nicht antwortender Policy-Endpunkt verweigert genauso wie ein explizites Deny. Ohne konfigurierten Policy-Endpunkt ist Föderation wirksam abgeschaltet.
- **Regelakzeptanz ist an die Softwareversion gebunden.** Das Regelwerk ist einkompiliert und unter `/.well-known/dcs-federation-rules.md` abrufbar; sein Hash steht im Agreement Credential. Zwei Instanzen derselben Version stimmen automatisch überein, abweichende Regelwerke scheitern laut am Hash-Vergleich. Regeländerungen sind Software-Releases.
- **Das Credential muss vom Peer selbst stammen.** Aussteller-Hostname und Bezugsquelle müssen übereinstimmen, und der Proof muss den dedizierten VC-Eintrag des Peer-DID-Dokuments referenzieren, damit kein fremdes, zufällig verifizierbares Credential untergeschoben werden kann.
- **Kein „Phone home“.** Jede Instanz befragt ausschließlich ihren eigenen Policy-Endpunkt; die Gegenseite ruft ihn nie auf. Die Standard-Auslieferung enthält einen minimalen Flow, den Betreiber um dataspace-spezifische Prüfungen erweitern.
- **Jede Ablehnung ist auditierbar.** Trust-Gate-Denials landen als Incident im Audit-Trail. Auf dem Versandpfad wird eine Agreement-Credential-Ablehnung erneut versucht, eine Policy-Endpunkt-Ablehnung ist dagegen terminal: kein Retry, genau ein Incident pro Versuch, dedupliziert pro Vertrag, Peer und Richtung.

Provenance und Vollmacht des Gegenseiten-Artefakts

Ein signierter Versand trägt zusätzlich die JAdES-Signatur des Absenders über dem Vertragsinhalt. Der Empfänger nimmt sie nicht auf Zuruf entgegen, sondern verifiziert sie vollständig, bevor er die Sendung akzeptiert: Das kompakte JWS muss gegen seine eigene Zertifikatskette verifizieren, der Leaf-Schlüssel muss der publizierte `did:web`-Schlüssel des Absenders sein, und die signierte Payload muss exakt die Kanonisierung aus Vertrags-DID, Version und dem im verschickten PDF eingebetteten Vertragsdokument sein. Die Challenge-Response-Signatur authentifiziert nur die Sitzung, die JAdES bindet den Vertragsinhalt an den Schlüssel des Absenders. Das verifizierte Artefakt wird persistiert und bleibt als unabhängig nachprüfbarer Nachweis abrufbar.

Zusätzlich reist die Handlungsvollmacht mit (ADR-31). Die Signatur-Ceremony bewahrt das Vollmachts-Credential auf, das der Signatar vorgelegt hat, und der Synchronizer schickt es mit, sobald der Vertrag Signaturen trägt. Der Empfänger prüft, bevor er etwas von der Sendung persistiert:

- Der Aussteller ist für den Zweck `peer` zugelassen und berechtigt, diese Organisation zu attestieren.
- Credential-Typ, Signatur und Gültigkeitsfenster halten.
- Die Statusliste sagt, dass das Credential nicht widerrufen ist.
- Das Credential autorisiert genau die Partei, die signiert hat, und wird von genau dem Unterzeichner gehalten, den der Knoten dieser Partei ausweist.
- Ein Peer darf nur für **seine eigene** Partei Nachweise schicken.

Was diese Prüfung feststellt, ist präzise begrenzt: **eine Attestierung von Vollmacht.** Ein Aussteller, dem diese Instanz vertraut und der für diese Organisation sprechen darf, sagt, dass der Inhaber für sie handeln darf, und hat das nicht widerrufen. Sie stellt **nicht** fest, wer der Mensch ist; die Identitätsprüfung fand auf der Instanz statt, auf der die Ceremony lief, gegen deren Vertrauensanker. Auch Frische und Bindung an genau diesen Vertrag lassen sich nicht feststellen: Dieselbe Präsentation verifiziert für einen anderen Vertrag zwischen denselben Parteien identisch, bis sie abläuft oder widerrufen wird.

Das Verhalten bei Fehlern ist bewusst asymmetrisch. Ein Nachweis, der **vorhanden ist und nicht verifiziert**, verweigert die Sendung und erzeugt einen Trust-Gate-Incident. Ein Nachweis, der **fehlt**, tut das nicht, denn ein Peer, der keine Nachweise aufbewahrt, muss weiter fördern können, und eine Partei ohne hinterlegte Vollmacht ist etwas, das der Signature Compliance Viewer ohnehin ausweist. Das hebt den Boden an (wer Nachweise schickt, kann keine falschen schicken), erzwingt aber niemanden zum Nachweis.

Ein weiterer Nebeneffekt ist zu kennen: Läuft ein Vollmachts-Credential nach der Signatur ab oder wird es widerrufen, scheitern spätere Sendungen desselben Vertrags. Die Lebensdauer einer Vollmacht muss den Austausch überdauern.

8.4 Löschung über die Instanzgrenze

Ein Vertrag existiert auf zwei Instanzen, jede mit eigenen Artefakten. Eine Löschung, die nur auf der anfragenden Instanz wirkt, ist keine Löschung. Der Löschpfad ist deshalb ein eigener Peer-Aufruf, mit derselben did:web-Challenge-Response authentifiziert wie der PDF-Austausch:

1. Die anfragende Instanz vernichtet ihre eigenen gewickelten Inhaltsschlüssel des Vertrags und schreibt dazu ein Zerstörungseignis in den Audit-Trail. Dieser lokale Schritt ist ein harter Fehler, wenn er scheitert.
2. Sie fordert jede Gegenpartei-Instanz auf, dasselbe zu tun. Eine nicht erreichbare Gegenseite blockiert nicht: Die Anforderung bleibt als offener Eintrag stehen und wird periodisch erneut zugestellt.
3. Beide Seiten emittieren dasselbe Zerstörungseignis, mit Akteur, Vertrag, Bereich und Grund, nie mit Inhalt.

Ein einmal vernichteter Bereich ist endgültig: Weder ein lokaler Schreibvorgang noch ein vom Peer mitgeschickter Schlüssel erzeugt für ihn einen neuen. Der Fortschritt der Kette ist über eine eigene Statusabfrage sichtbar (Kapitel 09). Die Grenzen dieser Zusage beschreibt Kapitel 07.

8.5 Federated-Catalogue-Integration

Der Federated Catalogue dient der **Vorlagen-Discovery, nicht der Geschäftsdatenhaltung**: Er verifiziert und speichert Self-Descriptions (JSON-LD) und macht sie abfragbar; mehrere FC-Instanzen können sich untereinander synchronisieren, wodurch Vorlagen föderationsweit auffindbar werden.

Das Datenmodell verknüpft drei Self-Description-Typen: Die Vorlage (Resource) verweist auf den Participant, das ServiceOffering auf denselben Participant und trägt die Endpunkt-URL der betreibenden DCS-Instanz. So lässt sich von einer gefundenen Vorlage zum zuständigen DCS-Endpunkt navigieren, an dem eine Verhandlung beginnen kann.

Aus DCS-Sicht:

Schnittstelle	Zweck
Publish einer Vorlage (Kapitel 03)	Eine registrierte Vorlage wird als Self-Description an den Katalog übermittelt. Aussteller ist die Instanz-DID , nicht der einzelne Mitarbeiter; der Participant ist eine Organisation, kein Endnutzer (ADR-18)
Katalog-Lesezugriff	Vorlagenliste abrufen, einzelne Vorlage per DID/Version auflösen, Metadaten durchsuchen

Zur Authentifizierung gegenüber dem Katalog verwendet die Instanz einen Keycloak-Service-Account. Dieser hält sämtliche funktionalen Katalogrollen einschließlich der administrativen (ADR-18). Daraus folgt eine harte Betriebsgrenze: Ein Katalog darf **nicht** von mehreren DCS-Deployments geteilt werden, die sich nicht vollständig vertrauen, da jede Instanz sonst Katalogeinträge anderer Mandanten ändern könnte. Das Deployment erzwingt diese Grenze: Wird die Katalog-Integration gegen einen entfernten, nicht mitdeployten Katalog konfiguriert, schlägt das Rendering fehl, solange der Betreiber die gemeinsame administrative Vertrauensgrenze nicht ausdrücklich bestätigt.

Betriebsdetaill der Katalog-Kopplung (Realm-Provisionierung, bekannte Upstream-Eigenheiten, Beispiel-Self-Descriptions) sammelt der FC-Integrationsleitfaden; das Deployment beschreibt der Deployment-Leitfaden.

8.6 Betriebsverhalten

- **Versand ist ereignisgetrieben und idempotent nachholbar.** Ein fehlgeschlagener Versand (Peer nicht erreichbar, PDF noch nicht regeneriert, Agreement-Credential-Ablehnung) hinterlässt einen Retry-Eintrag; ein Scheduler versucht ihn in konfigurierbarem Intervall erneut. Ein erfolgreicher Versand räumt den Eintrag auf. Diese Zustellung ist datenbankgestützt und deshalb unabhängig davon, ob eine Event-Bus-Nachricht verloren geht.
- **Ein Angebot vor fertigem PDF wird nie stillschweigend verworfen:** Ist der Vertrag versendbar, aber das PDF noch nicht verfügbar, wird der Versand explizit in die Retry-Queue gestellt.
- **Terminale Policy-Denials verlassen die Retry-Queue.** Eine Ablehnung durch den eigenen Policy-Endpunkt erzeugt einen Incident und entfernt einen etwaigen Retry-Eintrag atomar. Sie würde sonst endlos erneut anlaufen, obwohl die Entscheidung terminal ist.
- **Was der Betreiber sieht:** Trust-Gate-Incidents im Audit-Trail mit Peer-DID, Richtung und Begründung; Versand- und Empfangsfehler in den Logs; bei einem Inhalts-Mismatch eines empfangenen PDFs eine Diagnose mit der divergierenden Stelle und dem Hash der eingebetteten Payload; offene Löschanforderungen als Einträge der Erasure-Statusabfrage.

09 Audit und Compliance

Das DCS behandelt Nachvollziehbarkeit als kryptographische Eigenschaft, nicht als Log-Konvention: Jedes fachlich relevante Ereignis wird in einem manipulationsnachweisbaren Audit-Trail verankert, dessen Integrität über Hash-Verkettung, Merkle-Checkpoints und vertrauenswürdige Zeitstempel unabhängig prüfbar ist. Dieses Kapitel beschreibt den Audit-Trail, die Auswertungs- und Report-Funktionen, die Workflow-Gate-Evidenz, das kontinuierliche Compliance-Monitoring mit Incident-Erfassung sowie das Policy-Enforcement über ODRL und OPA.

9.1 Beteiligte Komponenten

Komponente	Rolle
Fachkomponenten	Schreiben ihre Ereignisse transaktional in eine Outbox-Tabelle, im selben Datenbank-Commit wie die fachliche Zustandsänderung
Outbox-Prozessor	Hintergrundprozess im Backend; publiziert Ereignisse auf dem Event-Bus und verankert audit-sichtbare Ereignisse als Audit-Einträge
Artefaktspeicher über IPFS	Hält die Audit-Einträge und Checkpoints content-adressiert; die CIDs sind die eigentliche Autorität des Trails
PostgreSQL	Index über den Trail: Kettenköpfe pro Ressource, Checkpoint-Tabellen, ausstehende Zeitstempel, Dead-Letter-Status, Risiko-Register, Workflow-Gate-Läufe
TSA (RFC 3161, über einen ORCE-Flow)	Stellt den vertrauenswürdigen Zeitstempel über jede Checkpoint-Root aus
ORCE, externer Notar	Holt periodisch den Checkpoint-Head ab und trägt ihn aus der Reichweite des Betreibers hinaus
ORCE, Audit-Executor	Bewertet einen Audit-Abruf: Das DCS sammelt die Evidenz, der Executor bildet die Befunde
ORCE, Workflow-Gate-Executor	Entscheidet vor jedem gate-pflichtigen Lebenszyklusübergang über den Vertragsschnappschuss (Kapitel 04)
OPA (eingebettet)	Wertet ODRL-Constraints als Rego-Policies aus; die Ergebnisse fließen als Policy-Findings in den Audit-Trail
<code>pacmonitor</code> (CronJob)	Führt den Compliance-Sweep planmäßig aus, ohne dass jemand fragt (ADR-32)
Process Audit & Compliance (API)	Audits, Reports, Monitoring, Incident-Reports, Checkpoint-Head und Inklusionsbeweise, Workflow-Gate-Läufe

9.2 Der Audit-Trail

Vom Ereignis zum verankerten Eintrag

Jede fachliche Operation schreibt ihr Ereignis in dieselbe Datenbank-Transaktion wie die Zustandsänderung (Outbox-Muster). Ein Ereignis kann daher nie verloren gehen, ohne dass auch die Zustandsänderung fehlt. Der Outbox-Prozessor arbeitet die Einträge anschließend in drei entkoppelten Schleifen ab:

1. **Publizieren.** Ereignisse werden auf dem Event-Bus republiziert, damit Abonnenten (Webhooks, PDF-Generierung, Auto-Deployment) reagieren können. Diese Schleife ist bewusst vom Verankern getrennt und läuft deutlich häufiger: Konsumenten brauchen nur die Ereignis-Payload, nie einen Anker-Wert, und sollen nicht hinter TSA- und Speicher-Roundtrips warten.
2. **Verankern.** Audit-sichtbare Ereignisse werden stapelweise als Audit-Einträge geschrieben und durch einen Merkle-Checkpoint committet. Reine Lookup-Ereignisse (Abruf- und Suchoperationen) werden als verarbeitet markiert, ohne verankert zu werden; sie würden sonst die Stapel dominieren und echte Audit-Ereignisse verdrängen.
3. **Nach-Zeitstempeln.** Checkpoints, die während einer TSA-Störung ohne Zeitstempel verankert wurden, erhalten ihn in einem Wiederholungslauf.

Hash-Verkettung pro Ressource

Jeder Audit-Eintrag verweist per CID auf seinen Vorgänger derselben Ressource. Die Historie eines Vertrags oder einer Vorlage ist damit eine strikte Hash-Kette: Ein nachträglich veränderter Eintrag hätte eine andere CID, und alle Nachfolger würden ins Leere zeigen. Ketten verschiedener Ressourcen sind voneinander unabhängig und werden nebenläufig geschrieben, so dass eine gestörte Ressource die übrigen nicht blockiert. Ein Audit-Lesevorgang läuft die Kette von ihrem Kopf rückwärts durch und rekonstruiert so die vollständige Historie.

Öffentlicher Kopf, privater Rumpf

Ein Audit-Eintrag zerfällt in zwei Teile (ADR-28):

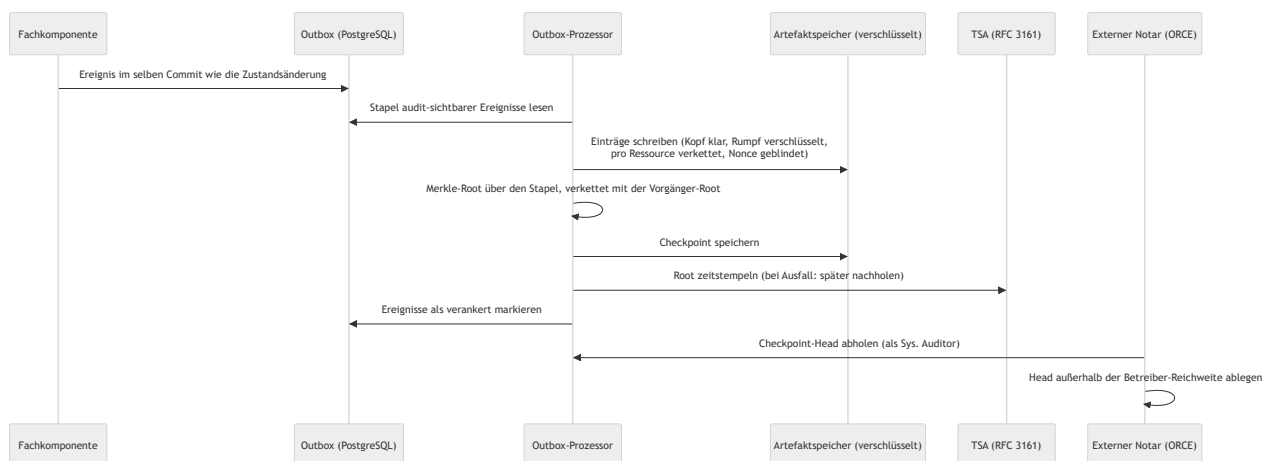
- **Kopf im Klartext:** Komponente, Ereignistyp, DID, Zeitstempel, Vorgänger-CID und die Blinding-Nonce, also alles, was die Kette und die Beweise tragen.
- **Rumpf verschlüsselt:** Die Ereignis-Payload liegt unter dem Inhaltsschlüssel des jeweiligen Vertrags oder der Vorlage. Ereignisse ohne Ressourcenbezug (Checkpoints, instanzweite Reports und die Löschnachweise selbst) liegen unter dem instanzweiten Schlüssel, den keine Löschanforderung zerstört. Ein Löschnachweis, der sich selbst löschen könnte, würde nichts beweisen.

Leaf-Hashes und Checkpoints werden über das gespeicherte Objekt gebildet, wie es ist. Deshalb macht eine Schlüsselvernichtung die Rümpfe dauerhaft unlesbar, während **jeder Inklusionsbeweis weiter verifiziert**: Manipulationsnachweis und Löscharbeit koexistieren, weil der Beweis nie davon abhing, den Klartext zu lesen.

Merkle-Checkpoints und Zeitstempel

Übergreifende Manipulationssicherheit stellt nicht ein globaler Link pro Ereignis her, sondern ein **Merkle-Checkpoint pro Verankerungs-Stapel** (ADR-16):

- Über alle Einträge des Stapels, in Outbox-Reihenfolge, wird eine Merkle-Root berechnet, mit Domänentrennung zwischen Blatt- und Knoten-Hashing.
- Jede Root verkettet sich mit der Root des vorherigen Checkpoints. Eine einzige veröffentlichte Root committet dadurch transitiv den gesamten Log davor und beweist zusätzlich dessen Append-only-Eigenschaft.
- Die Root wird **einmal pro Stapel** von der TSA nach RFC 3161 zeitgestempelt, statt einmal pro Ereignis. Ist die TSA nicht erreichbar, wird der Checkpoint trotzdem verankert und der Zeitstempel später nachgereicht: Ein TSA-Ausfall verzögert die Evidenz, blockiert aber nie den Trail.
- Jeder Eintrag trägt eine zufällige **Blinding-Nonce**, die in seinen Blatt-Hash eingeht. Audit-Einträge sind hochgradig erratbar (Komponente, Ereignistyp, DID, Zeitstempel); ohne Nonce ließe sich ein Blatt-Hash per Brute-Force bestätigen. Mit Nonce sind Inklusionsbeweise gefahrlos publizierbar. Den Inhalt rekonstruieren kann nur, wer den Eintrag samt Nonce bereits besitzt.



Was publizierbar ist

Publizierbar	Nie publiziert
Sequenznummer, Root, Vorgänger-Root, Blattanzahl, Erstelzeit, RFC-3161-Token	Blatt-CIDs (Abruffähigkeiten in den eigenen Speicher)
Inklusionsbeweise (ausschließlich Hashes)	Die Einträge selbst (enthalten DIDs, Identitäten, Workflow-Payloads)

Der Head-Abruf liefert nur den Kopf; der Beweis-Abruf liefert Blatt-Hash, Blatt-Index, Geschwister-Hashes und den zugehörigen Head, nie den Eintrag. Der Beweis wird vor der Herausgabe intern gegen die gespeicherte Root verifiziert, sodass ein korrupter Index im System auffällt und nicht erst beim Auditor. Ein einzelner Checkpoint ist zusätzlich über seine Sequenznummer abrufbar.

Externe Verankerung und unabhängige Verifikation

Ein Trail, den nur der Betreiber hält, beweist nichts **gegen** den Betreiber. Deshalb holt ein ORCE-Flow den Checkpoint-Head periodisch ab und reicht ihn an eine externe Senke weiter. Der Flow authentifiziert sich als Maschinen-Identität mit der Rolle **Sys. Auditor**; Rollen und Zurechnung liest das Backend anhand der Client-ID aus der Registry, nie aus Token-Claims (Kapitel 05). Diese Rolle darf ausschließlich den Checkpoint-Head lesen, keinen Vertragsinhalt und keinen Schreibpfad. Der Inklusionsbeweis-Endpunkt bleibt der menschlichen Auditor-Rolle vorbehalten.

Ein Dritter verifiziert einen Eintrag mit genau drei Zutaten: den Eintrags-Bytes samt Nonce (die er selbst erhalten hat), dem Inklusionsbeweis und einem Head, den er **vom externen Anker bezieht, nicht vom Betreiber**. Welche Senke der Flow beliefert (Notariat, Chain, Gegenpartei), bestimmt der Betreiber. Ohne konfigurierte Senke findet keine externe Verankerung statt, und der Trail bleibt vollständig in der Reichweite des Betreibers.

9.3 Audits und Reports

Audit-Abruf

Auditoren (Rollen `Auditor`, `Archive Manager`) fordern die Audit-Trails eines **Scopes** an: Templates, Verträge, Archiv oder Signaturen, optional gefiltert auf eine einzelne Ressourcen-DID. Jede Anfrage erfordert eine **Begründung**; der Abruf selbst wird als Audit-Ereignis in den Trail geschrieben, auch das Auditieren ist auditierbar. Ein Archive Manager ohne Auditor-Rolle darf ausschließlich den Archiv-Scope einsehen.

Der Abruf ist zweistufig: Das DCS **sammelt die Evidenz**, also die Hash-Ketten der Ressourcen des Scopes, ergänzt um die frisch berechneten Inhalts- und Policy-Prüfungen, und übergibt sie an den externen **Audit-Executor**. Der bildet daraus die Befunde und liefert sie mit Executor-Identität, -Version und Ausführungszeitpunkt zurück; Anfrage, Antwort und Rohantwort werden persistiert. Damit ist die Auditbewertung austauschbar, ohne den Evidenzpfad zu berühren, und ihr Ergebnis nachträglich zuordenbar. Ist kein Executor konfiguriert oder erreichbar, antwortet der Abruf mit einem eigenen Fehler; ein stilles Ausweichen auf eine interne Bewertung gibt es nicht.

Bei einem vertragsbezogenen Abruf wird die Kette der Audit- und Compliance-Komponente mitgelesen, damit ein Befund, der am Vertrag hängt, aber der Compliance-Domäne gehört (etwa eine abgelehnte Föderationssignatur), im Audit auftaucht statt nur die Workflow-Ereignisse.

Reports

Aus demselben Datenbestand erzeugt der Report-Pfad Reports in JSON, CSV oder PDF. Ein Report gliedert sich in Zusammenfassung, Ressourcenliste, Lebenszyklus-Ereignisse und Compliance-Findings. Jeder Report erhält eine deterministische Report-ID und einen Content-Hash; die Report-Bytes werden zusätzlich abgelegt, und die Erzeugung selbst wird mit Hash, CID und Begründung als Ereignis im Audit-Trail festgehalten. Ein einmal ausgehändigter Report ist damit später gegen den Trail verifizierbar.

Workflow-Gate-Läufe als Evidenz

Jeder Lauf eines Workflow-Gates (Kapitel 04) ist selbst ein Compliance-Artefakt: Er hält Korrelations-ID, Schnappschuss-Identität, Vertrag und Version, Zustand, Inhalts-Hash, die wirksamen Shapes, die Profilverversion, die lokale Bewertung und die Executor-Antwort fest. Ein Compliance Officer kann einen Lauf einsehen und einen Lauf im Zustand `REVIEW` mit Begründung entscheiden; die Entscheidung wird mit Entscheider und Zeitpunkt persistiert und setzt genau den angehaltenen Übergang fort.

9.4 Kontinuierliches Monitoring und Incidents

Die vier Risikoklassen

Der Compliance-Sweep meldet vier Klassen:

- **MISSING_APPROVAL** : Verträge, die in einem freigabepflichtigen Zustand auf eine noch ausstehende, erforderliche Freigabe warten (ein Risiko je Vertrag-Freigeber-Paar).
- **CONTRACT_UNDERPERFORMANCE** : laufende Verträge, deren vom Zielsystem zurückgemeldete KPI-Werte die eigenen ODRL-Verpflichtungen verletzen. Die Bewertung passiert bereits beim Eintreffen der Rückmeldung: Der Deployment-Callback prüft jeden gemeldeten Wert gegen die ODRL-Policies des Vertrags und persistiert das Urteil mit dem Wert. Die Verletzung wird als Alert gemeldet und nicht als Request-Fehler abgewiesen, denn der Vertrag ist in Kraft und der Verstoß ist ein zu dokumentierender Fakt. Der Sweep liest dieses Urteil zurück und benennt Metrik, beobachteten Wert und Beobachtungszeitpunkt, statt eine zweite, möglicherweise abweichende Bewertung vorzunehmen.
- **CONTRACT_DEPLOYMENT_FAILED** : Deployments, deren Versand an das designierte Zielsystem gescheitert ist. Der Vertrag ist auf dieser Seite in Kraft, beim Zielsystem aber nie angekommen; genau das ist der zu meldende Zustand.
- **UNAUTHORIZED_ACCESS** : Verträge mit persistierten Zugriffsverweigerungen: Weist die Parteiprüfung einen Abruf ab, wird die Verweigerung als Audit-Artefakt festgehalten, bevor die Ablehnung zurückgeht; der Sweep flaggt daraus ein Risiko je Vertrag-Akteur-Paar.

Geplanter Sweep statt Nachfrage (ADR-32)

Ein Abruf auf Anfrage erfüllt „auf Nachfrage überwachen“, nicht „kontinuierlich überwachen“: Eine Verletzung existierte erst ab dem Moment, in dem jemand zufällig fragt. Deshalb läuft dieselbe Erkennungslogik zusätzlich als **planmäßiger Sweep**:

- Ein eigenes Kommando (`pacmonitor`) führt genau einen Sweep aus und endet. Es öffnet nur die Datenbank; insbesondere durchläuft es nicht die Startprüfungen des Servers, damit das Compliance-Monitoring genau dann weiterarbeitet, wenn eine Nachbarkomponente gestört ist. Es tritt auch dem Event-Bus nicht bei: Erkennungen gehen in die transaktionale Outbox, und der laufende Server verankert sie und verteilt sie an die Webhook-Abonnenten.
- Ein **CronJob** ruft es in festem Takt auf, mit unterbundener Nebenläufigkeit. Er ist im Auslieferungszustand aktiv; ihn abzuschalten bedeutet, dass Risiken nur noch gefunden werden, wenn jemand fragt.

- Ein **Risiko-Register** (`compliance_risk_findings`) hält eine Zeile je offener Verletzung, geschlüsselt über Vertrag, Risikoklasse und einen Hash des Detailtexts. Ein Risiko alarmiert, wenn es zuerst erkannt wird, und erneut, wenn es nach einer Behebung wiederkehrt, nie auf den Sweeps dazwischen. Ohne dieses Register würde derselbe Verstoß bei jedem Lauf neu gemeldet: ein Alarm-Sturm und ein Audit-Trail, in dem eine Verletzung hundertfach auftaucht und der Moment ihrer Erkennung darin untergeht.
- Der Detailschlüssel ist Teil des Schlüssels, weil ein Vertrag mehrere Risiken derselben Klasse gleichzeitig tragen kann: eine je ausstehendem Freigeber, eine je abgewiesenem Akteur, eine je verletzter Metrik. Nur über Vertrag und Klasse geschlüsselt würde ein zweiter abgewiesener Akteur stillschweigend als „schon gemeldet“ verschluckt.
- Der Zeitpunkt der Ersterkennung wird bei einem Wiederauftreten zurückgesetzt: Eine wiederkehrende Verletzung ist ein neuer Vorfall, und die daran gemessene Erkennungszeit muss diesen Vorfall beschreiben. Die abgelösten Vorfälle sind nicht verloren, denn jede Erkennung ist im Audit-Trail verankert.
- Ein erkanntes Risiko ist **kein Job-Fehlschlag**: Der Sweep meldet Risiken auf der Standardausgabe und endet erfolgreich. Ein Fehlschlag bedeutet, dass der Sweep nicht laufen konnte, und das ist der einzige Zustand, der einen Betreiber wecken sollte.

Die Antwort des Abrufs auf Anfrage bleibt unverändert vollständig: Sie listet jedes aktuell zutreffende Risiko, weil sie die Frage „was ist gerade falsch?“ beantwortet. Die Deduplikation steuert nur das Alarmieren.

Jeder Sweep wird selbst als Ereignis verankert, inklusive der gefundenen Risiken, und jedes Risiko zusätzlich in der Audit-Kette des betroffenen Vertrags.

Latenz. Die zeitbasierten Regeln (fehlende Freigaben, Abläufe) sind mit dem Standardtakt gut bedient; die ereignisgetriebenen (unautorisierter Zugriff, gescheitertes Deployment) werden entsprechend verzögert gemeldet. Wo Sekundenlatenz gefordert ist, ist der Weg ein Event-Bus-Abonnent **neben** dem Sweep und kein engerer Takt: Die zugrundeliegenden Ereignisse fließen bereits über den Bus, und das Risiko-Register macht beide Quellen kombinierbar, weil alarmiert, wer zuerst beobachtet.

Alerts an externe Subscriber

Die Webhook-Plattform liefert benannte Ereignisse aktiv aus. Neben den Lebenszyklus-Ereignissen gehört dazu `compliance.risk_detected`, das genau bei der Ersterkennung eines Risikos ausgelöst wird. Jede Zustellung wird mit Ergebnis protokolliert und ist per Callback quittierbar (Anhang A).

Incident-Reports

Non-Compliance-Befunde, die außerhalb des automatischen Monitorings entstehen (etwa aus einer manuellen Untersuchung), reicht der Compliance Officer als Incident-Report ein: eine Liste von Findings aus Risikoklasse und Beschreibung, verpflichtend verknüpft mit der DID des betroffenen Vertrags oder Templates. Jedes Finding wird als eigenes Ereignis in der Audit-Kette dieser Ressource verankert; ein späterer Audit-Abruf beweist, dass und wann der Befund erfasst wurde.

Startup-Attestierung der Konfiguration

Beim Start hasht das Backend die sicherheitskritischen, als Datei gemounteten Konfigurationsartefakte (das DID-Dokument, das Issuer-Trust-Dokument, die Zertifikats-Vertrauensanker und die Autorisierungs-Policy des Issuer-Vertrauens) und verankert die Hashes als Attestierungsereignis im Audit-Trail: das Konfigurations-Integritätsprotokoll der Instanz. Pinnt der Betreiber erwartete Hashes, wird aus der Attestierung eine erzwungene Authentisierung der Konfiguration: Eine gepinnte Datei, die fehlt oder vom Pin abweicht, bricht den Start ab, ebenso eine syntaktisch fehlerhafte Pin-Liste, damit ein Deployment nie mit stillschweigend unwirksamen Pins läuft. Dass die Policy mit attestiert wird, ist kein Detail: Sie rangiert über dem Trust-Dokument, und eine gepinnte Datei unter einer ungepinnten Regel wäre wirkungslos (Kapitel 05).

Löschnachweise

Die Vernichtung der Inhaltsschlüssel eines Vertrags (Kapitel 04 und 07) erzeugt auf jeder beteiligten Instanz ein eigenes Ereignis mit Akteur, Vertrag, Bereich und Grund, nie mit Inhalt. Diese Ereignisse liegen unter dem instanzweiten Schlüssel und überleben damit jede Löschung, die sie dokumentieren. Der Fortschritt der Löschkette über die Instanzgrenze ist über eine eigene Statusabfrage sichtbar: welche gewickelten Schlüsselzeilen zerstört sind, wann und durch wen, und welche Peer-Anforderungen noch offen sind.

9.5 Policy-Enforcement: ODRL auf OPA

Vertragsbedingungen sind als ODRL 2.0 unter einem DCS-Profil formalisiert (ADR-6). Ausgewertet werden sie durch **Open Policy Agent**, eingebettet als Bibliothek (ADR-11). Die Operator-Semantik ist als Rego-Modul formuliert (Vergleichsoperatoren mit case-insensitivem String-Vergleich, numerischer Koerzierung mit Toleranz und RFC-3339-Zeitvergleich), und jede Auswertung ist eine In-Process-Abfrage: kein Sidecar, keine zusätzliche Latenz auf dem Freigabe- und Signaturpfad. Die Übereinstimmung mit der Referenzsemantik ist durch ein Paritäts-Testorakel abgesichert.

Was das Rego-Modul beantwortet, ist bewusst klein: die Wahrheit **einer** Bedingung. Die deontische Logik darüber (Verbot verletzt, wenn erfüllt; `and` / `or` / `xone`; Pflichten und ihre Konsequenzen; die Behandlung von Operanden, die erst zur Nutzungszeit bekannt sind) liegt im auswertenden Code, nicht in Rego.

Zwei Enforcement-Momente teilen sich denselben Evaluator und dieselben Policies; nur die Quelle des Weltzustands unterscheidet sich:

Moment	Weltzustand	Wirkung
Vor der Signatur (Freigabe, Signaturvorbereitung)	Die im Vertrag inline getragenen Feldwerte	Serverseitige Ablehnung constraint-verletzender Werte; Findings im Trail
Ausführung / KPI-Monitoring	Extern zurückgemeldete Laufzeit-KPI-Werte	Violation-Erkennung gegen dieselben Policies beim Eintreffen jeder Rückmeldung; das persistierte Urteil speist die Underperformance-Risiken

Grenzen, die man kennen muss. Bedingungen, deren linker Operand ein Nutzungskontext ist (Zeitpunkt, Ort, Menge, Empfänger ...), können vor der Signatur nicht entschieden werden; sie erscheinen als Hinweisbefund „wird zur Nutzungszeit durchgesetzt“. Der einzige Kanal, der diese Nutzungszeit tatsächlich beobachtet, ist die KPI-Rückmeldung des Zielsystems. Ein Vertrag, dessen Bedingungen überwiegend Kontextoperanden tragen, ist damit vor der Signatur formal geprüft, aber inhaltlich erst im Betrieb überwachbar, und nur soweit ein Zielsystem die passenden Werte meldet.

Ergänzend zu den vertragsindividuellen ODRL-Policies definieren zwei versionierte **Policy-Sets** unter `docs/policies/` die instanzweiten Prüfreden: ein Template-Policy-Set (kanonische JSON-LD-Struktur, deklarierte Vertragsfelder, Policy-Operanden referenzieren deklarierte Felder, Lebenszyklus- und Domänenfeld-Regeln) und ein Contract-Content-Policy-Set (kanonische Shapes und Validierungsprofil). Jede Regel trägt eine stabile Rule-ID, einen Schweregrad und den betroffenen Ontologie-Begriff; jedes Finding (Regel, Operator, Erwartungs- und Ist-Wert, Feld-IRI) wird als Audit-Ereignis festgehalten und erscheint in Audits und Reports. Ein Befund der Schwere „error“ aus diesem Satz blockiert zusätzlich den zugehörigen Workflow-Gate-Übergang (Kapitel 04).

Eine Präzisierung, die häufig missverstanden wird: Diese Regeln prüfen **Struktur und Vorhandensein** deklarerter Felder, nicht deren Werte. Wertgrenzen werden dort durchgesetzt, wo sie als SHACL formuliert sind, im Klausel-Katalog und in registrierten Shape-Bibliotheken (Kapitel 03).

9.6 Schnittstellen

Endpunkt	Zweck	Rollen
POST /pac/audit	Audit-Trail eines Scopes abrufen und extern bewerten lassen (Begründung Pflicht, Abruf wird auditert)	Auditor, Archive Manager
GET /pac/report	Audit-Report erzeugen (JSON/CSV/PDF; Hash und CID im Trail)	Auditor, Archive Manager
POST /pac/report	Incident-Report: Non-Compliance-Findings zu einer Ressource erfassen	Compliance Officer
GET /pac/monitor	Compliance-Sweep auf Anfrage; auch ein Lauf ohne Befund wird auditert	Compliance Officer
GET /pac/audit/checkpoint/head	Neuester Checkpoint-Head (nur Hashes, Zählwerte, Zeitstempel, publizierbar)	Auditor, Archive Manager, Sys. Auditor
GET /pac/audit/checkpoint/{seq}	Ein bestimmter Checkpoint	Auditor, Archive Manager, Sys. Auditor
GET /pac/audit/checkpoint/proof/{entry_cid}	Merkle-Inklusionsbeweis für einen verankerten Eintrag	Auditor
GET /pac/workflow-gates/{run_id}	Persistierten Workflow-Gate-Lauf und seinen Prüfzustand lesen	Compliance Officer
POST /pac/workflow-gates/review	Entscheidung zu einem Lauf im Zustand REVIEW festhalten	Compliance Officer
GET /archive/erasure-status	Stand der Schlüsselvernichtung eines Vertrags, lokal und je Peer	Archive Manager, Auditor

9.7 Betriebsverhalten

- **Transiente Anker-Fehler** (Speicher oder TSA kurz nicht erreichbar) werden pro Ereignis gezählt und im nächsten Tick erneut versucht; der betroffene Eintrag fällt lediglich aus dem aktuellen Checkpoint und wandert in den nächsten. Sein fachlicher Zeitstempel bleibt unverändert, so dass der Trail zwischen „wann geschah es“ und „bis wann war seine Existenz bewiesen“ unterscheidet.
- **Dead-Lettering:** Nach 50 fehlgeschlagenen Verankerungsversuchen wird ein Ereignis totgelegt, deutlich geloggt und nicht mehr angefasst. Dead-Letter-Ereignisse sind Lücken im Trail und brauchen einen Operator; sie sind in der Outbox-Tabelle an ihrer Dead-Letter-Markierung samt letztem Fehler auffindbar.
- **TSA-Ausfall:** Checkpoints werden ohne Zeitstempel verankert und regelmäßig nachgestempelt; im Head ist der Zustand am fehlenden Zeitstempel sichtbar.
- **Volumen-Leck:** Publierte Heads verraten Stapelgröße und Takt, also Aktivitätsvolumen. Das ist eine bewusste, akzeptierte Abwägung (ADR-16).

- **Lese-Grenzen:** Ein Voll-Trail-Read läuft maximal 500 Checkpoints rückwärts; komponentenweite Audits lesen die unabhängigen Ressourcen-Ketten parallel, damit der Abruf innerhalb der Request-Deadline bleibt.
- **Gelöschte Verträge im Audit:** Die Kette eines Vertrags, dessen Schlüssel vernichtet wurden, bleibt vorhanden und beweisbar, ihre Rümpfe sind unlesbar. Integritätsprüfungen des Archivs schließen Einträge aus, die diese Instanz gelöscht hat; deren Audit-Kette dokumentiert die Löschung weiterhin.

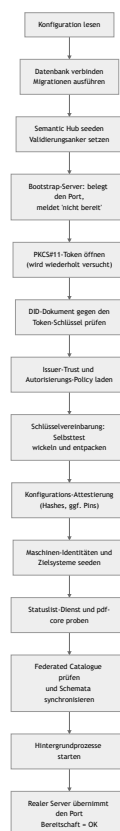
10 Betrieb und Monitoring

Dieses Kapitel beschreibt, wie sich eine DCS-Instanz im Betrieb verhält: was beim Start passiert, welche Hintergrundprozesse dauerhaft laufen, welche Signale es gibt, wie ein Fehlerfall sichtbar wird und woran der Betreiber ihn erkennt.

Es enthält bewusst keine Befehle, Werte oder Prozeduren. Installation, Konfigurationsreferenz, Umgebungsmatrix, Upgrade und Rollback stehen im Deployment-Leitfaden; Sicherung und Wiederherstellung im Backup-Leitfaden.

10.1 Startverhalten: fail fast, aber geduldig an einer Stelle

Das Backend prüft seine harten Abhängigkeiten selbst und läuft nie degradiert weiter. Der Start verläuft in einer festen Reihenfolge, und an jeder Stufe entscheidet sich, ob der Prozess gesund wird:



Wesentliche Eigenschaften:

- **Der Port ist von Anfang an belegt.** Ein Bootstrap-Server beantwortet den Bereitschaftsendpunkt mit „nicht bereit“, solange die Initialisierung läuft. Dadurch unterscheidet Kubernetes einen startenden Prozess von einem betriebsbereiten DCS und sieht keinen Verbindungsabbruch.
- **Genau eine Stufe wartet statt abubrechen:** das Öffnen des PKCS#11-Tokens. Bei einer frischen Installation kann die Token-Provisionierung noch laufen; deshalb wird das Öffnen wiederholt versucht und der Wartezustand geloggt. Alles danach, insbesondere das Laden einzelner Schlüssel, ist hart, weil ein vorhandenes Token alle Schlüssel in einem Zug bekommt.

- **Passt das DID-Dokument nicht zum Token-Schlüssel, startet die Instanz nicht.** Das ist die häufigste Folge einer Neu-Provisionierung des Tokens ohne Neuerzeugung des Dokuments.
- **Der Selbsttest der Schlüsselvereinbarung ist entscheidungsrelevant:** Er beweist, dass der im DID-Dokument publizierte Schlüssel und der Token-Schlüssel zusammenpassen und dass das Token die nötige Ableitung beherrscht. Schlägt er fehl, wäre kein gespeichertes Artefakt lesbar, und die Instanz startet nicht (Kapitel 07).
- **Statuslist-Dienst und pdf-core werden per HTTP geprobt,** mit Wiederholungen über ein begrenztes Fenster, damit eine lediglich langsam startende Abhängigkeit keinen Crash-Loop auslöst; bleibt sie unerreichbar, beendet sich der Prozess mit klarer Meldung.
- **Ein konfigurierter Federated Catalogue ist eine harte Abhängigkeit.** Ist er unvollständig konfiguriert oder nicht funktional erreichbar, startet die Instanz nicht; ist er gar nicht konfiguriert, läuft die Instanz ohne Katalogfunktionen.
- **Der Audit- und der Workflow-Gate-Executor sind Pflicht.** Fehlt ihre Adresse, startet die Instanz nicht. Sie säßen sonst als fail-closed-Gates in jedem Lebenszyklusübergang und würden jeden blockieren (Kapitel 04 und 09).
- **Ungültige Deklarationen brechen den Start ab,** statt still zu wirken: eine unbekannte Rolle im Seed der Maschinen-Identitäten, eine fehlerhafte Zielsystem-Deklaration, eine nicht übersetzbare Autorisierungs-Policy, eine syntaktisch falsche Pin-Liste, ein Trust-Eintrag mit einem Mechanismus, den dieser Stand nicht auflösen kann.

10.2 Probes und Bereitschaft

Signal	Bedeutung
Bereitschaftsendpunkt antwortet mit „nicht bereit“	Der Prozess läuft, die Initialisierung ist nicht abgeschlossen. Normal während Installation und Neustart; dauerhaft ist es ein Symptom (10.6)
Bereitschaftsendpunkt antwortet mit OK	Alle Startprüfungen sind durchlaufen, der reale Server bedient die API
Prozess beendet sich mit Fehlermeldung	Eine harte Startabhängigkeit fehlt oder ist falsch. Die Meldung benennt sie; der Orchestrator startet den Container neu

Bereitschaft ist bewusst von Prozess-Lebendigkeit getrennt. Der Bereitschaftsendpunkt bleibt so lange negativ, bis die Initialisierung einschließlich Katalog-Gate und Schema-Synchronisation abgeschlossen ist.

Für den Betrieb wichtig: **Der Backend-Container wird ausschließlich auf Bereitschaft geprüft. Ein Liveness-Signal wird nicht abgefragt.** Ein Prozess, der zwar läuft, aber nicht mehr arbeitet, wird deshalb nie automatisch neu gestartet. Antwortet er weiterhin auf dem Bereitschaftsendpunkt, bleibt er sogar im Service-Endpoint und nimmt Anfragen entgegen. Das folgt aus dem Bootstrap-Verhalten: Eine Lebendigkeitsprobe gegen denselben Endpunkt würde einen Prozess töten, der korrekt auf eine noch nicht provisionierte Abhängigkeit wartet. Wer eine Lebendigkeitsprobe will, fügt sie bewusst hinzu und muss dabei den Startfall abdecken.

pdf-core wird über seinen eigenen Endpunkt geprobt und hat sowohl Bereitschafts- als auch Lebendigkeitsprüfung. Die mitgelieferten Begleitdienste bringen ihre eigenen Probes mit.

10.3 Was dauerhaft im Hintergrund läuft

Eine laufende Instanz betreibt eine Reihe von Schleifen. Sie zu kennen erklärt fast jedes Verhalten, das von außen „verzögert“ aussieht.

Prozess	Aufgabe	Verhalten bei Störung
Outbox-Publikation	Ereignisse auf den Event-Bus stellen	Hoher Takt; ein nicht erreichbarer Bus verzögert Abonnenten, nicht die Fachlogik
Outbox-Verankerung	Audit-Einträge schreiben, Merkle-Checkpoints bilden, Roots zeitstempeln	Zählt Fehlversuche je Ereignis; nach 50 Versuchen Dead-Letter mit deutlichem Log
Zeitstempel-Nachlauf	Checkpoints ohne Zeitstempel nachträglich stempeln	Läuft, bis die TSA wieder antwortet
PDF-Regenerierung	Auf Lifecycle-Ereignisse hin rendern/amendieren, Ergebnis ablegen	Ein Ereignis wird höchstens einmal zugestellt; Fehlschläge fängt der Retry-Sweep
Regenerations-Retry-Sweep	Entitäten finden, deren gespeichertes PDF nicht dem aktuellen Dokument entspricht	Begrenzte Stapel, begrenzte Versuche je Entität, wachsender Abstand; ein dauerhaft unrenderbarer Vertrag blockiert die heilbaren nicht
Föderations-Versand	Vertrags-PDFs an Peers ausliefern	Fehlschläge landen in der Retry-Tabelle; ein Scheduler versucht sie erneut
Löschungs-Retry	Offene Peer-Löschanforderungen erneut zustellen	Eigene Warteschlange, gleicher Takt wie der Föderations-Retry
Statuslisten-Publikation	Widerrufseinträge der Lifecycle-Credentials veröffentlichen	Eigener Arbeiter, unabhängig vom Request-Pfad
Ablauf-Job	Verträge nach Ablaufdatum auf EXPIRED setzen und das Ereignis publizieren	Loggt Fehler je Vertrag und macht weiter; ein Vertrag ohne Expiration-Policy wird übersprungen (Kapitel 04)
Webhook-Zustellung	Ereignisse an registrierte Abonnenten liefern	Zustellungen werden protokolliert und sind quittierbar
Compliance-Sweep (CronJob)	Compliance-Risiken planmäßig erkennen (ADR-32)	Eigenes Kommando, eigener Pod; ein erkanntes Risiko ist kein Job-Fehlschlag

Der Compliance-Sweep ist bewusst als eigener geplanter Job und nicht als Schleife im Server gebaut. Ein fehlgeschlagener Lauf ist damit ein sichtbares Objekt im Orchestrator statt einer Logzeile, der Takt lässt sich ohne Neustart der Instanz ändern, und Mehrfachläufe sind strukturell ausgeschlossen. Er öffnet nur die Datenbank und läuft deshalb weiter, wenn eine Nachbarkomponente gestört ist, also genau dann, wenn er gebraucht wird (Kapitel 09).

10.4 Metriken

Das Backend exponiert einen Prometheus-Endpunkt mit zwei HTTP-Basismetriken: Anzahl und Dauer der Requests, jeweils nach Methode, Pfad und Statuscode. Damit sind Fehlerraten, Latenzverteilungen und Lastprofile pro Endpunkt auswertbar, insbesondere die typischen Sättigungssignale (Anteil 429 aus dem Anfragebudget, Anteil 5xx, Latenz der PDF-Export- und Signatur-Pfade).

Der Endpunkt liegt außerhalb der authentifizierten API und außerhalb des Anfragebudgets. Ein Scraping-Ziel bringt die Instanz nicht mit; das Einsammeln der Metriken richtet der Betreiber in seiner eigenen Monitoring-Infrastruktur ein.

Anwendungsfachliche Metriken (Anzahl verankerter Checkpoints, Länge der Retry-Warteschlangen, Zahl offener Compliance-Risiken) exportiert das Backend **nicht** als Prometheus-Zeitreihen. Die entsprechenden Beobachtungen laufen über Logs, über die Compliance-API und über die Datenbank (10.5).

10.5 Was der Betreiber sieht

Logs

Die Meldungen, auf die es ankommt:

- **Jeder verankerte Checkpoint** mit Sequenz, Eintragszahl, Root und Zeitstempel-Status. Bleiben diese Meldungen aus, während das System Ereignisse produziert, steht die Verankerung.
- **Jede fehlgeschlagene Verankerung** mit Wiederholungshinweis.
- **Jedes dead-letterte Audit-Ereignis**, unübersehbar formuliert. Ein solches Ereignis ist **nicht** im Audit-Trail und braucht einen Operator. Es ist der einzige Weg, auf dem der Trail eine Lücke bekommt.
- **Wartemeldungen des Token-Öffnens** beim Start.
- **Wiederholversuche des Föderations-Versands** und des Regenerations-Sweeps.
- **Nachgeholte TSA-Zeitstempel.**
- **Diagnose bei einem Inhalts-Mismatch** eines empfangenen Peer-PDFs: die divergierende Stelle und der Hash der eingebetteten Payload.

Zustände in der Datenbank

Drei Tabellenbereiche sind Betriebssicht, nicht nur Fachdaten:

Bereich	Frage, die er beantwortet
Outbox mit Dead-Letter-Markierung	Gibt es Ereignisse, die nie in den Trail gelangt sind, und warum?
Retry-Tabellen für Föderationsversand und Peer-Löschung	Welche Sendungen bzw. Löschanforderungen hängen, seit wann und mit welchem Fehler?
Risiko-Register des Compliance-Monitorings	Welche Verstöße sind offen, seit wann sind sie bekannt, welche wurden geschlossen?

API-Sichten

- **Checkpoint-Head:** die publizierbare Spitze des Audit-Trails; fehlender Zeitstempel zeigt eine TSA-Störung.
- **Compliance-Sweep auf Anfrage:** was gerade falsch ist, unabhängig davon, was zuletzt alarmiert wurde.
- **Workflow-Gate-Läufe:** warum ein Lebenszyklusübergang blockiert wurde oder auf eine Prüfentscheidung wartet.

- **Erasure-Status:** wie weit eine Löschung über die Instanzgrenze gediehen ist.
- **Schlüsselinventar:** welche Schlüssel die Instanz hält, mit Zweck und aktiver Version.
- **Webhook-Zustellprotokoll:** ob ein Abonnent seine Ereignisse bekommen und quittiert hat.

Ereignisse für externe Systeme

Registrierte Abonnenten erhalten benannte Lebenszyklus-Ereignisse (Vertrag angelegt, eingereicht, freigegeben, abgelehnt, verhandelt, beendet, abgelaufen; Vorlage angelegt, freigegeben, aktualisiert, registriert, ausgemustert) sowie den Compliance-Alarm bei der Ersterkennung eines Risikos.

Subscriptions und Zustellprotokoll liegen im Prozessspeicher: Sie überleben keinen Neustart und werden zwischen mehreren Replikaten nicht geteilt. Wer dauerhafte Zustellung braucht, registriert seine Subscriptions nach jedem Neustart neu oder betreibt die Instanz mit einem Replikat.

10.6 Fehlerbilder

Die folgenden Bilder sind aus dem Verhalten der Artefakte abgeleitet bzw. im Betrieb der Demonstrationsumgebung beobachtet worden. Konkrete Behebungsschritte stehen im Deployment-Leitfaden.

Symptom	Ursache	Richtung der Behebung
Pod läuft, Bereitschaft bleibt dauerhaft negativ, Log wiederholt Wartemeldungen zum PKCS#11-Token	Das Token ist nicht provisioniert oder nicht gemountet; die Provisionierung ist nicht gelaufen oder fehlgeschlagen	Provisionierungs-Job und Token-Volume prüfen
Prozess beendet sich sofort mit einer Meldung zum DID-Dokument	Das ausgelieferte DID-Dokument passt nicht zum Schlüssel im Token, typisch nach einer Neu-Provisionierung ohne Neuerzeugung des Dokuments	DID-Dokument aus dem Token neu ableiten
Prozess beendet sich mit einer Meldung zur Schlüsselvereinbarung	Der publizierte keyAgreement -Schlüssel und der Token-Schlüssel passen nicht zusammen, oder das Token kann die Ableitung nicht	Dokument und Token abgleichen; bei einem produktiven HSM die Ableitungsfähigkeit für die verwendete Kurve prüfen
Prozess beendet sich mit einer Meldung zum Federated Catalogue	Katalog unvollständig konfiguriert oder nicht funktional erreichbar	Katalog-Konfiguration vervollständigen oder Katalog abschalten
Prozess beendet sich mit einer Meldung zu einer gepinnten Konfigurationsdatei	Eine attestierte Datei fehlt oder weicht vom Pin ab; oder die Pin-Liste ist syntaktisch falsch	Datei oder Pin korrigieren (Kapitel 09)
Jeder Lebenszyklusübergang wird blockiert, obwohl der Vertrag korrekt ist	Der Workflow-Gate-Executor ist nicht erreichbar; das Gate ist fail-closed	Executor-Flow prüfen; der Gate-Lauf nennt die Ursache
Ein Audit-Abruf antwortet mit einem Executor-Fehler	Der Audit-Executor ist nicht erreichbar oder antwortet nicht verwertbar	Executor-Flow prüfen
Ein Vertrag ist nicht exportierbar und wird nicht an den Peer verschickt	Die PDF-Regeneration ist fehlgeschlagen	Der Retry-Sweep holt es nach; die Ursache steht im Log des Regenerators
Export antwortet mit „wird gerade regeneriert“	Eine Regeneration lief länger als das Wartefenster des Exports	Erneut abrufen; anhaltend deutet es auf eine pdf-core- oder Speicherstörung
Verifikation meldet „Artefakt nicht authentisch“	Die gespeicherten Bytes sind nicht die geschriebenen: Manipulation oder Substitution im Speicher	Als Sicherheitsvorfall behandeln; die Merkle-Beweise bleiben gültig und erlauben den Abgleich (Kapitel 07, 09)
Export, Verifikation oder Bundle eines Vertrags antworten „Inhalt gelöscht“	Die Inhaltsschlüssel des Vertrags wurden vernichtet	Erwartetes Verhalten nach einer Löschung; Erasure-Status zeigt Zeitpunkt und Akteur
Peer-Versand hängt in der Retry-Tabelle	Peer nicht erreichbar, DID-Auflösung scheitert oder das Agreement Credential wird abgelehnt	Peer-Erreichbarkeit und DID-Auflösung prüfen; Details im Log und im Retry-Eintrag
Föderation ist abgeschaltet, obwohl konfiguriert	Der Trust-PDP ist nicht gesetzt oder nicht erreichbar; der Gate ist fail-closed	Policy-Endpunkt prüfen; jede Ablehnung liegt als Incident im Trail
Eine Sendung einer Gegenseite wird abgelehnt, obwohl der Peer	Der mitgeschickte Vollmachtsnachweis verifiziert nicht: Aussteller nicht für peer	Issuer-Vertrauen der empfangenden Instanz prüfen

Symptom	Ursache	Richtung der Behebung
bekannt ist	zugelassen, Organisation nicht zugestanden, Credential abgelaufen oder widerrufen	(Kapitel 05, 08)
Ein Wallet-Login schlägt reproduzierbar fehl	Aussteller nicht für <code>login</code> zugelassen, Organisation nicht zugestanden, Statusliste nicht erreichbar, oder Zertifikatskette ohne konfigurierte Vertrauensanker	Trust-Konfiguration prüfen; der Fehler benennt die Stufe
Antworten mit <code>429</code>	Das Anfragebudget je Credential ist ausgeschöpft	Aufrufverhalten prüfen; das Budget ist konfigurierbar
Webhook-Abonnenten erhalten nach einem Neustart nichts mehr	Subscriptions liegen im Prozessspeicher	Neu registrieren
Signatur-Einreichung wird abgelehnt, obwohl das Dokument gültig signiert ist	Kein DSS konfiguriert oder nicht erreichbar; oder das Byte-Prefix stimmt nicht, weil ein anderes als das vorbereitete Dokument signiert wurde	DSS prüfen; Signatar muss exakt das vorbereitete Dokument signieren (Kapitel 06)
Externe PDF-Prüfprogramme melden „nach der Signatur geändert“	Der Provenance-Re-Anchor nach der Signatur (ADR-26)	Erwartetes Verhalten; die eigene Verifikation weist genau diese eine Revision nach (Kapitel 07)

10.7 Kapazitäts- und Betriebsgrenzen, die man kennen muss

- **Ein Replikat pro Instanz.** Die Webhook-Subscriptions liegen im Prozessspeicher; mehrere Replikate hätten unterschiedliche Sichten. Die Skalierung liegt heute in der Vertikalen.
- **Die Verankerung ist strikt sequenziell.** Sie hängt an TSA- und Speicher-Roundtrips. Große Ereignis-Rückstände arbeiten sich in Stapeln ab, nicht sprunghaft.
- **Ein Voll-Trail-Read ist begrenzt** (maximal 500 Checkpoints rückwärts), damit ein Audit-Abruf innerhalb seiner Frist bleibt.
- **Sicherungen verzögern die Vollendung einer Löschung.** Ein heute vernichteter Schlüssel existiert in der gestrigen Datenbanksicherung weiter; die Löschzusage reicht so weit wie das Aufbewahrungsfenster, und nach einer Wiederherstellung müssen die seither erfolgten Löschungen nachgezogen werden (Backup-Leitfaden).
- **Die Schlüsselvernichtung entfernt keine Chiffre aus dem Speicher.** Entfernt werden die Archiv-Snapshots; die verschlüsselten Vertragsartefakte bleiben unlesbar liegen (Kapitel 07).

Anhang A Schnittstellenreferenz

Dieser Anhang listet die Schnittstellen einer DCS-Instanz auf drei Ebenen: die HTTP-API-Gruppen des Backends, die Ereignisse auf dem internen Event-Bus und die öffentlichen well-known-Endpunkte, über die andere Instanzen und externe Prüfer die Instanz auflösen.

Die Referenz beschreibt Gruppen und Zweck, nicht einzelne Parameter. Die vollständige, maschinenlesbare API-Beschreibung liefert jede laufende Instanz selbst: eine Swagger-Oberfläche und eine OpenAPI-3-Spezifikation.

Alle fachlichen APIs sind, sofern nicht anders vermerkt, durch das OIDC-Bearer-Token geschützt; die Scopes im Access Token entsprechen den Rollen-Claims des Nutzers. Maschinelle Aufrufer (Maschinen-Identitäten und Contract Target Systems) authentifizieren sich mit Client-Credentials-Tokens ihrer eigenen, über die Registry ausgestellten OAuth2-Clients; ihre Rollen liest das Backend anhand der Client-ID aus der Registry, nie aus Token-Claims. Öffentlich (ohne Login) sind die well-known-Endpunkte, der C2PA-Manifestabruf, die Semantic-Hub-Auflösungsendpunkte sowie die von Wallets und Peer-Instanzen aufgerufenen Callback-Pfade.

A.1 HTTP-API-Gruppen

Gruppe	Basispfad	Zweck
Auth	/auth/...	Endnutzer-Login: OIDC-Session-Management und OpenID4VP-Presentation (inkl. PID-Presentation)
TemplateRepository	/template/...	Lebenszyklus der Vertragsvorlagen
SemanticHub	/semantic/...	Versionierte JSON-LD-Kontexte, SHACL-Shapes, Ontologien, Validierungsprofile, Klausel-Katalog
TemplateCatalogueIntegration	/catalogue/template/...	Lesende Anbindung an den XFSC Federated Catalogue
ContractWorkflowEngine	/contract/..., /machine-identities/...	Vertragslebenszyklus; Zielsystem-Registry und Maschinen-Identitäten
SignatureManagement	/signature/...	Signatur-Ceremonies, Prüfung, Provenance, Compliance
PDFGeneration	/pdf/..., /contract/export/..., /template/export/...	Export und unabhängige Verifikation der PDF-Repräsentationen
ContractStorageArchive	/archive/...	Langzeitarchiv abgeschlossener Verträge, Löschung und Löschststatus
ProcessAuditAndCompliance	/pac/...	Audit-Abruf, Reports, Monitoring, Merkle-Checkpoints, Workflow-Gate-Läufe
DcsToDcs	/peer/...	Föderation: PDF-/JAdES-Austausch und Löschanforderung zwischen DCS-Instanzen
KeyInventory	/admin/hsm-keys	Lesendes Inventar der HSM-Schlüssel (Sys. Administrator)
DIDService	/.well-known/...	DID-Dokument, Agreement Credential, Föderationsregeln (öffentlich)
C2PAService	/c2pa/...	Öffentlicher Abruf des C2PA-Manifests eines Vertrags
Webhook-Plattform	/orice/...	Abonnements, Zustellungen und Callbacks für externe Ereignisempfänger

Auth (/auth/...)

Zwei Ablauffamilien: der OIDC-Login mit OpenID4VP-Presentation (Session-basierte Anmeldung am DCS) und die davon unabhängige, einmalige PID-Presentation (Identitätsnachweis ohne Session).

Endpunktgruppe	Endpunkte	Zweck
Login-Flow	POST /auth/login, POST /auth/login/renew, POST /auth/login/challenge, GET /auth/login/status	OID4VP-Login starten/verlängern, Login-Challenge binden, Status-Polling für das Frontend
OIDC-Session	GET /auth/consent, GET /auth/callback, POST /auth/refresh, GET /auth/logout, GET /auth/logout-complete	Consent und Callback, Token-Refresh über HttpOnly-Cookie, Abmeldung
Wallet-Presentation	GET/POST /auth/presentation/request/{state}, POST /auth/presentation/callback	Signiertes Authorization-Request-Objekt (Request-by-Reference) für die Wallet; Direct-Post-Rückkanal
PID-Presentation	POST /auth/pid/presentation, POST /auth/pid/presentation/renew, GET/POST /auth/pid/presentation/request/{state}, POST /auth/pid/presentation/callback, GET /auth/pid/presentation/status	Einmalige PID-Vorlage ohne Session, gleiche Request-/Callback-/Polling-Mechanik

TemplateRepository (/template/...)

Endpunktgruppe	Endpunkte	Zweck
Anlage und Pflege	POST /template/create, POST /template/copy, PUT /template/update, POST /template/update_manage	Vorlage anlegen, kopieren, inhaltlich bzw. verwaltend ändern
Freigabeprozess	POST /template/submit, POST /template/verify, POST /template/approve, POST /template/reject	Zur Prüfung einreichen, semantisch verifizieren, freigeben oder ablehnen
Abruf und Suche	GET /template/search, GET /template/retrieve, GET /template/retrieve/{did}, GET /template/{did}, GET /template/history/{did}	Vorlagen suchen, listen, per DID auflösen, Historie einsehen
Provenance und Audit	GET /template/provenance/{did}, GET /template/audit	Herkunftsnachweis und Audit-Einträge einer Vorlage
Verteilung	POST /template/register, POST /template/publish, POST /template/archive	Version registrieren, im Federated Catalogue veröffentlichen, ausmustern

SemanticHub (/semantic/...)

Sämtliche lesenden Endpunkte dieser Gruppe sind öffentlich: Erzeugte Artefakte tragen Hub-Anker, die externe Prüfer ohne DCS-Login auflösen können müssen. Schreibend ist nur die Schema-Verwaltung, und die ausschließlich für den Template Manager.

Endpunktgruppe	Endpunkte	Zweck
Schema-Verwaltung	POST /semantic/schema/register , POST /semantic/schema/rollback	Neue Schema-Version registrieren, auf eine frühere zurücksetzen (Template Manager)
Schema-Abruf	GET /semantic/schema/retrieve , GET /semantic/schema/versions , GET /semantic/schema/list	Aktive oder benannte Version abrufen, Versions- und Schemaliste
Artefakt-Auflösung	GET /semantic/context/{name} , GET /semantic/shapes/{name} , GET /semantic/ontology/{name} , GET /semantic/profile/{name}	Kontext, Shapes, geparste RDF-Konfiguration und Validierungsprofil unter stabilen Ankern ausliefern
Klausel-Katalog	GET /semantic/clauses	Klauseltypen für die Palette des Vorlagen-Builders

TemplateCatalogueIntegration (/catalogue/template/...)

Endpunkte	Zweck
GET /catalogue/template/retrieve , GET /catalogue/template/retrieve/{did} , GET /catalogue/template/search	Im Federated Catalogue veröffentlichte Vorlagen listen, per DID abrufen und durchsuchen (menschliche Vertragsrollen)

ContractWorkflowEngine (/contract/... , /machine-identities/...)

Endpunktgruppe	Endpunkte	Zweck
Anlage und Pflege	POST /contract/create , PUT /contract/update , POST /contract/renew	Vertrag aus Vorlage anlegen, im Entwurf ändern, Folgevertrag ableiten
Angebot	POST /contract/offer , POST /contract/withdraw , POST /contract/submit	Der Gegenpartei anbieten, vor Freigabe zurückziehen, Zustandsübergang einreichen
Verhandlung	POST /contract/negotiate , PUT /contract/negotiation_draft , GET/DELETE /contract/negotiation_draft/{did} , POST /contract/respond	Change Requests stellen, Verhandlungsentwurf zwischenspeichern, Gegenvorschlag annehmen/ablehnen
Entscheidung	POST /contract/approve , POST /contract/reject , GET /contract/review	Freigeben oder ablehnen, Review-Sicht der offenen Aufgaben
Abruf und Suche	GET /contract/retrieve , GET /contract/retrieve/{did} , GET /contract/{did} , GET /contract/history/{did} , GET /contract/search , GET /contract/templates	Verträge listen, per DID auflösen (parteibeschränkt), Historie, Suche, verfügbare Vorlagen
Abschluss und Betrieb	POST /contract/store , POST /contract/terminate , GET /contract/kpis/{did} , POST /contract/audit	Evidenz sichern, beenden, KPI-Beobachtungen einsehen, Audit-Auszug
Deployment	POST /contract/deploy , POST /contract/target/designate , POST /contract/deployment/callback	An das designierte (oder ein explizit gewähltes) Zielsystem ausrollen; Designation setzen oder löschen; Rückmeldung des Zielsystems (Ack/Status, KPI-Werte), authentifiziert als dessen eigener OAuth2-Client und nur für Deployments, die an genau dieses Ziel gingen
Zielsystem-Registry	GET/POST/PUT/DELETE /contract/targets , POST /contract/targets/{id}/credential	Registry administrieren (Schreibzugriff Sys. Administrator und Integration Manager, Lesen zusätzlich Contract Manager); Löschen wird verweigert, solange ein Vertrag das Ziel designiert; Callback-Credential ausstellen/rotieren, wobei das Secret genau einmal erscheint und das vorherige sofort seine Gültigkeit verliert
Maschinen-Identitäten	GET/POST /machine-identities , PUT/DELETE /machine-identities/{id} , POST /machine-identities/{id}/credential	Registry verwalten: Anlegen provisioniert den OAuth2-Client und liefert das Secret genau einmal; Deaktivieren weist Aufrufe sofort ab; Löschen entfernt auch den Client; Rotation invalidiert das alte Secret unmittelbar (Sys. Administrator)

SignatureManagement (/signature/...)

Endpunktgruppe	Endpunkte	Zweck
Wallet-Ceremony	POST /signature/request , GET /signature/request/{ceremony_id} , POST /signature/request/{ceremony_id}/publish , GET/POST /signature/request/{ceremony_id}/object , GET /signature/request/{ceremony_id}/document , GET /signature/request/{ceremony_id}/payload , POST /signature/request/{ceremony_id}/callback	Ceremony starten, Status abfragen, Request-Objekt, zu signierendes PDF und kanonische JSON-LD-Payload an die Wallet ausliefern, Ergebnis über den nonce-gebundenen Callback entgegennehmen
Direkter Signaturfluss	POST /signature/prepare , POST /signature/submit	Vorbereitetes, byte-gepinntes PDF ausliefern und fertige externe Signatur einreichen
Prüfung	POST /signature/verify , POST /signature/validate , POST /signature/compliance	Integrität und Provenance verifizieren, DSS-gestützte Validierung, Niveau-Prüfung
Abruf und Nachweis	GET /signature/retrieve , GET /signature/retrieve/{did} , GET /signature/provenance/{did} , GET /signature/view , GET /signature/audit	Signierliste, Signaturumschlag je Vertrag, Provenance-Kette, Compliance-Viewer mit Detail je Signatur, Audit-Einträge
Verwaltung	POST /signature/revoke	Signatur widerrufen

PDFGeneration (/pdf/... , Export-Bundles)

Endpunktgruppe	Endpunkte	Zweck
PDF-Export	GET /pdf/export/contract/{did} , GET /pdf/export/template/{did}	Deterministisch gerendertes PDF eines Vertrags bzw. einer Vorlage
Bundle-Export	GET /contract/export/{did} , GET /template/export/{did}	ZIP-Bundle mit PDF und maschinenlesbaren Artefakten
Verifikation	GET /pdf/verify/contract/{did} , GET /pdf/verify/template/{did}	Unabhängiger Neurender-Abgleich mit benannter Fehlerklasse

ContractStorageArchive (/archive/...)

Endpunkte	Zweck
POST /archive/store	Abgeschlossenen Vertrag mit Evidenz archivieren
GET /archive/retrieve , GET /archive/search	Archiveinträge abrufen und strukturiert durchsuchen (Volltext, Name, Zustand, Partei, Gültigkeitszeitraum, Annotations-Tag)
POST /archive/annotate	Eintrag mit Zusammenfassung/Tags versehen
DELETE /archive/delete	Eintrag mit Begründung löschen: Soft-Delete des Eintrags, Entfernen der Snapshots und Vernichtung der Inhaltsschlüssel auf beiden Instanzen (ADR-28)
GET /archive/erasure-status	Stand der Schlüsselvernichtung: lokale Zerstörungsnachweise und offene Peer-Anforderungen
GET /archive/statistics	Archiv-Dashboard: Bestands- und Speicherstatistik, letzte Archivaktionen, demnächst auslaufende Verträge, Evidenz-Vollständigkeit
GET /archive/audit	Audit-Einträge des Archivs

ProcessAuditAndCompliance (/pac/...)

Endpunkte	Zweck
POST /pac/audit	Audit-Abruf: Evidenz sammeln und extern bewerten lassen (Begründung Pflicht, Abruf wird auditert)
GET /pac/report	Audit-Report erzeugen (JSON/CSV/PDF)
POST /pac/report	Incident-Report zu einer Ressource erfassen
GET /pac/monitor	Compliance-Sweep auf Anfrage; auch ein Lauf ohne Befund wird auditert
GET /pac/audit/checkpoint/head , GET /pac/audit/checkpoint/{seq} , GET /pac/audit/checkpoint/proof/{entry_cid}	Aktueller Merkle-Checkpoint-Kopf, ein bestimmter Checkpoint, Inklusionsbeweis für einen Audit-Eintrag
GET /pac/workflow-gates/{run_id} , POST /pac/workflow-gates/review	Workflow-Gate-Lauf lesen; eine REVIEW - Entscheidung mit Begründung festhalten

DcsToDcs (/peer/...)

Föderationsschnittstelle zwischen zwei DCS-Instanzen. Alle Endpunkte stehen unter dem Trust Gate (Agreement Credential plus lokaler Policy-Endpunkt, fail-closed) und werden per did:web-Challenge-Response im Request-Body authentifiziert, nicht per Session-Token.

Endpunkte	Zweck
POST /peer/contracts/pdf	Eingang eines Vertrags-PDFs mit Zustand, optional JAdES, Vollmachtsnachweis und gewickeltem Inhaltsschlüssel
POST /peer/contracts/erase	Löschanforderung: Vernichtung der Inhaltsschlüssel dieses Vertrags auf der empfangenden Instanz
GET /peer/contracts/provenance	Gespeicherte JAdES-Provenienz eines empfangenen Vertrags

KeyInventory (/admin/hsm-keys)

Endpunkt	Zweck
GET /admin/hsm-keys	Lesendes Inventar: Label, Zweck und aktive Version jedes HSM-Schlüssels der Instanz (Sys. Administrator). Rotation selbst ist ein Betriebsverfahren, keine API-Handlung

C2PAService (/c2pa/...)

Endpunkt	Zweck
GET /c2pa/manifest/{contract_id}	Öffentlicher, nicht authentifizierter Abruf des C2PA-Manifest-Stores eines Vertrags; optional die Kettenaufzählung

A.2 Ereignisse auf dem Event-Bus

Alle Backend-Domänen publizieren ihre Ereignisse als CloudEvents über einen gemeinsamen NATS-Topic. Die Zustellung folgt dem Transactional-Outbox-Muster: Der Handler persistiert das Ereignis in derselben Datenbanktransaktion wie seine Änderung; ein Outbox-Prozessor publiziert es auf dem Bus und verankert es als hash-verketteten Audit-Trail-Eintrag. Konsumenten unterscheiden Ereignisse über den CloudEvent-Typ, nicht über Subtopics.

Abonnenten innerhalb der Instanz:

Konsument	Reagiert auf	Wirkung
DCS-to-DCS-Synchronizer	Lebenszyklus- und Remote-Sync-Ereignisse, PDF_REGENERATED	Versand des Vertrags-PDFs an die Peer-Instanz bei versandpflichtigen Zuständen
PDF-/C2PA-Regenerator	Lebenszyklusereignisse von Verträgen und Vorlagen	Hintergrund-Neurendering und Anfügen einer C2PA-Lifecycle-Assertion; Exporte werden nie on demand erzeugt
Webhook-Plattform	Domänenereignisse mit Webhook-Abbildung	Weiterleitung an registrierte externe Abonnenten
Auto-Deployment	APPLIED_SIGNATURE	Anstoß der Zielsystem-Auslieferung über dasselbe Kommando wie der manuelle Deploy
Event-Logger (optional)	alle Ereignisse	Diagnose-Mitschnitt, nur wenn ausdrücklich eingeschaltet

Ereignistypen je Domäne (CloudEvent-Typ auf dem Bus; zugleich der Ereignistyp im Audit-Trail):

Domäne	Ereignistypen
Contract Workflow Engine	CREATE_CONTRACT , UPDATE_CONTRACT , SUBMIT_CONTRACT , OFFER_CONTRACT , WITHDRAW_CONTRACT , NEGOTIATE_CONTRACT , ACCEPT_RESPOND_CONTRACT , REJECT_RESPOND_CONTRACT , INCREASE_CONTRACT_VERSION , APPROVE_CONTRACT , REJECT_CONTRACT , VERIFY_CONTRACT , REVIEW_CONTRACT , TERMINATE_CONTRACT , RENEW_CONTRACT , CONTRACT_EXPIRED , RECORD_EVIDENCE , AUDIT_CONTRACT , EXPORT , SEARCH_CONTRACT , RETRIEVE_ALL_CONTRACTS , RETRIEVE_CONTRACT_BY_ID , RETRIEVE_CONTRACT_HISTORY_BY_DID , RETRIEVE_ALL_TEMPLATES , CONTRACT_ACCESS_DENIED
Föderation und Synchronisation	REMOTE_SYNC , REMOTE_SYNC_REQUEST , REMOTE_ACTION_REQUEST , OUTDATED_PEER , PDF_REGENERATED
Archiv und Löschung	STORE_ARCHIVED_CONTRACT , RETRIEVE_ARCHIVED_CONTRACTS , DELETE_ARCHIVED_CONTRACT , ANNOTATE_ARCHIVED_CONTRACT , KEY_SHREDDER
Template Repository	CREATE_CONTRACT_TEMPLATE , COPY_CONTRACT_TEMPLATE , UPDATE_CONTRACT_TEMPLATE , SUBMIT_CONTRACT_TEMPLATE , VERIFY_CONTRACT_TEMPLATE , APPROVE_CONTRACT_TEMPLATE , REJECT_CONTRACT_TEMPLATE , REGISTER_CONTRACT_TEMPLATE , PUBLISH_CONTRACT_TEMPLATE , ARCHIVE_CONTRACT_TEMPLATE , AUDIT_CONTRACT_TEMPLATE , SEARCH_CONTRACT_TEMPLATE , RETRIEVE_ALL_CONTRACT_TEMPLATES , RETRIEVE_CONTRACT_TEMPLATE_BY_ID
Signature Management	SIGNING_REQUEST , APPLY_SIGNATURE , APPLIED_SIGNATURE , VERIFY_SIGNATURE , VALIDATE_SIGNATURE , REVOKE_SIGNATURE , COMPLIANCE_VALIDATION
Process Audit & Compliance	PAC_AUDIT_EXECUTED , PAC_REPORT_GENERATED , PAC_COMPLIANCE_MONITOR , PAC_COMPLIANCE_RISK , PAC_INCIDENT_REPORT , PAC_TRUST_GATE_DENIAL , CONFIG_INTEGRITY_ATTESTATION , TEMPLATE_POLICY_AUDIT_FINDING , TEMPLATE_APPROVAL_PROVENANCE_AUDIT_FINDING , CONTRACT_CONTENT_POLICY_AUDIT_FINDING
Catalogue Integration	RETRIEVE_ALL_TEMPLATE_CATALOGUE , RETRIEVE_TEMPLATE_CATALOGUE_BY_ID , SEARCH_TEMPLATE_CATALOGUE
Auth	OID4VP_PRESENTATION_SUCCEEDED , OID4VP_PRESENTATION_FAILED

Webhook-Plattform (/orce/...)

Die Webhook-Plattform liegt außerhalb der generierten API und übersetzt interne Ereignisse in benannte, abonnierbare Alert-Ereignisse.

Abonnierbare Ereignisse: contract.created , contract.submitted , contract.approved ,
contract.rejected , contract.negotiated , contract.terminated , contract.expired , template.created ,
template.approved , template.updated , template.registered , template.deprecated ,
compliance.risk_detected .

Endpunkt	Zweck
GET /orce/events	Katalog der abonnierbaren Ereignisnamen
GET /orce/webhooks , POST /orce/webhooks , DELETE /orce/webhooks/{id}	Subscriptions listen, registrieren (Ereignis, Callback-URL, Secret), löschen
POST /orce/callbacks	Quittung des Abonnenten für eine Zustellung
GET /orce/deliveries	Zustellprotokoll (Ergebnis, Statuscode, Quittungsstatus)

Subscriptions und Zustellprotokoll liegen im Prozessspeicher und überdauern keinen Neustart (Kapitel 10).

A.3 Well-known-Endpunkte

Öffentliche, nicht authentifizierte Endpunkte an der Origin-Wurzel der Instanz; dieselben Dokumente sind zusätzlich unter dem API-Präfix erreichbar. Sie sind die Auflösungsbasis des Föderations-Trust-Modells: Peer-Instanzen und externe Prüfer holen sich hier Identität, Regelakzeptanz und Regeln einer Instanz, bevor Vertragsdaten fließen.

Endpunkt	Inhalt	Zweck
GET /.well-known/did.json	DID-Dokument der Instanz (did:web)	Instanzidentität; öffentliche Schlüssel zu den im HSM gehaltenen privaten Schlüsseln, einschließlich des Schlüsselvereinbarungs-Schlüssels
GET /.well-known/dcs-agreement-credential.json	Selbstsigniertes Agreement Credential	Nachweis, dass die Instanz die Föderationsregeln akzeptiert hat; Prüfgegenstand des Trust Gates auf Ein- und Ausgangspfad
GET /.well-known/dcs-federation-rules.md	Föderationsregeln	Das Regeldokument, auf das sich das Agreement Credential per Hash bezieht

Ergänzend zur Auflösung durch Dritte:

- **OID4VP-Metadaten** werden nicht als eigenes well-known-Dokument ausgeliefert: Die Verifier-Metadaten stecken im signierten Authorization-Request-Objekt, das die Wallet per Request-by-Reference abruft; der Verifikationsschlüssel wird über die im JAR-Header mitgelieferte Zertifikatskette und den DNS-gebundenen Client-Identifizierer verankert.
- Die **OIDC-Discovery** liefert nicht das DCS-Backend, sondern der mitbetriebene OAuth2-Provider.
- GET /c2pa/manifest/{contract_did} ist das öffentliche Gegenstück für die Provenance-Auflösung eines Vertrags.
- Der **Metrik-Endpunkt** liegt außerhalb der authentifzierten API und außerhalb des Anfragebudgets (Kapitel 10).

Anhang B Entscheidungsarchiv

Verweisliste auf die Architecture Decision Records unter `docs/` . Jeder Eintrag nennt die These des ADRs und, wo relevant, seine Beziehung zu anderen ADRs. Begründungen und Konsequenzen stehen ausschließlich im jeweiligen ADR. ADRs ohne abweichenden Vermerk sind unverändert in Kraft.

Projektreihe

ADR	These und Status
ADR-1	PKCS#11/HSM ist der einzige Key-Custody-Mechanismus für alle DCS-eigenen Signaturschlüssel. In Teilen superseded durch ADR-12: PAdES-Vertragssignaturen zählen nicht zu den DCS-Key-Custody-Touchpoints.
ADR-2	Eine einzige Vertrags-State-Machine mit einer Transitionstabelle. Der ursprünglich implizierte Full-State-Broadcast-Synchronizer ist durch ADR-13 zurückgezogen.
ADR-3	Signatursemantik: Identitätsbindung unter der Signatur, Embed-then-Sign. Die org-key-basierte AES über das PDF ist durch ADR-12 und ADR-20 zurückgezogen.
ADR-4	C2PA-Provenance wird doppelt ausgeliefert: eingebettet als JUMBF im PDF und als standardkonformes Remote-Manifest.
ADR-5	Festlegung je XFSC-Komponente, ob sie übernommen oder substituiert wird; schließt Graph-Store und Deployment-Lifecycle des übernommenen Federated Catalogue ein.
ADR-6	Vertragsbedingungen sind echtes ODRL unter einem DCS-Profil und werden serverseitig durchgesetzt. Nur der Evaluator-Mechanismus ist durch ADR-11 superseded.
ADR-7	Vertragshierarchie-Links zeigen ausschließlich vom Kind zum Elternvertrag, nie umgekehrt.
ADR-8	Enforcement bezieht SHACL-Shapes und Validierungsprofil ausschließlich aus versionsgepinnten Semantic-Hub-Ständen.
ADR-9	Festlegung der SHACL-Engine für die semantische Validierung.
ADR-10	Der Klausel-Katalog wird als vorverdautes JSON zusammen mit dem rohen Turtle in einer Antwort transportiert.
ADR-11	ODRL-Constraints werden auf eingebettetem OPA evaluiert. Supersedet den Evaluator-Mechanismus aus ADR-6, dessen Enforcement-Entscheidung unverändert steht.
ADR-12	Die Vertragssignatur ist wallet-getrieben: Das DCS ist OpenID4VP-Relying-Party und Signaturvalidator und hält keinen Vertragssignaturschlüssel. Supersedet die Organisationssignatur-Klausel von ADR-3 und beschneidet ADR-1; die Akzeptanzpfad-Klauseln sind ihrerseits durch ADR-20 superseded.
ADR-13	DCS-to-DCS-Föderation ist ein Austausch von Vertrags-PDF und JAdES, kein State-Broadcast; jede Instanz führt Workflow und RBAC lokal. Supersedet den Synchronizer aus ADR-2; die dritte Vertrauensschicht ist durch ADR-19 superseded.
ADR-14	Intern soll genau eine JSON-LD-Form existieren, expandiert, mit Expansion nur an den Ingestion-Kanten. Als Entscheidung angenommen, nicht umgesetzt: Die ausgelieferte interne Form ist die kanonische <code> dcs: </code> -präfixierte Hülle (Kapitel 03).
ADR-15	Ein verhandelbarer Datenpunkt ist genau ein typisierter, SHACL-abgeleiteter, per <code>@id</code> verlinkter Knoten statt einer mehrstufigen Placeholder-Indirektion. Superseded durch ADR-22, das den Kern behält und den Knoten umbenennt.
ADR-16	Manipulationsnachweis des Audit-Trails entsteht über Merkle-Checkpoints je Verankerungs-Tick mit externer Verankerung. Verfeinert den event-sourced Audit-Trail, dessen Hash-Ketten je Ressource bestehen bleiben.
ADR-17	Eine Maschinen-Identität kann keine AES erzeugen: Die Klasse System Contract Signer erhält keinerlei Signatur-Scope. Um die Siegel-Option erweitert durch ADR-21.
ADR-18	Gegenüber dem Federated Catalogue tritt das DCS-Deployment als ein Participant mit einem Service-Account auf, nicht der einzelne Publisher. Ein entfernter, nicht mitdeployter Katalog erfordert eine explizite

ADR	These und Status
	Trust-Boundary-Bestätigung, sonst schlägt das Rendering fehl.
ADR-19	Föderationsvertrauen ist geschichtet: did:web-Identität mit Zertifikatskette, Regelakzeptanz per Agreement Credential und ein lokaler Policy-Endpunkt (fail-closed). Ersetzt die dritte Vertrauensschicht von ADR-13.
ADR-20	Härtung des Signatur-Akzeptanzpfads: Nonce-Bindung des Ceremony-Callbacks, Byte-Pinning des zu signierenden Dokuments, atomare Ceremony-Konsumption, level-bewusste Gates, Zertifikat-zu-Identität-Prüfung, JAdES-Validierung. Supersedet die Akzeptanzpfad-Klauseln von ADR-12.
ADR-21	System Contract Sealer: das von ADR-17 zurückgestellte elektronische Siegel über den generischen Verify-on-Reupload-Pfad statt einer Lockerung des AES-Verbots. Als Richtung akzeptiert, nicht implementiert.
ADR-22	Der typisierte Knoten ist ein Feld, kein Placeholder: <code>dcs:contractFields</code> trägt die Felddeklarationen, <code>dcs:contractData</code> typisierte Domänenobjekte mit Feldreferenzen. Supersedet ADR-15; durch ADR-23 amendiert.
ADR-23	Der Contract-Data-Graph ist generisch: beliebige SHACL-Vokabulare aus dem Semantic Hub, Properties als Literal, Feldreferenz oder Objektreferenz, In-Document-Auflösung und Validierung einer feld-materialisierten Kopie. Amendiert ADR-22, dessen Zwei-Ebenen-Form als Spezialfall gültig bleibt.
ADR-24	v1-Signatur-Scope: wallet-basierte AES; das erreichte Level bestimmt der angebundene Vertrauensdienst, nicht die Codebasis. QES und weitere Formate sind als kundenbestätigte Abweichungen dokumentiert; die technischen Konformitäts-Gates bleiben erhalten.
ADR-25	Zielsysteme sind eine persistierte, administrierte Registry, und jeder Vertrag designiert sein eigenes Deployment-Ziel; ein Vertrag ohne Designation deployt nicht. Die Callback-Authentifizierung ist durch ADR-27 superseded.
ADR-26	Die Signatur wird über der Provenance angebracht, und die Provenance danach über der Signatur neu verankert (provenance-only C2PA-Update-Manifest als inkrementelles Update); Signaturvalidität behält in jedem Konflikt Vorrang.
ADR-27	Maschinen-Credentials werden ausgestellt, nicht konfiguriert: Jeder maschinelle Aufrufer erhält einen eigenen, über den OAuth2-Provider provisionierten Client mit genau einmal angezeigtem Secret; die Deployment-Deklaration ist auf einen Seed reduziert. Supersedet die Callback-Authentifizierung aus ADR-25.
ADR-28	Jedes gespeicherte Artefakt ist unter einem zufälligen Schlüssel je Löschbereich verschlüsselt, der Schlüssel existiert im Ruhezustand nur HSM-gewickelt, und Art.-17-Löschung ist die protokollierte Vernichtung dieser Kopien auf beiden Föderationsinstanzen.
ADR-29	Die organisatorische Autorität einer Handlungsvollmacht ist an ihren Aussteller gebunden; die Rolle Integration Manager umfasst Integrationen, nicht Zugriff.
ADR-30	Ein Artefakt, das die authentifizierte Entschlüsselung nicht besteht, ist ein negatives Prüfergebnis, kein Serverfehler; das Verifikationsergebnis benennt seine Fehlerklasse direkt.
ADR-31	Ausstellervertrauen ist zweckgebunden (<code>login</code> / <code>peer</code> / <code>pid</code>), an Organisationen gebunden und über einen deklarierten Mechanismus aufgelöst; die Handlungsvollmacht hinter einer Signatur reist mit und wird von der Gegenseite geprüft.
ADR-32	Kontinuierliches Compliance-Monitoring läuft als geplanter Sweep über ein eigenes Kommando statt als Schleife im Server, mit einem Risiko-Register zur Deduplizierung der Alarmer.
ADR (OCM-W)	VC-Signierung läuft direkt über das HSM, nicht über die OCM-W-Dienste. Einziger unnummerierter ADR der Reihe.

Backend-interne Reihe

Ältere, backend-lokale Entscheidungen unter `docs/backend/en/decisions/`. Sie beschreiben Grundmuster, auf denen die Projektreihe aufsetzt.

ADR	These
0001	CQRS-Aufteilung je Fachdomäne (command/query/db/datatype/event).
0002	Design-first-API mit generiertem Transport.
0003	Event-sourced Audit-Trail mit Hash-Ketten je Ressource, verfeinert durch ADR-16.
0004	did:web-Identität mit eIDAS-Zertifikatskette als Vertrauensanker.
0005	Ein Schreiber je Vertrag (die Origin-Instanz); die Transportform ist durch ADR-13 abgelöst.
0006	Versionierungsstrategien für Vorlagen und Verträge.
0007	Optimistische Nebenläufigkeitskontrolle über den Änderungszeitstempel.